

# Important Information

Thank you for choosing Freenove products!

## Getting Started

First, please read the **Read Me First.pdf** document in the unzipped folder you created.

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.

## Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

[support@freenove.com](mailto:support@freenove.com)

## Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

**Any concerns?** ✉ [support@freenove.com](mailto:support@freenove.com)

## About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

<http://www.freenove.com>

[sale@freenove.com](mailto:sale@freenove.com)

## Copyright

All the files, materials and instructional guides provided are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.



Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

## Contents

|                                                        |    |
|--------------------------------------------------------|----|
| Important Information .....                            | 1  |
| Contents.....                                          | 1  |
| Preface.....                                           | 1  |
| ESP32-WROOM .....                                      | 3  |
| CH340 (Importance).....                                | 5  |
| Programming Software .....                             | 15 |
| Environment Configuration .....                        | 18 |
| Notes for GPIO.....                                    | 22 |
| Chapter 1 LED .....                                    | 23 |
| Project 1.1 Blink .....                                | 23 |
| Chapter 2 WS2812 .....                                 | 28 |
| Project 2.1 WS2812 .....                               | 28 |
| Chapter 3 Bluetooth.....                               | 33 |
| Project 3.1 Bluetooth Passthrough.....                 | 33 |
| Project 3.2 Bluetooth Low Energy Data Passthrough..... | 39 |
| Chapter 4 WiFi Working Modes.....                      | 52 |
| Project 4.1 Station mode .....                         | 52 |
| Project 4.2 AP mode .....                              | 56 |
| Project 4.3 AP+Station mode .....                      | 61 |
| Chapter 5 TCP/IP.....                                  | 65 |
| Project 5.1 As Client .....                            | 65 |
| Project 5.2 As Server .....                            | 77 |
| What's next? .....                                     | 82 |
| End of the Tutorial.....                               | 82 |

## Preface

ESP32 is a micro control unit with integrated Wi-Fi launched by Espressif, which features strong properties and integrates rich peripherals. It can be designed and studied as an ordinary Single Chip Microcontroller (SCM) chip, or connected to the Internet and used as an Internet of Things device.

ESP32 can be developed using the Arduino platform, which will definitely make it easier for people who have learned Arduino to master. Moreover, the code of ESP32 is completely open-source, so beginners can quickly learn how to develop and design IOT smart household products including smart curtains, fans, lamps and clocks.

Generally, ESP32 projects consist of code and circuits. Don't worry even if you've never learned code and circuits, because we will gradually introduce the basic knowledge of C programming language and electronic circuits, from easy to difficult. Our products contain all the electronic components and modules needed to complete these projects. It's especially suitable for beginners.

We divide each project into four parts, namely Component List, Component Knowledge, Circuit and Code. Component List helps you to prepare material for the experiment more quickly. Component Knowledge allows you to quickly understand new electronic modules or components, while Circuit helps you understand the operating principle of the circuit. And Code allows you to easily master the use of ESP32 and accessory kit. After finishing all the projects in this tutorial, you can also use these components and modules to make products such as smart household, smart cars and robots to transform your creative ideas into prototypes and new and innovative products.

In addition, if you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through [support@freenove.com](mailto:support@freenove.com)

## ESP32-WROOM

ESP32-WROOM has launched a total of two antenna packages, PCB on-board antenna and IPEX antenna respectively. The PCB on-board antenna is an integrated antenna in the chip module itself, so it is convenient to carry and design. The IPEX antenna is a metal antenna derived from the integrated antenna of the chip module itself, which is used to enhance the signal of the module.

PCB on-board antenna

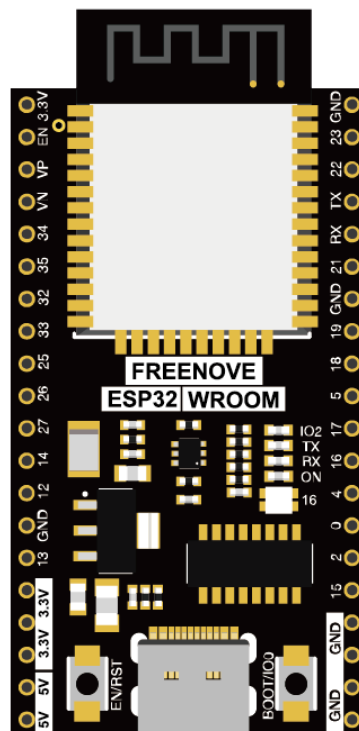


IPEX antenna

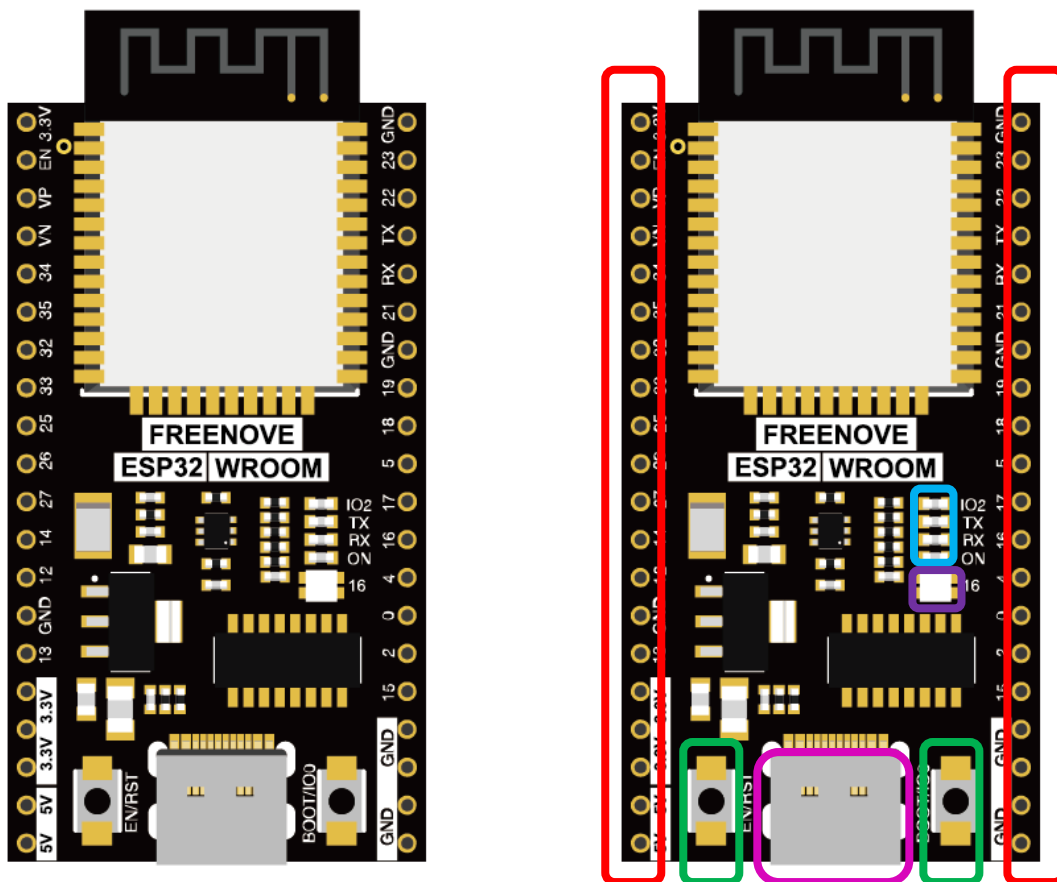


In this tutorial, the ESP32-WROOM is designed based on the PCB on-board antenna-packaged ESP32-WROOM-32E module.

ESP32-WROOM



The hardware interfaces of ESP32-WROOM are distributed as follows:



Compare the left and right images. We've boxed off the resources on the ESP32-WROOM in different colors to facilitate your understanding of the ESP32-WROOM.

| Box color                                                                           | Corresponding resources introduction     |
|-------------------------------------------------------------------------------------|------------------------------------------|
|  | GPIO pin                                 |
|  | LED indicator                            |
|  | WS2812                                   |
|  | Reset button, Boot mode selection button |
|  | Type C port                              |

For more information, please visit: [https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32e\\_esp32-wroom-32ue\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf)

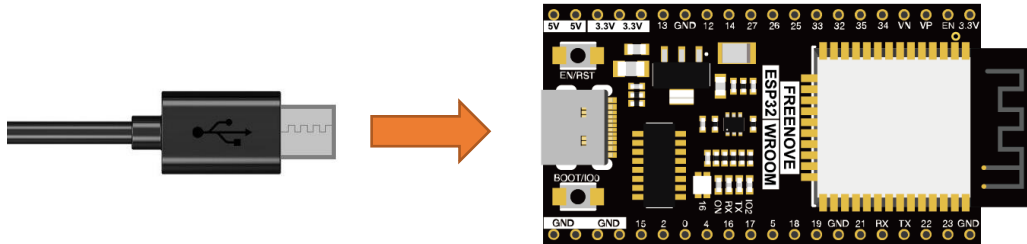
## CH340 (Importance)

ESP32 uses CH340 to download codes. So before using it, we need to install CH340 driver in our computers.

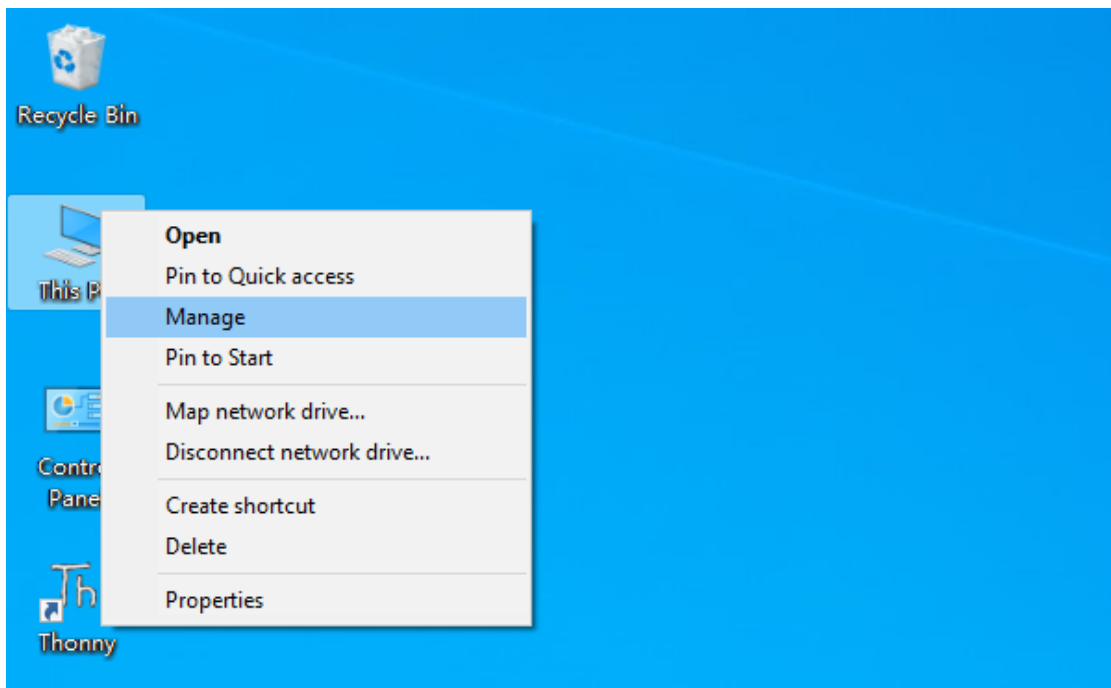
### Windows

Check whether CH340 has been installed

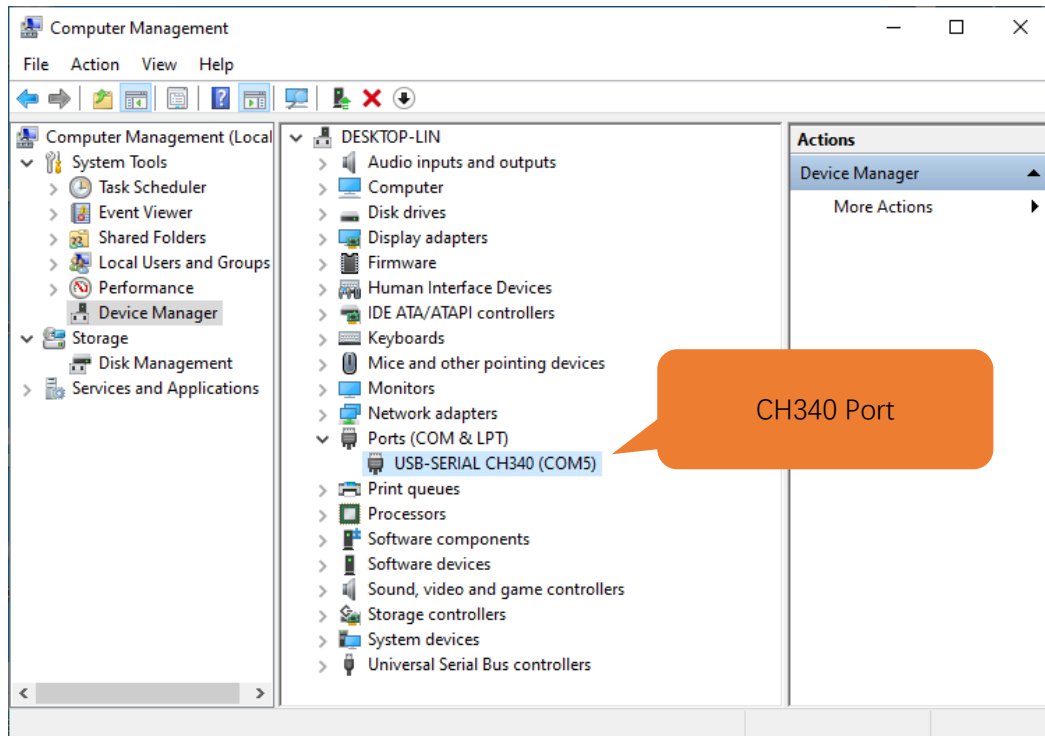
1. Connect your computer and ESP32 with a USB cable.



2. Turn to the main interface of your computer, select "This PC" and right-click to select "Manage".



- Click “Device Manager”. If your computer has installed CH340, you can see “USB-SERIAL CH340 (COMx)”. And you can click [here](#) to move to the next step.



#### Installing CH340

- First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.



index / search / search CH340

All (14)

Downloads (7)

Products (4)

Application (2)

Video (1)

News (0)

### keyword CH340

Downloads(7)

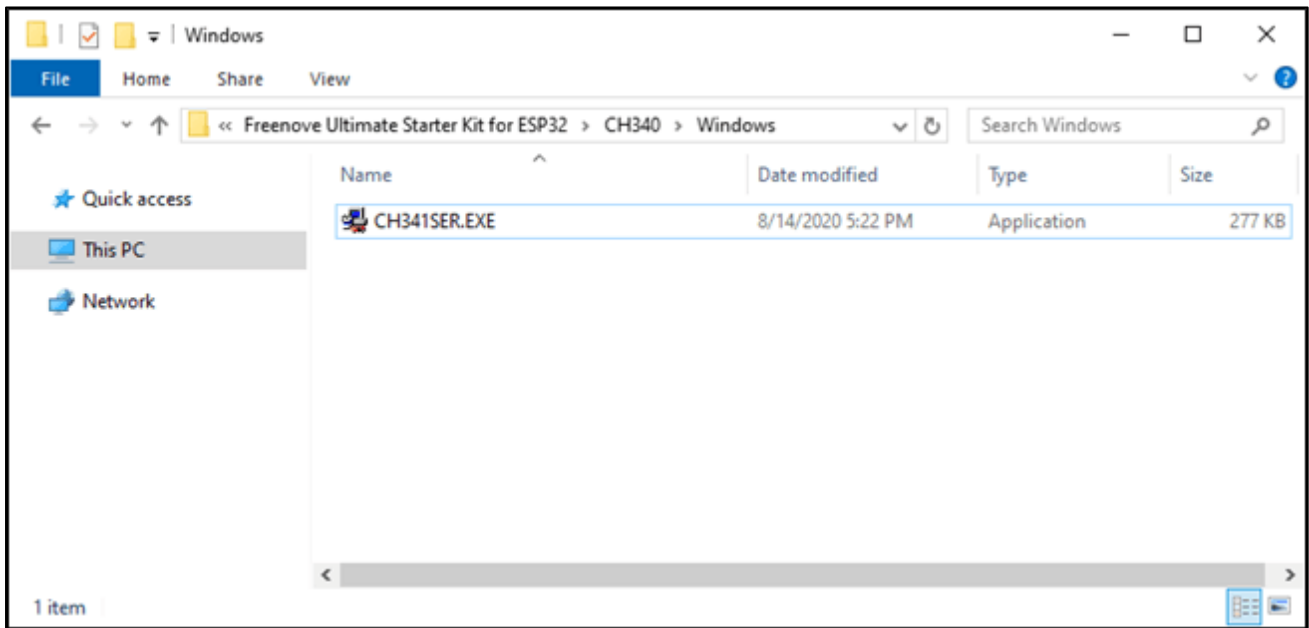
| file category       | file content                                                                                                                                                                                                                                                                                    | version | upload time |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------------|
| Driver&Tools        |                                                                                                                                                                                                                                                                                                 |         |             |
| CH341SER.EXE        | CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98                                                                                                                                                           | 3.5     | 2019-03-18  |
| CH341SER.ZIP        | CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98                                                                  | 3.5     | 2019-03-05  |
| CH341SER_ANDROID... | CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library, file (Host Driver), App Demo Example (USB to UART Demo) | 1.6     | 2019-04-19  |
| CH341SER_LINUX...   | CH340/CH341 USB to serial port LINUX driver                                                                                                                                                                                                                                                     | 1.5     | 2018-03-18  |
| CH341SER_MAC.ZIP    | CH340/CH341 USB to serial port MAC OS driver                                                                                                                                                                                                                                                    | 1.5     | 2018-07-05  |
| Others              |                                                                                                                                                                                                                                                                                                 |         |             |
| PRODUCT_GUIDE.P...  | Electronic selection of product selection manual, please refer to related product technical manual for more technical information.                                                                                                                                                              | 1.4     | 2018-12-29  |
| InstallNoteOn64...  | Instructions for the driver after 18 years of August cannot be installed under some 64-bit WIN7 (English)                                                                                                                                                                                       | 1.0     | 2019-01-10  |

If you would not like to download the installation package, you can open

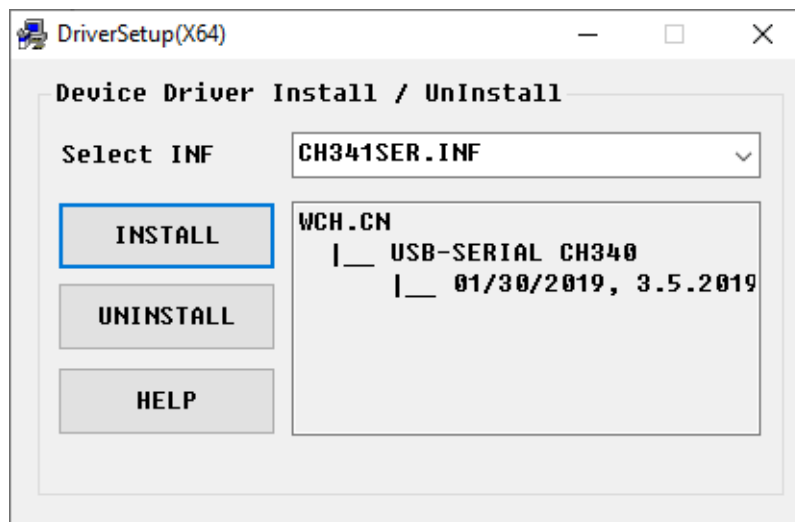
“Freenove\_ESP32\_WROOM\_Board/CH340”, we have prepared the installation package.

| Name    | Date modified     | Type        | Size |
|---------|-------------------|-------------|------|
| Linux   | 8/14/2020 5:24 PM | File folder |      |
| MAC     | 8/14/2020 5:23 PM | File folder |      |
| Windows | 8/14/2020 5:23 PM | File folder |      |

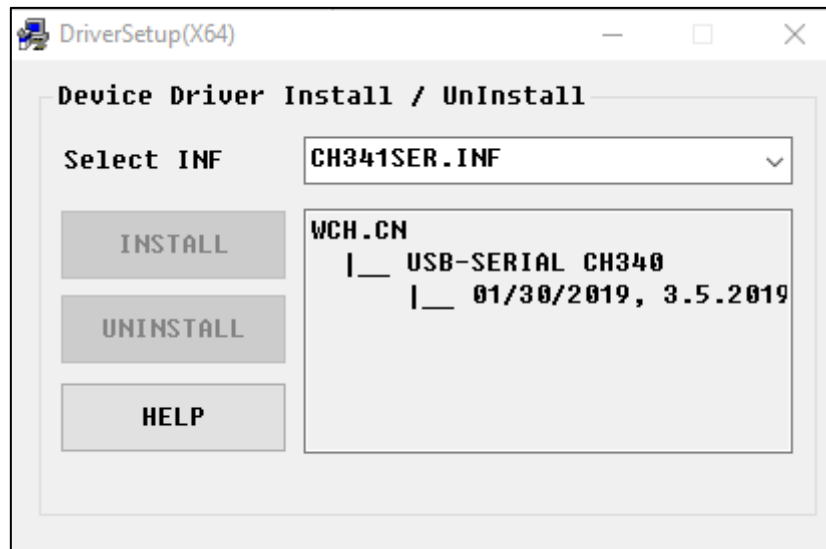
2. Open the folder "Freenove\_ESP32\_WROOM\_Board/CH340/Windows/"



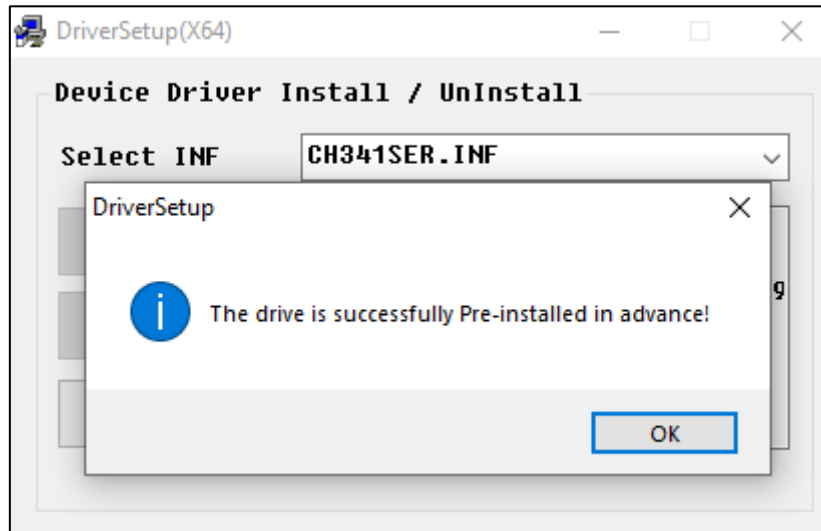
3. Double click "CH341SER.EXE".



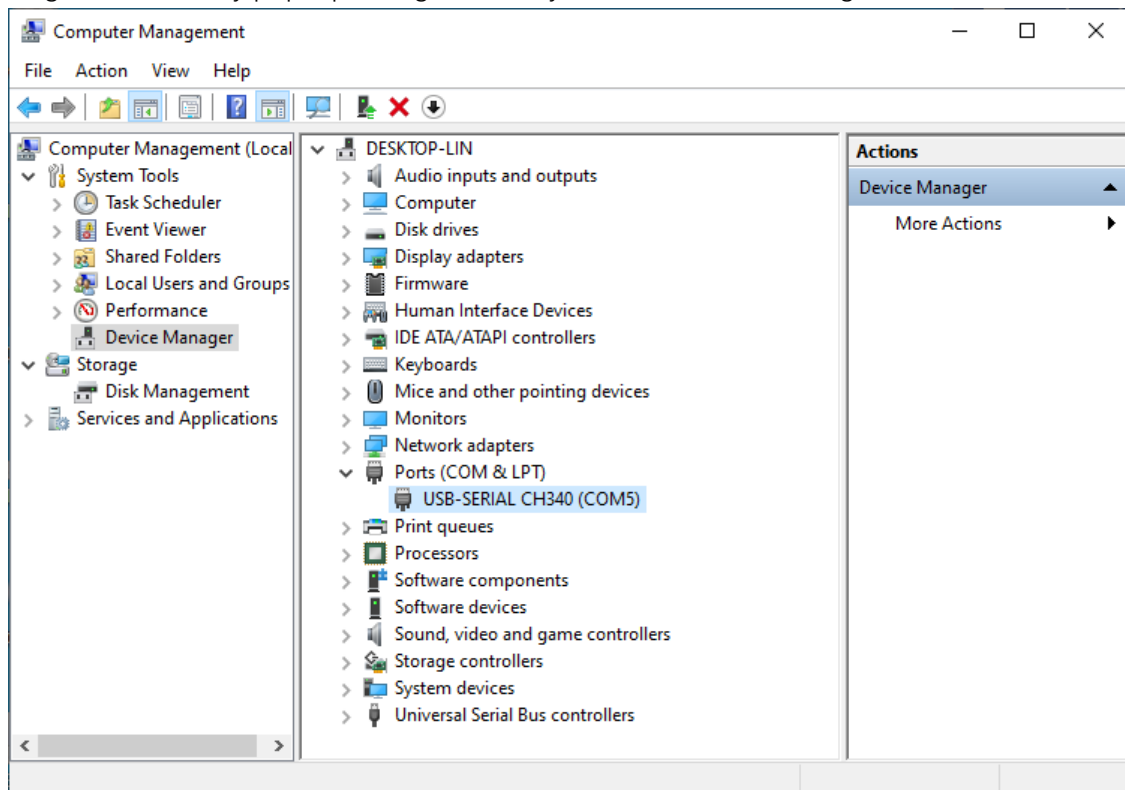
4. Click "INSTALL" and wait for the installation to complete.



5. Install successfully. Close all interfaces.



6. When ESP32 is connected to computer, select “This PC”, right-click to select “Manage” and click “Device Manager” in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH340 has been installed successfully. Close all dialog boxes.

## MAC

First, download CH340 driver, click <http://www.wch-ic.com/search?q=CH340&t=downloads> to download the appropriate one based on your operating system.

The screenshot shows the WCH website with a search bar at the top right containing the text 'ch340'. On the left, there is a sidebar with navigation links: All (14), Downloads (7), Products (4), Application (2), Video (1), and News (0). The main content area displays a table of search results under the heading 'keyword ch340' and 'Downloads( 7 )'.

| file category       | file content                                                                                                                                                                                                                                                                               | version | upload time |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------------|
| Driver&Tools        |                                                                                                                                                                                                                                                                                            |         |             |
| CH341SER.EXE        | CH340/CH341 USB to serial port Windows driver, supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98                                                                                                                                                      | 3.5     | 2019-03-18  |
| CH341SER.ZIP        | CH340/CH341 USB to serial port Windows driver, includes DLL dynamic library and non-standard baud rate settings and other instructions. Supports 32/64-bit Windows 10/8.1/8/7/VISTA/XP, Server 2016/2012/2008/2003, 2000/ME/98                                                             | 3.5     | 2019-03-05  |
| CH341SER_ANDROID... | CH340/CH341 USB to serial port Android free drive application library, for Android OS 3.1 and above version which supports USB Host mode already, no need to load Android kernel driver, no root privileges. Contains apk, lib library file (Java Driver), App Demo Example, AT Demo SDK). | 1.6     | 2019-04-19  |
| CH341SER_LINUX...   | CH340/CH341 USB to serial port LINUX driver                                                                                                                                                                                                                                                | 1.5     | 2018-03-18  |
| CH341SER_MAC.ZI...  | CH340/CH341 USB to serial port MAC OS driver                                                                                                                                                                                                                                               | 1.5     | 2018-07-05  |

Orange callout boxes highlight the following files:

- Windows**: CH341SER.EXE
- Linux**: CH341SER\_LINUX...
- MAC**: CH341SER\_MAC.ZI...

If you would not like to download the installation package, you can open “Freenove\_ESP32\_WROOM\_Board/CH340”, we have prepared the installation package. Second, open the folder “Freenove\_ESP32\_WROOM\_Board/CH340/MAC/”

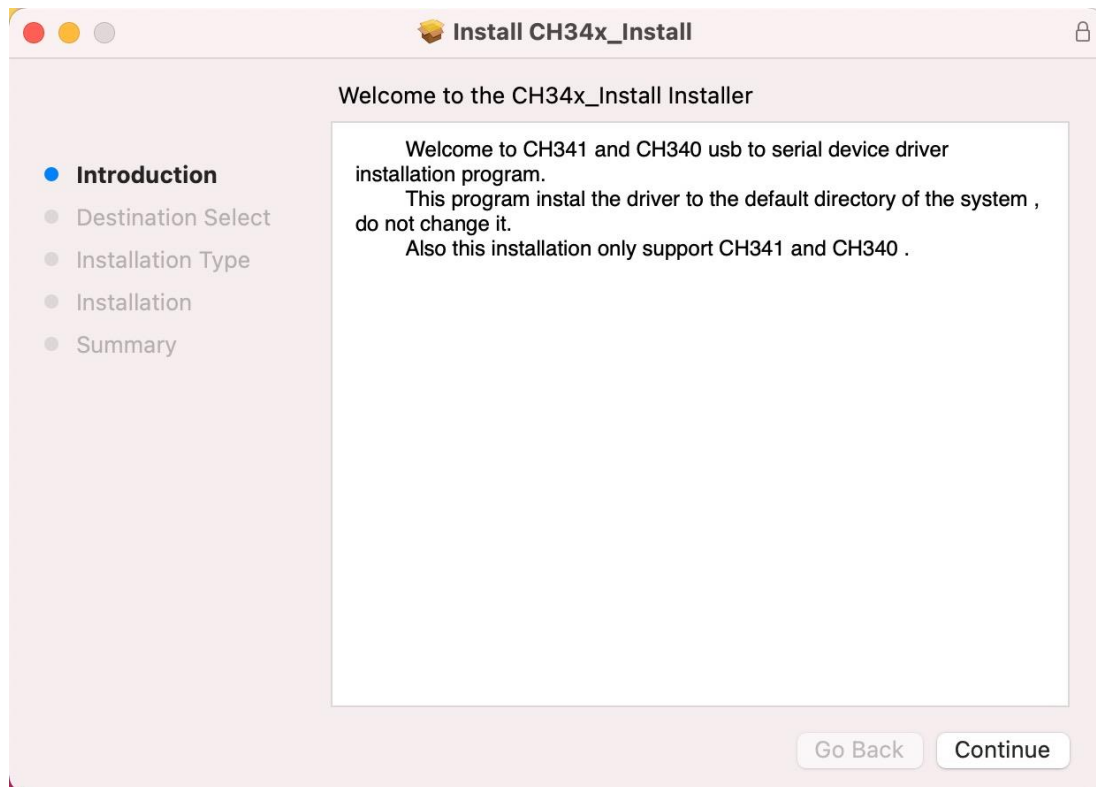
The screenshot shows a macOS file explorer window titled 'MAC'. The left sidebar shows the 'Favorites' section with links to AirDrop, Recents, Applications, Desktop, Documents, and Downloads. The main pane displays a list of files:

| Name                   | Modified               | Size   | Kind              |
|------------------------|------------------------|--------|-------------------|
| CH34x_Install_V1.5.pkg | Dec 8, 2016 at 2:00 PM | 26 KB  | Installer package |
| ReadMe.pdf             | Dec 8, 2016 at 4:09 PM | 146 KB | Adobe...ocument   |

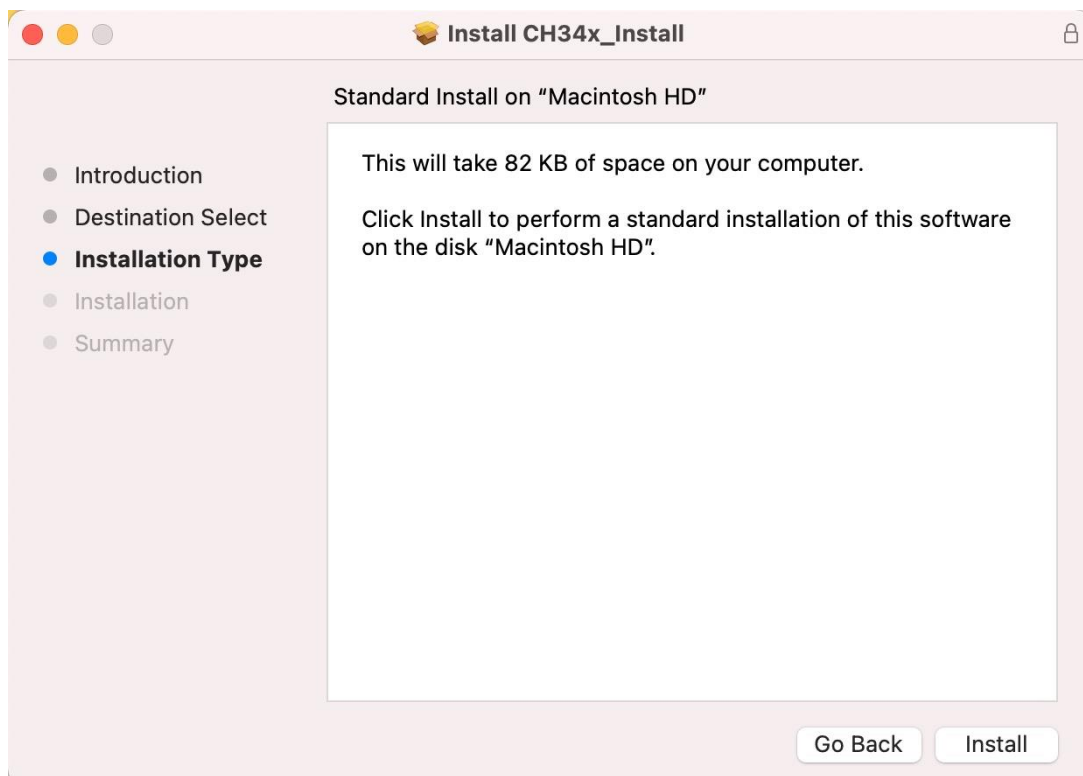
An orange callout box points to the 'CH34x\_Install\_V1.5.pkg' file with the text 'Run it.'

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

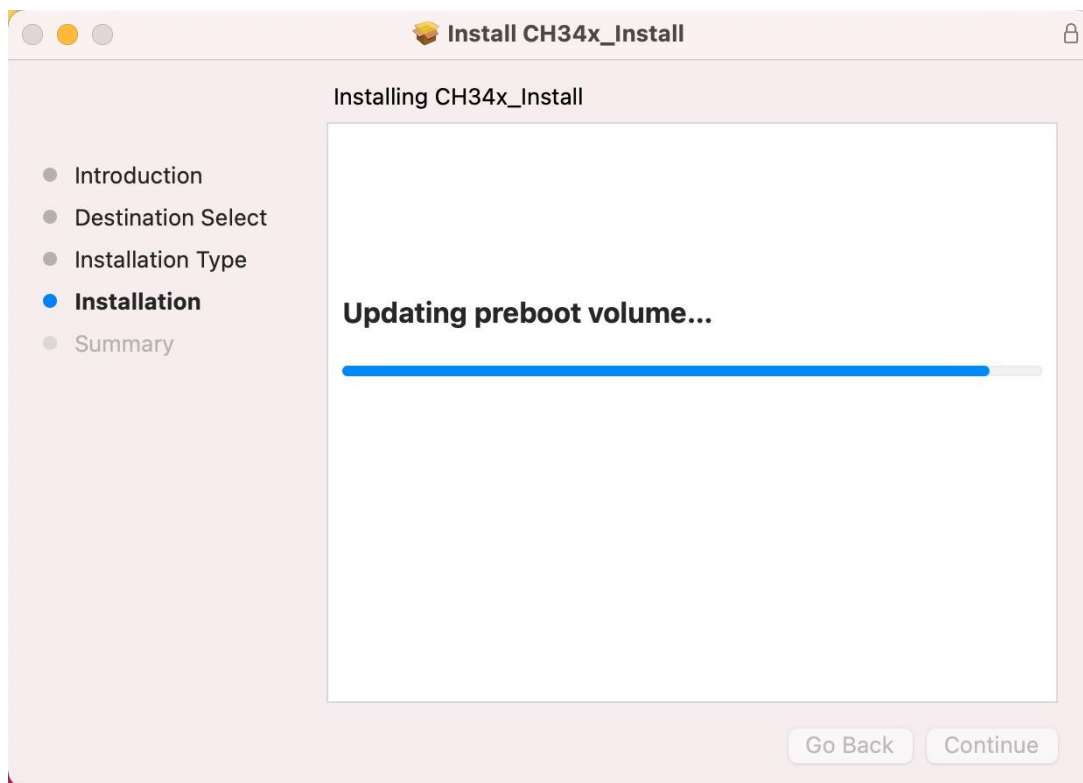
Third, click Continue.



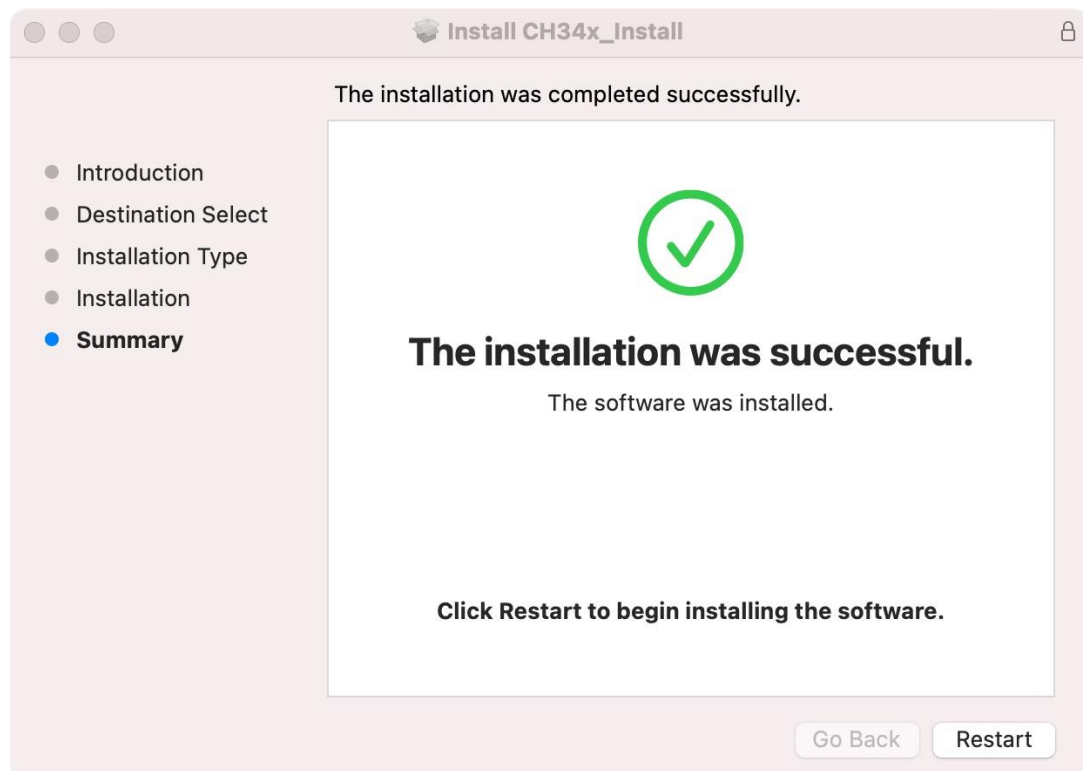
Fourth, click Install.



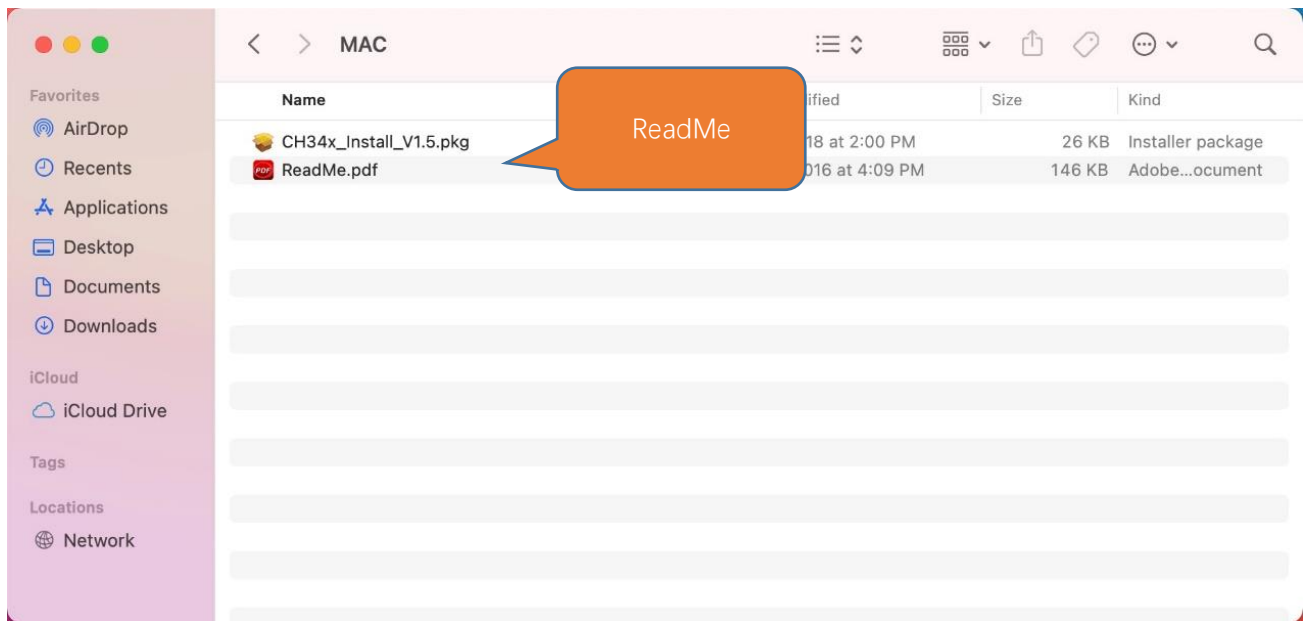
Then, waiting Finish.



Finally, restart your PC.



If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.



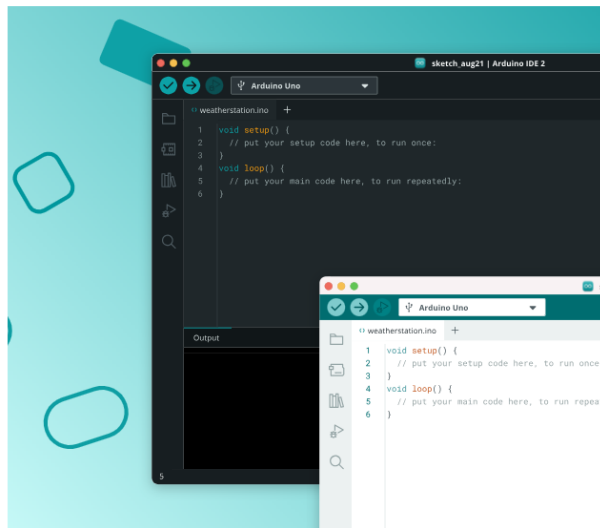


## Programming Software

Arduino Software (IDE) is used to write and upload the code for Arduino Board.

First, install Arduino Software (IDE): visit <https://www.arduino.cc/en/software/>

### Bring Your Projects to Life with Arduino Software



#### Arduino IDE 2.3.6

##### Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD



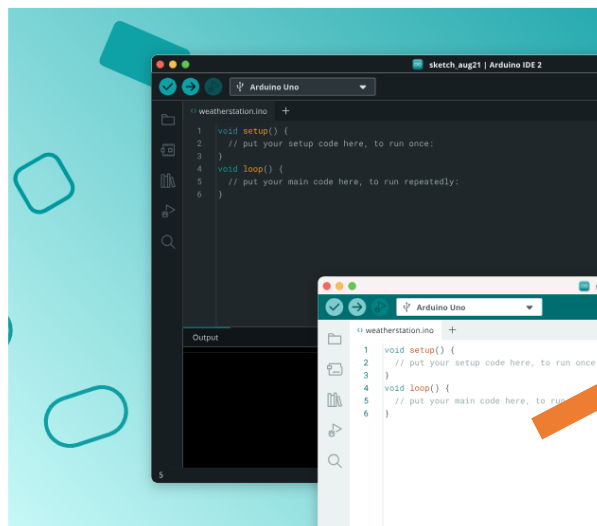
#### Nightly Builds

Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Select and download corresponding installer based on your operating system. If you are a Windows user, please select the "Windows" to download and install the driver correctly.

### Bring Your Projects to Life with Arduino Software



#### Arduino IDE 2.3.6

##### Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD

Windows Win 10 or newer (64-bit)

Windows MSI installer

Windows ZIP file

Linux Appliance (64-bit X86-64)

Linux ZIP file (64-bit X86-64)

macOS Intel 10.15 Catalina or newer (64-bit)

macOS Apple Silicon 11 Big Sur or newer (64-bit)



#### Legacy IDE (1.8.19)

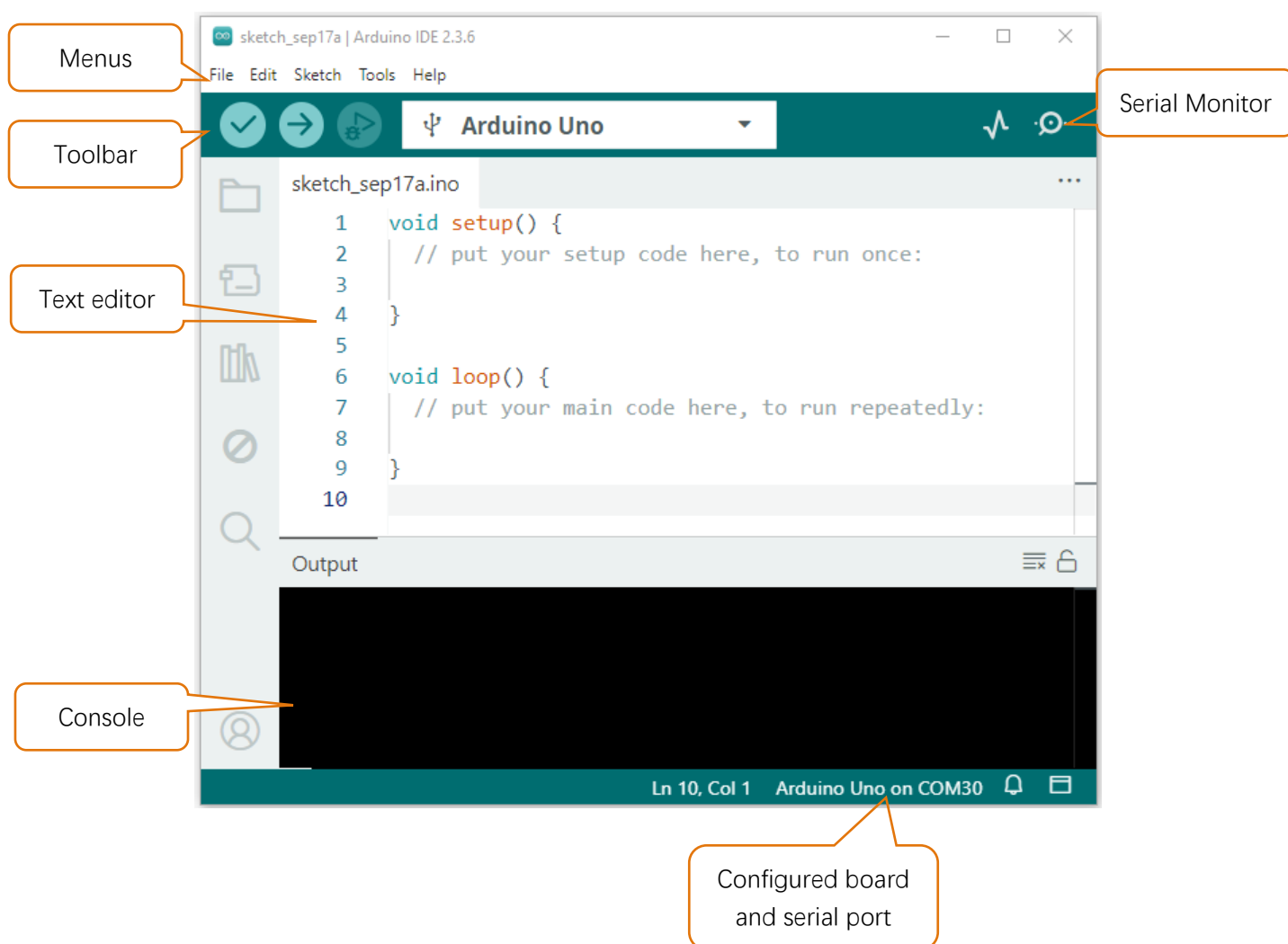
Download a legacy version of the Arduino IDE.

Any concerns? [support@freenove.com](mailto:support@freenove.com)

After the downloading completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it is popped up, please allow the installation. After installation is completed, an shortcut will be generated in the desktop.



Run it. The interface of the software is as follows:

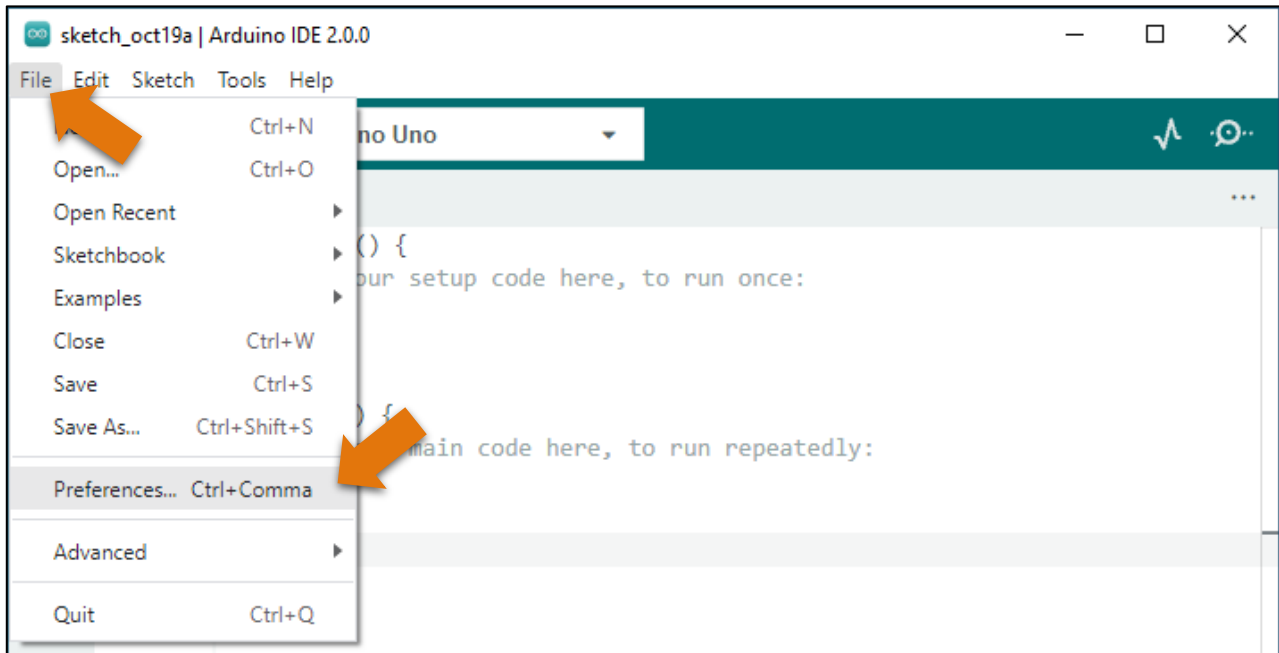


Programs written with Arduino IDE are called sketches. These sketches are written in a text editor and are saved with the file extension.ino. The editor has features for cutting/pasting and for searching/replacing text. The console displays text output by the Arduino IDE, including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, open the serial monitor, and access the serial plotter.

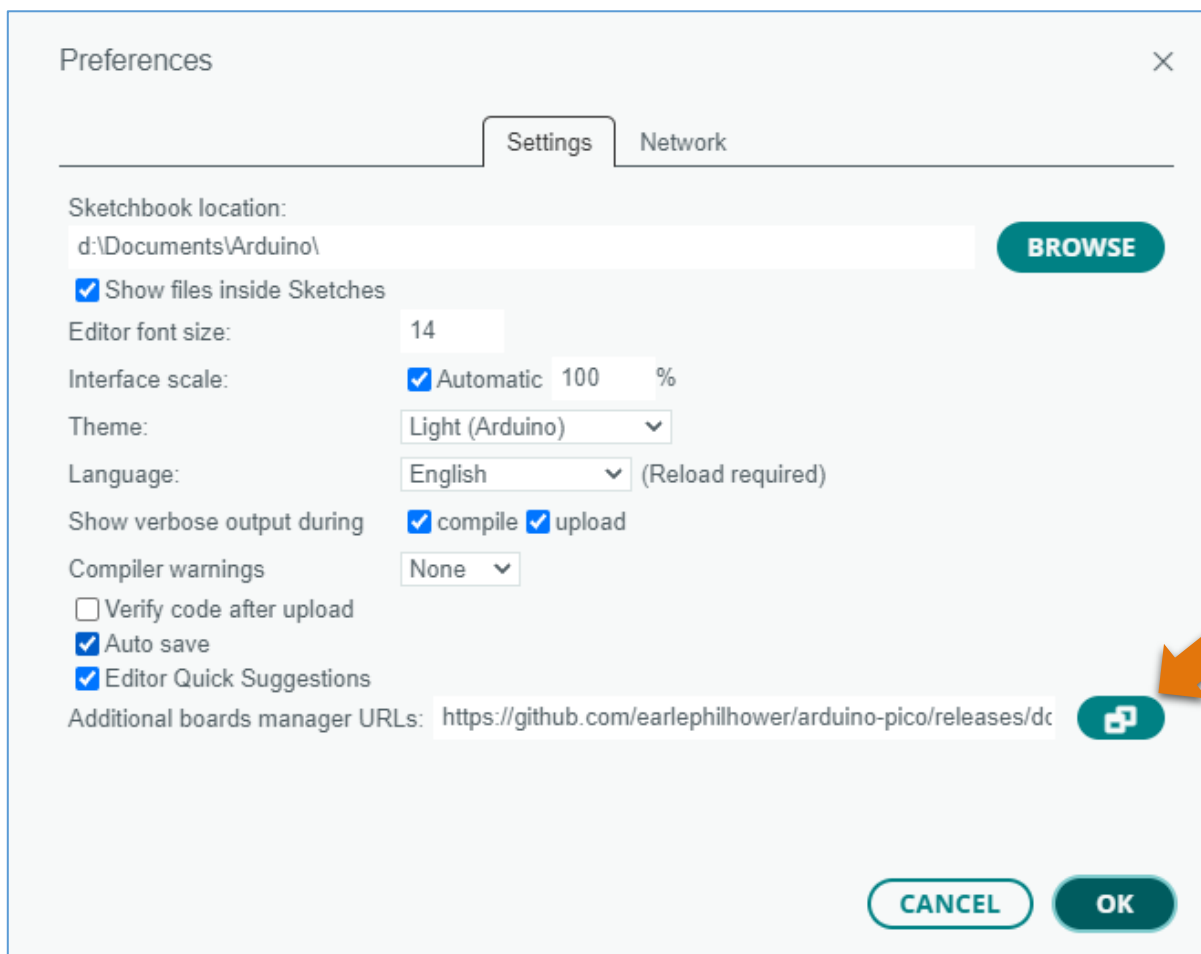
|                                                                                   |                                                                                              |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
|  | <b>Verify</b><br>Checks your code for errors compiling it.                                   |
|  | <b>Upload</b><br>Compiles your code and uploads it to the configured board.                  |
|  | <b>Debug</b><br>Troubleshoot code errors and monitor program running status.                 |
|  | <b>Serial Plotter</b><br>Real-time plotting of serial port data charts.                      |
|  | <b>Serial Monitor</b><br>Used for debugging and communication between devices and computers. |

## Environment Configuration

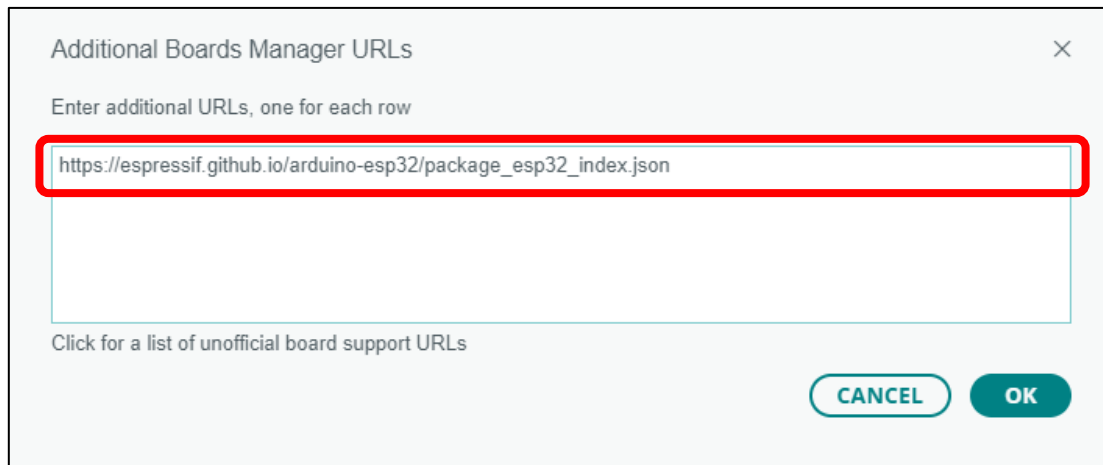
First, open the software platform arduino, and then click File in Menus and select Preferences.



Second, click on the symbol behind "Additional Boards Manager URLs"

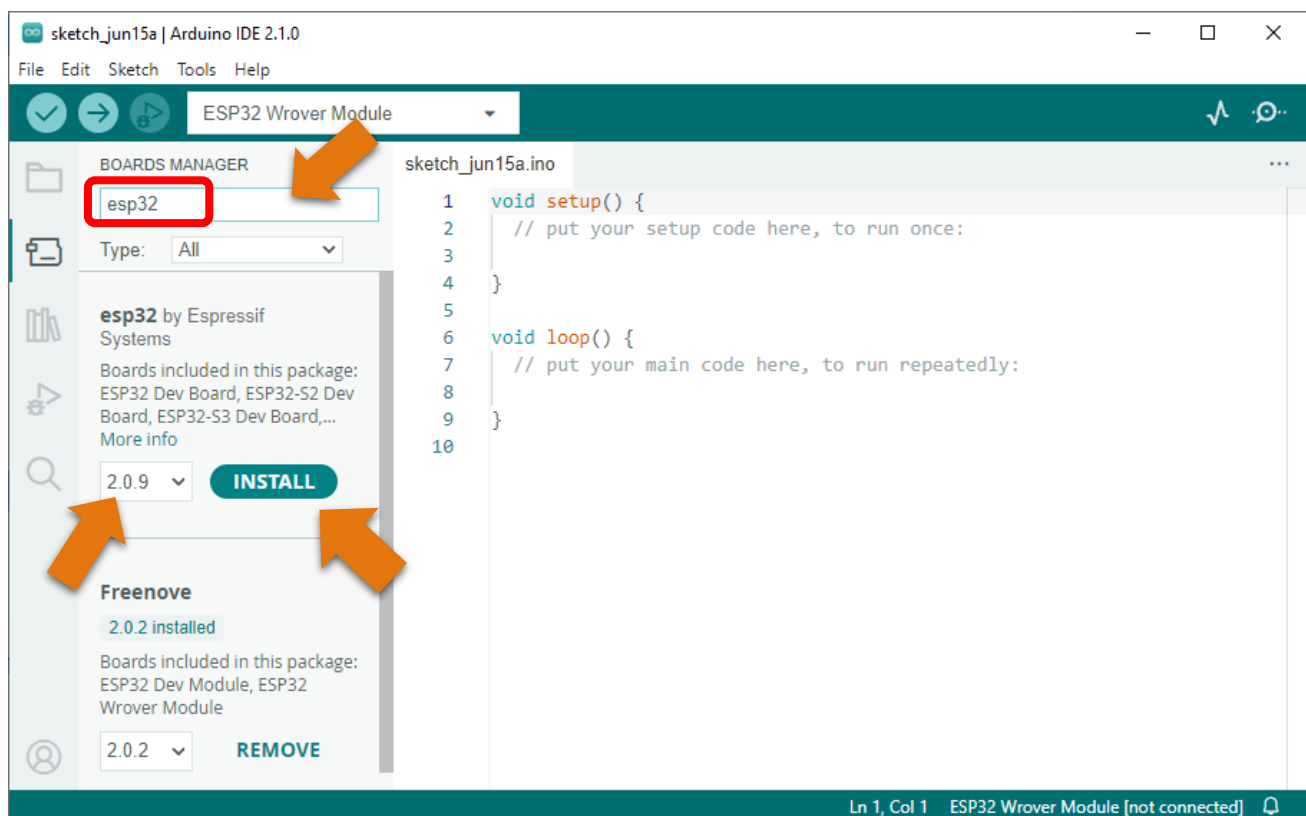


Third, fill in [https://espressif.github.io/arduino-esp32/package\\_esp32\\_index.json](https://espressif.github.io/arduino-esp32/package_esp32_index.json) in the new window, click OK, and click OK on the Preferences window again.



**Note:** if you copy and paste the URL directly, you may lose the "-". Please check carefully to make sure the link is correct.

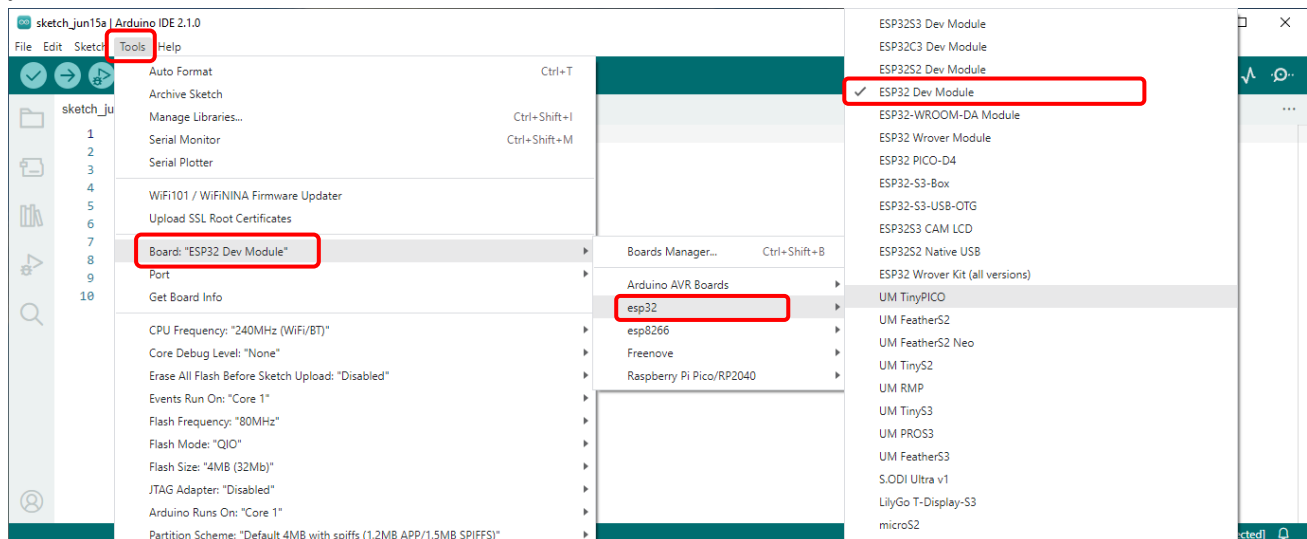
Fourth, click "Boards Manager". Enter "esp32" in Boards manager, select 2.0.9, and click "INSTALL".



Arduino will download these files automatically. Wait for the installation to complete.



When finishing installation, click Tools in the Menus again and select Board: "ESP32 Dev Module", and then you can see information of ESP32.



Keep the default Settings.

|                                                                       |              |
|-----------------------------------------------------------------------|--------------|
| Auto Format                                                           | Ctrl+T       |
| Archive Sketch                                                        |              |
| Manage Libraries...                                                   | Ctrl+Shift+I |
| Serial Monitor                                                        | Ctrl+Shift+M |
| Serial Plotter                                                        |              |
| WiFi101 / WiFiNINA Firmware Updater                                   |              |
| Upload SSL Root Certificates                                          |              |
| Board: "ESP32 Dev Module"                                             | ▶            |
| Port: "COM27"                                                         | ▶            |
| Get Board Info                                                        |              |
| CPU Frequency: "240MHz (WiFi/BT)"                                     | ▶            |
| Core Debug Level: "None"                                              | ▶            |
| Erase All Flash Before Sketch Upload: "Disabled"                      | ▶            |
| Events Run On: "Core 1"                                               | ▶            |
| Flash Frequency: "80MHz"                                              | ▶            |
| Flash Mode: "QIO"                                                     | ▶            |
| Flash Size: "4MB (32Mb)"                                              | ▶            |
| JTAG Adapter: "Disabled"                                              | ▶            |
| Arduino Runs On: "Core 1"                                             | ▶            |
| Partition Scheme: "Default 4MB with spiiffs (1.2MB APP/1.5MB SPIFFS)" | ▶            |
| PSRAM: "Disabled"                                                     | ▶            |
| Upload Speed: "921600"                                                | ▶            |
| Programmer                                                            | ▶            |
| Burn Bootloader                                                       |              |

## Notes for GPIO

### Strapping Pin

There are five Strapping pins for ESP32: MTDI、GPIO0、GPIO2、MTDO、GPIO5。

With the release of the chip's system reset (power-on reset, RTC watchdog reset, undervoltage reset), the strapping pins sample the level and store it in the latch as "0" or "1" , and keep it until the chip is powered off or turned off.

Each Strapping pin is connecting to internal pull-up/pull-down. Connecting to high-impedance external circuit or without an external connection, a strapping pin's default value of input level will be determined by internal weak pull-up/pull-down. To change the value of the Strapping, users can apply an external pull-down/pull-up resistor, or use the GPIO of the host MCU to control the level of the strapping pin when the ESP32's power on reset is released.

**When releasing the reset, the strapping pin has the same function as a normal pin.**

The followings are default configurations of these five strapping pins at power-on and their functions under the corresponding configuration.

| Voltage of Internal LDO (VDD_SDIO)                                |           |                                              |                                             |                                             |                                            |
|-------------------------------------------------------------------|-----------|----------------------------------------------|---------------------------------------------|---------------------------------------------|--------------------------------------------|
| Pin                                                               | Default   | 3.3 V                                        |                                             | 1.8 V                                       |                                            |
| MTDI                                                              | Pull-down | 0                                            |                                             | 1                                           |                                            |
| Bootling Mode                                                     |           |                                              |                                             |                                             |                                            |
| Pin                                                               | Default   | SPI Boot                                     |                                             | Download Boot                               |                                            |
| GPIO0                                                             | Pull-up   | 1                                            |                                             | 0                                           |                                            |
| GPIO2                                                             | Pull-down | Don't-care                                   |                                             | 0                                           |                                            |
| Enabling/Disabling Debugging Log Print over U0TXD During Bootling |           |                                              |                                             |                                             |                                            |
| Pin                                                               | Default   | U0TXD Active                                 |                                             | U0TXD Silent                                |                                            |
| MTDO                                                              | Pull-up   | 1                                            |                                             | 0                                           |                                            |
| Timing of SDIO Slave                                              |           |                                              |                                             |                                             |                                            |
| Pin                                                               | Default   | Falling-edge Sampling<br>Falling-edge Output | Falling-edge Sampling<br>Rising-edge Output | Rising-edge Sampling<br>Falling-edge Output | Rising-edge Sampling<br>Rising-edge Output |
| MTDO                                                              | Pull-up   | 0                                            | 0                                           | 1                                           | 1                                          |
| GPIO5                                                             | Pull-up   | 0                                            | 1                                           | 0                                           | 1                                          |

**Note:**

- Firmware can configure register bits to change the settings of "Voltage of Internal LDO (VDD\_SDIO)" and "Timing of SDIO Slave" after bootling.
- The MTDI is internally pulled high in the module, as the flash and SRAM in ESP32-WROVER only support a power voltage of 1.8 V (output by VDD\_SDIO).

If you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through [support@freenove.com](mailto:support@freenove.com) at any time.



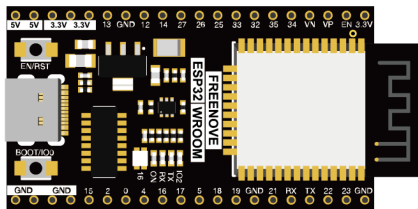
# Chapter 1 LED

This chapter is the Start Point in the journey to build and explore ESP32 electronic projects. We will start with simple “Blink” project.

## Project 1.1 Blink

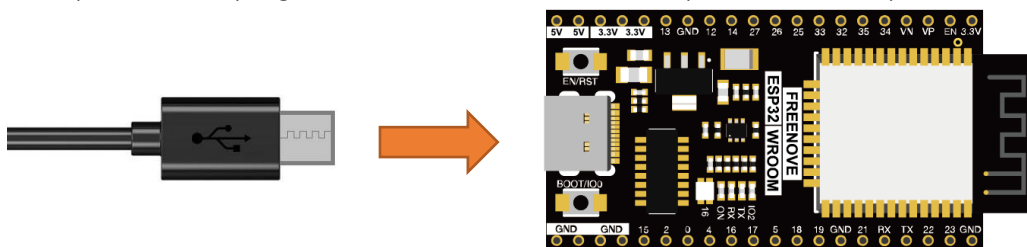
In this project, we will use ESP32 to control blinking a common LED.

### Component List

|                                                                                                      |                                                                                                 |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| ESP32-WROOM x1<br> | USB cable<br> |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|

#### Power

ESP32-WROOM needs 5v power supply. In this tutorial, we need connect ESP32-WROOM to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



In the following projects, we only use USB cable to power ESP32-WROOM by default.

## Sketch

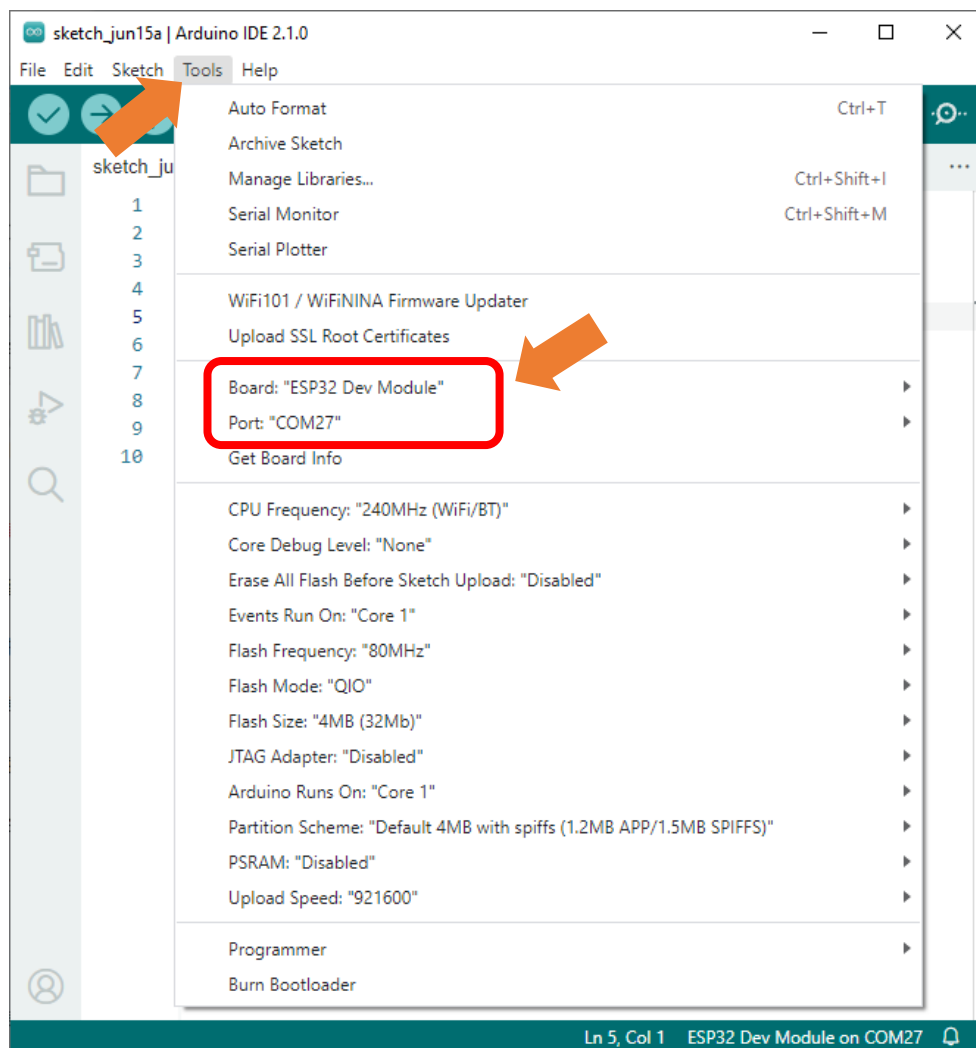
According to the circuit, when the GPIO2 of ESP32-WROOM output level is high, the LED turns ON. Conversely, when the GPIO2 ESP32-WROOM output level is low, the LED turns OFF. Therefore, we can let GPIO2 circularly output high and low level to make the LED blink.

Upload the following Sketch: s

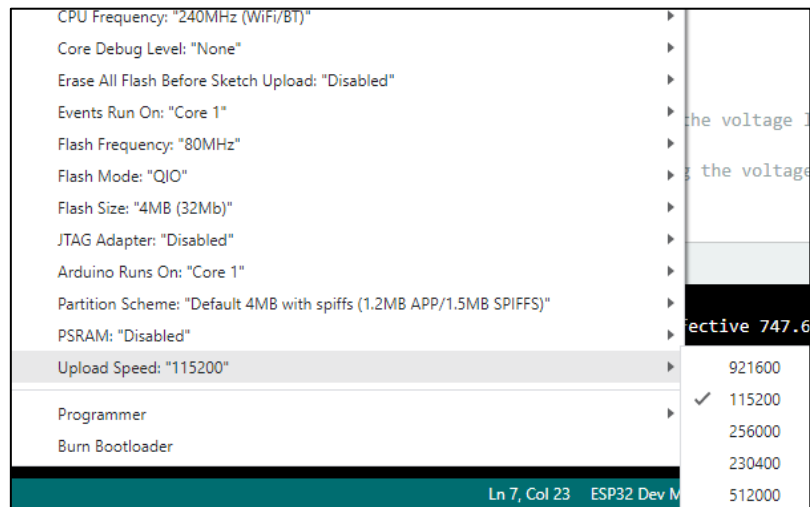
**Freenove\_ESP32\_WROOM\_Board\Sketches\Sketch\_01.1\_Blink.**

Before uploading the code, click "Tools", "Board" and select "ESP32 Dev Module".

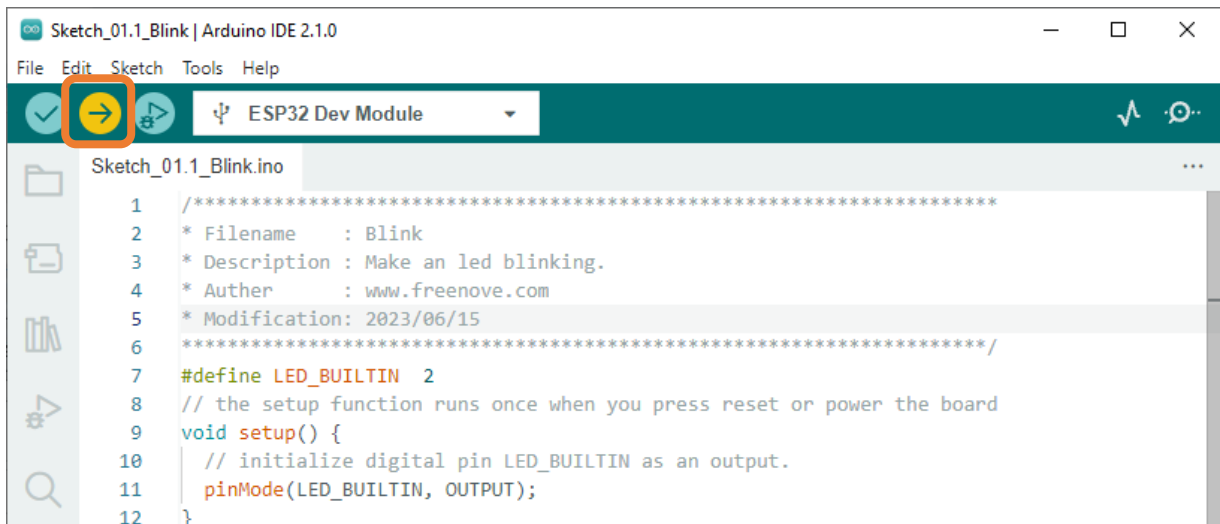
Select the serial port.



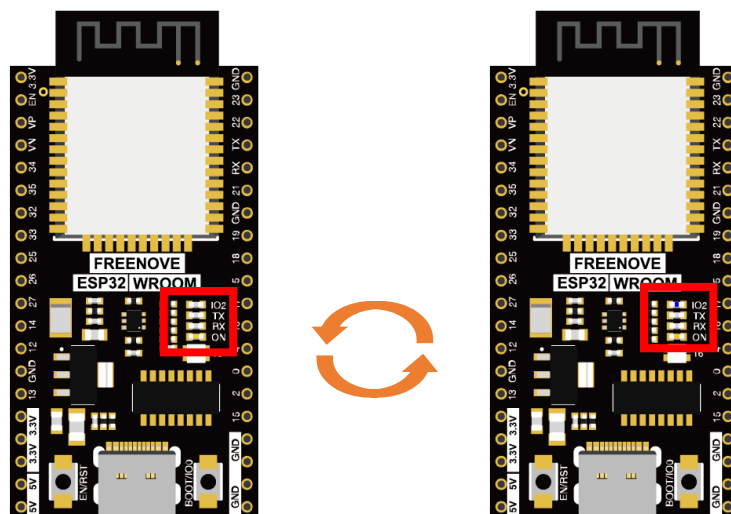
**Note:** For macOS users, if the uploading fails, please set the baud rate to 115200 before clicking “Upload Using Programmer”.



### Sketch\_01.1\_Blink



Click “Upload”, Download the code to ESP32-WROOM and your LED in the circuit starts Blink.



If you have any concerns, please contact us via: [support@freenove.com](mailto:support@freenove.com)

Any concerns? ✉ [support@freenove.com](mailto:support@freenove.com)

The following is the program code:

```

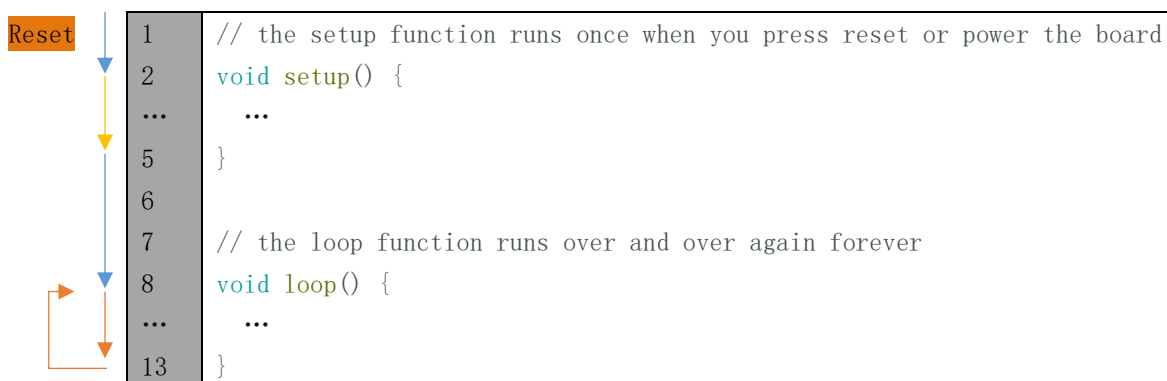
1  #define PIN_LED 2
2  // the setup function runs once when you press reset or power the board
3  void setup() {
4      // initialize digital pin LED_BUILTIN as an output.
5      pinMode(PIN_LED, OUTPUT);
6  }
7
8  // the loop function runs over and over again forever
9  void loop() {
10     digitalWrite(PIN_LED, HIGH); // turn the LED on (HIGH is the voltage level)
11     delay(1000);                 // wait for a second
12     digitalWrite(PIN_LED, LOW);  // turn the LED off by making the voltage LOW
13     delay(1000);                 // wait for a second
14 }

```

The Arduino IDE code usually contains two basic functions: void setup() and void loop().

After the board is reset, the setup() function will be executed firstly, and then the loop() function.

setup() function is generally used to write code to initialize the hardware. And loop() function is used to write code to achieve certain functions. loop() function is executed repeatedly. When the execution reaches the end of loop(), it will jump to the beginning of loop() to run again.



#### Reset

Reset operation will lead the code to be executed from the beginning. Switching on the power, finishing uploading the code and pressing the reset button will trigger reset operation.

In the circuit, ESP32-WROOM's GPIO2 is connected to the LED, so the LED pin is defined as 2.

```

1  #define PIN_LED 2

```

This means that after this line of code, all PIN\_LED will be treated as 2.

In the setup () function, first, we set the PIN\_LED as output mode, which can make the port output high level or low level.

```

4  // initialize digital pin PIN_LED as an output.
5  pinMode(PIN_LED, OUTPUT);

```

Then, in the loop () function, set the PIN\_LED to output high level to make LED light up.

```

10 digitalWrite(PIN_LED, HIGH); // turn the LED on (HIGH is the voltage level)

```

Wait for 1000ms, that is 1s. Delay () function is used to make control board wait for a moment before executing the next statement. The parameter indicates the number of milliseconds to wait for.

|    |                           |                                   |
|----|---------------------------|-----------------------------------|
| 11 | <code>delay(1000);</code> | <code>// wait for a second</code> |
|----|---------------------------|-----------------------------------|

Then set the PIN\_LED to output low level, and LED light off. One second later, the execution of loop () function will be completed.

|    |                                          |                                                            |
|----|------------------------------------------|------------------------------------------------------------|
| 12 | <code>digitalWrite(PIN_LED, LOW);</code> | <code>// turn the LED off by making the voltage LOW</code> |
| 13 | <code>delay(1000);</code>                | <code>// wait for a second</code>                          |

The loop() function is constantly being executed, so LED will keep blinking.

## Reference

|                                               |
|-----------------------------------------------|
| <code>void pinMode(int pin, int mode);</code> |
|-----------------------------------------------|

Configures the specified pin to behave either as an input or an output.

### Parameters

pin: the pin number to set the mode of.

mode: INPUT, OUTPUT, INPUT\_PULLDOWN, or INPUT\_PULLUP.

|                                                      |
|------------------------------------------------------|
| <code>void digitalWrite (int pin, int value);</code> |
|------------------------------------------------------|

Writes the value HIGH or LOW (1 or 0) to the given pin which must have been previously set as an output.

For more related functions, please refer to <https://www.arduino.cc/reference/en/>

## Chapter 2 WS2812

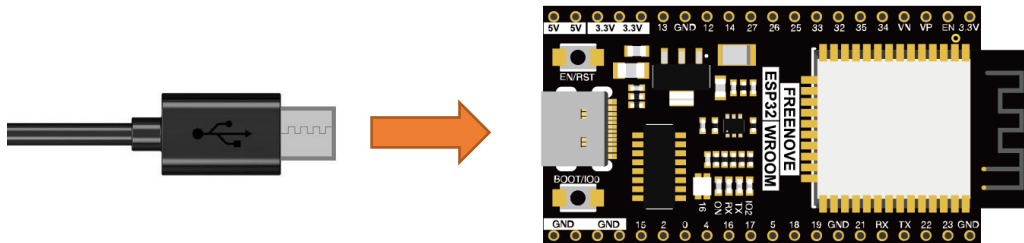
This chapter will help you learn to use a more convenient RGB LED lamp, which requires only one GPIO control and can be connected in infinite series in theory. Each LED can be controlled independently.

### Project 2.1 WS2812

Learn the basic usage of ws2812 and use it to flash red, green, blue and white.

#### Circuit

Connect your computer and ESP32 with a USB cable.



#### Sketch

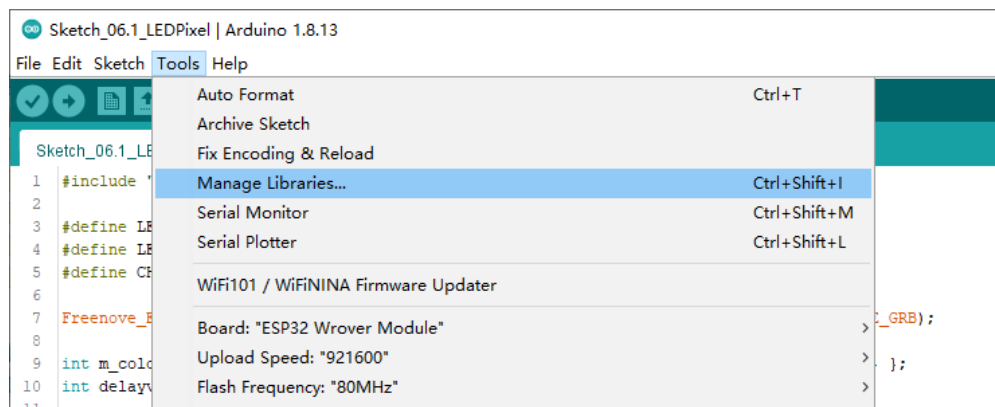
This code uses a library named "Freenove\_WS2812\_Lib\_for\_ESP32", if you have not installed it, please do so first.

Library is an important feature of the open source world, and we know that Arduino is an open source platform that everyone can contribute to. Libraries are generally licensed under the LGPL, which means you can use them for free to apply to your creations.

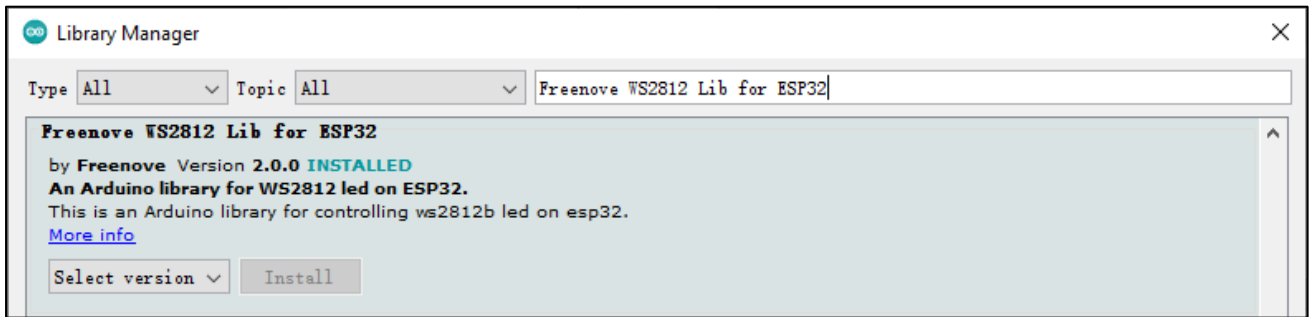
##### How to install the library

There are two ways to add libraries.

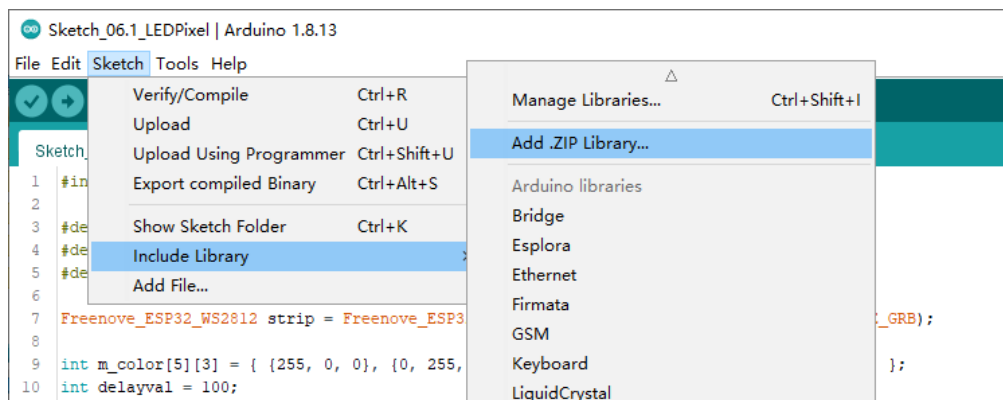
The first way, open the Arduino IDE, click Tools → Manager Libraries.



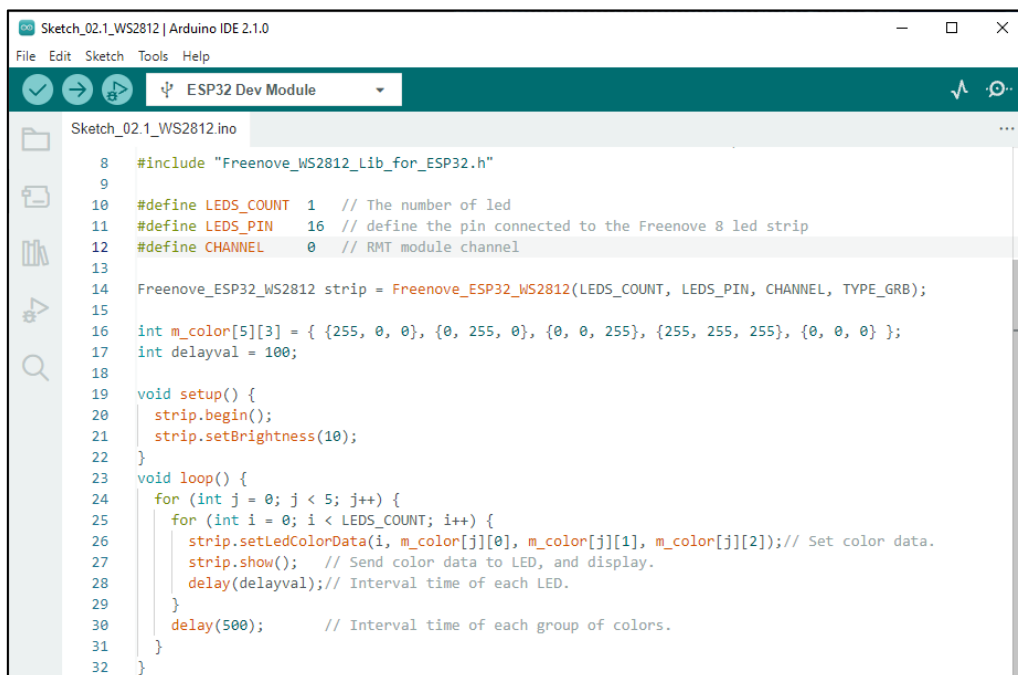
In the pop-up window, Library Manager, search for the name of the Library, "Freenove WS2812 Lib for ESP32". Then click Install.



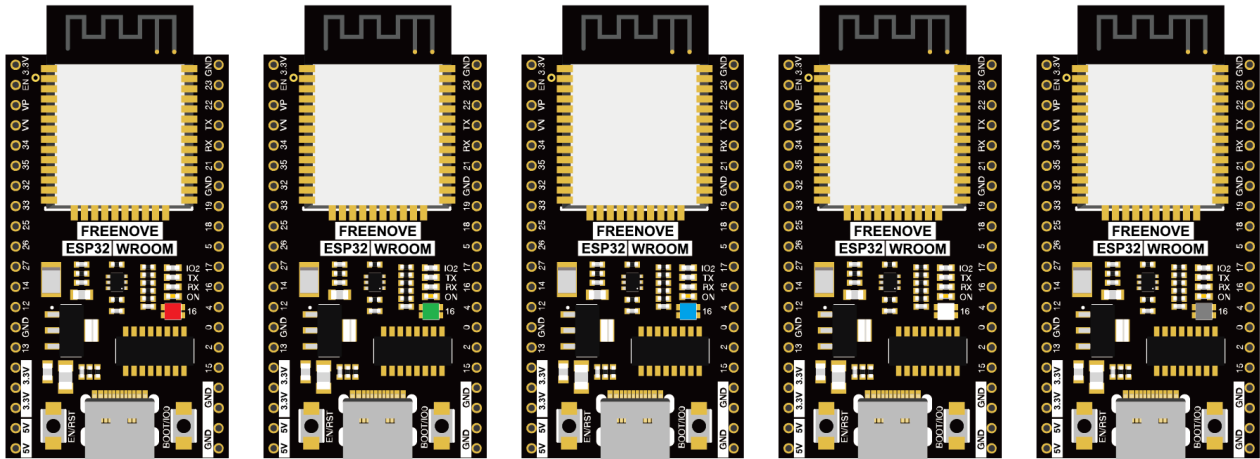
The second way, open Arduino IDE, click Sketch→Include Library→Add .ZIP Library, In the pop-up window, find the file named “./Libraries/Freenove\_WS2812\_Lib\_for\_ESP32.Zip” which locates in this directory, and click OPEN.



### Sketch\_02.1\_WS2812



Download the code to ESP32-WROOM and RGB LED begins to light up in red, green, blue, white and black.



The following is the program code:

```

1  #include "Freenove_WS2812_Lib_for_ESP32.h"
2
3  #define LEDS_COUNT 1 // The number of led
4  #define LEDS_PIN 16 // define the pin connected to the led strip
5  #define CHANNEL 0 // RMT channel
6
7  Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);
8
9  u8 m_color[5][3] = { {255, 0, 0}, {0, 255, 0}, {0, 0, 255}, {255, 255, 255}, {0, 0, 0} };
10 int delayval = 100;
11
12 void setup() {
13     strip.begin();
14     strip.setBrightness(10);
15 }
16 void loop() {
17     for (int j = 0; j < 5; j++) {
18         for (int i = 0; i < LEDS_COUNT; i++) {
19             strip.setLedColorData(i, m_color[j][0], m_color[j][1], m_color[j][2]);
20             strip.show();
21             delay(delayval);
22         }
23         delay(500);
24     }
25 }

```



To use some libraries, first you need to include the library's header file.

```
1 #include "Freenove_WS2812_Lib_for_ESP32.h"
```

Define the pins connected to the ring, the number of LEDs on the ring, and RMT channel values.

```
3 #define LEDS_COUNT 1 // The number of led
4 #define LEDS_PIN 16 // define the pin connected to the led strip
5 #define CHANNEL 0 // RMT channel
```

Use the above parameters to create a ws2812 object strip.

```
7 Freenove_ESP32_WS2812 strip = Freenove_ESP32_WS2812(LEDS_COUNT, LEDS_PIN, CHANNEL, TYPE_GRB);
```

Define the color values to be used, as red, green, blue, white, and black.

```
9 u8 m_color[5][3] = { {255, 0, 0}, {0, 255, 0}, {0, 0, 255}, {255, 255, 255}, {0, 0, 0} };
```

Define a variable to set the time interval for each led to light up. The smaller the value is, the faster it will light up.

```
10 int delayval = 50;
```

Initialize strip() in setup() and set the brightness.

```
13 strip.begin();
14 strip.setBrightness(10);
```

In the loop(), there are two "for" loops, the internal for loop to light the LED one by one, and the external for loop to switch colors. strip.setLedColorData() is used to set the color, but it does not change immediately. Only when strip.show() is called will the color data be sent to the LED to change the color.

```
17 for (int j = 0; j < 5; j++) {
18     for (int i = 0; i < LEDS_COUNT; i++) {
19         strip.setLedColorData(i, m_color[j][0], m_color[j][1], m_color[j][2]);
20         strip.show();
21         delay(delayval);
22     }
23     delay(500);
24 }
```

## Reference

```
Freenove_ESP32_WS2812(u16 n = 8, u8 pin_gpio = 2, u8 chn = 0, LED_TYPE t = TYPE_GRB)
```

Constructor to create a Ws2812 object.

Before each use of the constructor, please add `"#include "Freenove_WS2812_Lib_for_ESP32.h"`

### Parameters

**n**: The number of led.

**pin\_gpio**: A pin connected to an led.

**Chn**: RMT channel, which uses channel 0 by default, has a total of eight channels, 0-7. This means that you can use eight LEDPixel modules for the display at the same time, and these modules do not interfere with each other

**t**: Types of LED.

**TYPE\_RGB**: The sequence of Ws2812 module loading color is red, green and blue.

**TYPE\_RBG**: The sequence of Ws2812 module loading color is red, blue and green.

**TYPE\_GRB**: The sequence of Ws2812 module loading color is green, red and blue.

**TYPE\_GBR**: The sequence of Ws2812 module loading color is green, blue and red.

**TYPE\_BRG**: The sequence of Ws2812 module loading color is blue, red and green.

**TYPE\_BGR**: The sequence of Ws2812 module loading color is blue, green and red.

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

```
void begin(void);
```

Initialize the Ws2812 object

```
void setLedColorData (u8 index, u8 r, u8 g, u8 b);
```

```
void setLedColorData (u8 index, u32 rgb);
```

```
void setLedColor (u8 index, u8 r, u8 g, u8 b);
```

```
void setLedColor (u8 index, u32 rgb);
```

Set the color of led with order number n.

```
void show(void);
```

Send the color data to the led and display the set color immediately.

```
void setBrightness(uint8_t);
```

Set the brightness of the LED.

If you want to learn more about this library, you can visit the following website:

[https://github.com/Freenove/Freenove\\_WS2812\\_Lib\\_for\\_ESP32](https://github.com/Freenove/Freenove_WS2812_Lib_for_ESP32)

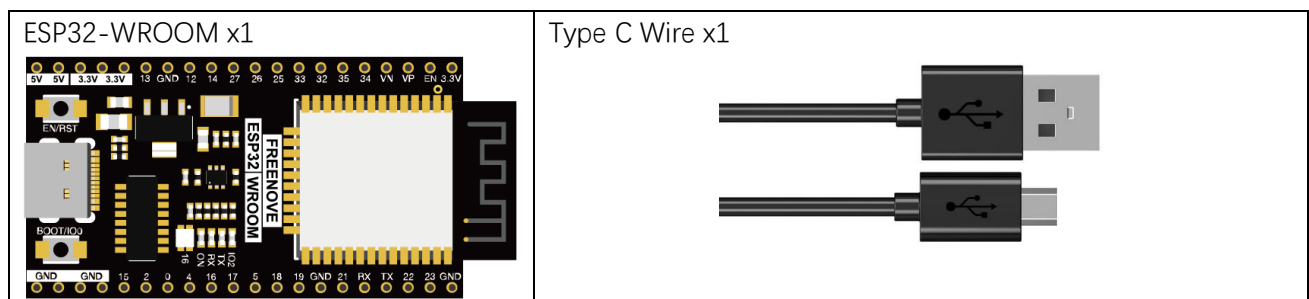
## Chapter 3 Bluetooth

This chapter mainly introduces how to make simple data transmission through Bluetooth of ESP32-WROOM and mobile phones.

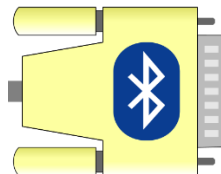
Project 3.1 is classic Bluetooth and Project 3.2 is low power Bluetooth. If you are an iPhone user, please start with Project 3.2.

### Project 3.1 Bluetooth Passthrough

#### Component List



In this tutorial we need to use a Bluetooth APP called Serial Bluetooth Terminal to assist in the experiment. If you've not installed it yet, please do so by clicking: <https://www.appsapk.com/serial-bluetooth-terminal/> The following is its logo.



## Component knowledge

ESP32's integrated Bluetooth function Bluetooth is a short-distance communication system, which can be divided into two types, namely Bluetooth Low Energy(BLE) and Classic Bluetooth. There are two modes for simple data transmission: master mode and slave mode.

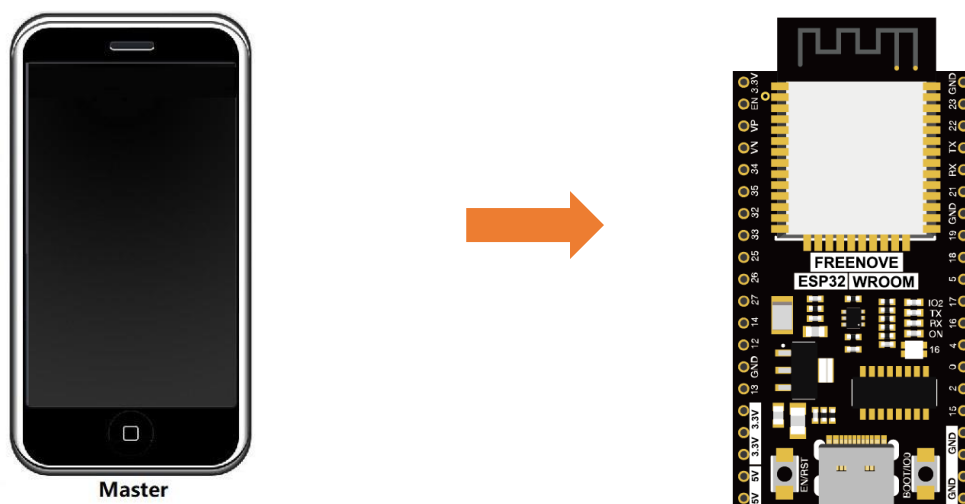
### Master mode

In this mode, works are done in the master device and it can connect with a slave device. And we can search and select slave devices nearby to connect with. When a device initiates connection request in master mode, it requires information of the other Bluetooth devices including their address and pairing passkey. After finishing pairing, it can connect with them directly.

### Slave mode

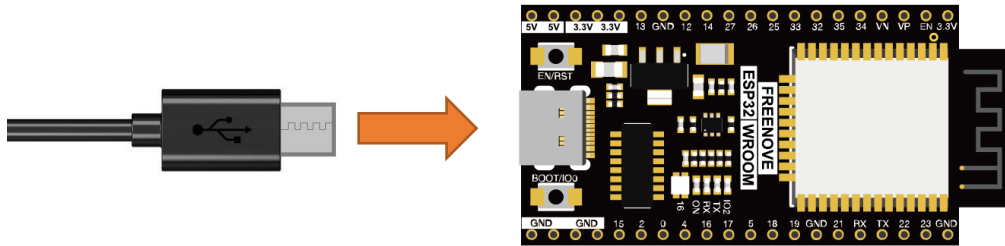
The Bluetooth module in slave mode can only accept connection request from a host computer, but cannot initiate a connection request. After connecting with a host device, it can send data to or receive from the host device.

Bluetooth devices can make data interaction with each other, as one is in master mode and the other in slave mode. When they are making data interaction, the Bluetooth device in master mode searches and selects devices nearby to connect to. When establishing connection, they can exchange data. When mobile phones exchange data with ESP32, they are usually in master mode and ESP32 in slave mode.



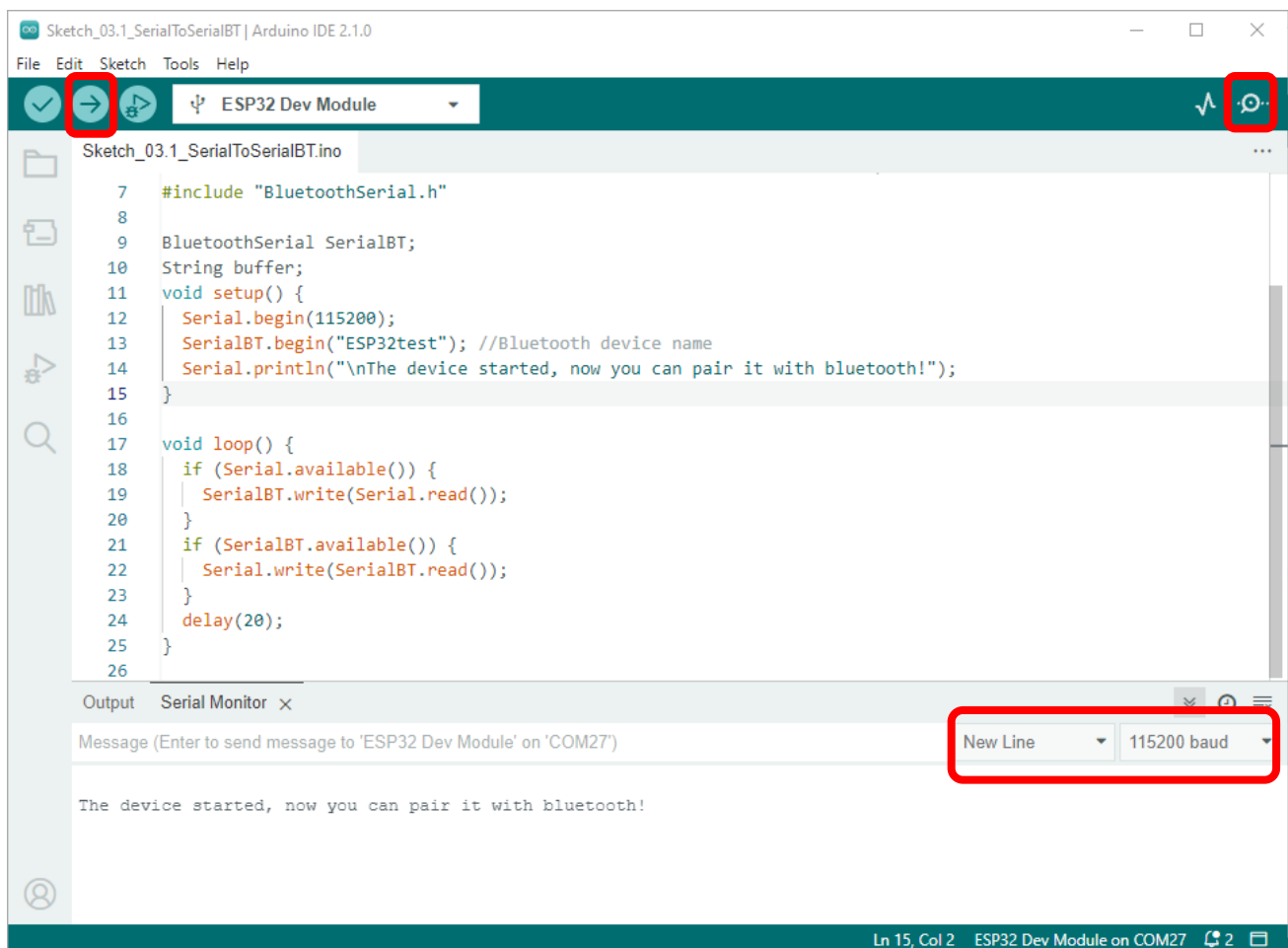
## Circuit

Connect Freenove ESP32 to the computer using the USB cable.



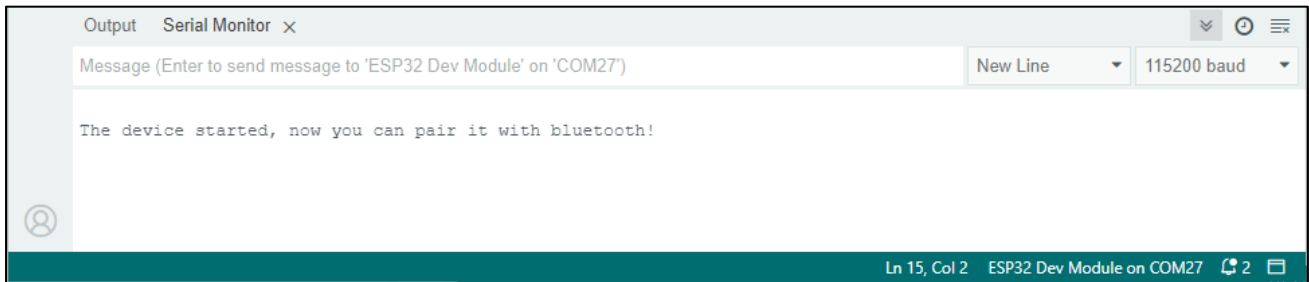
## Sketch

### Sketch\_03.1\_SerialToSerialBT

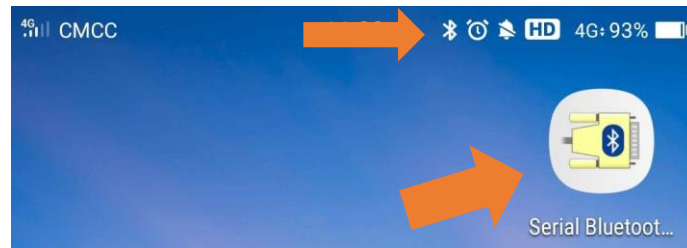


Compile and upload the code to the ESP32-WROOM, open the serial monitor, and set the baud rate to 115200. When you see the serial printing out the character string as below, it indicates that the Bluetooth of ESP32 is ready and waiting to connect with the mobile phone.

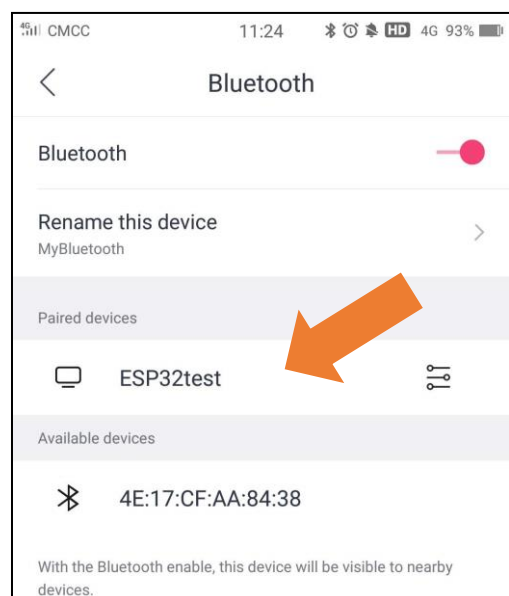
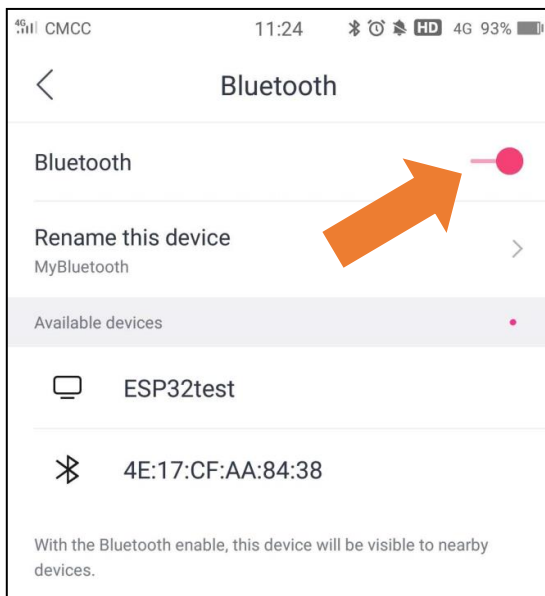
Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)



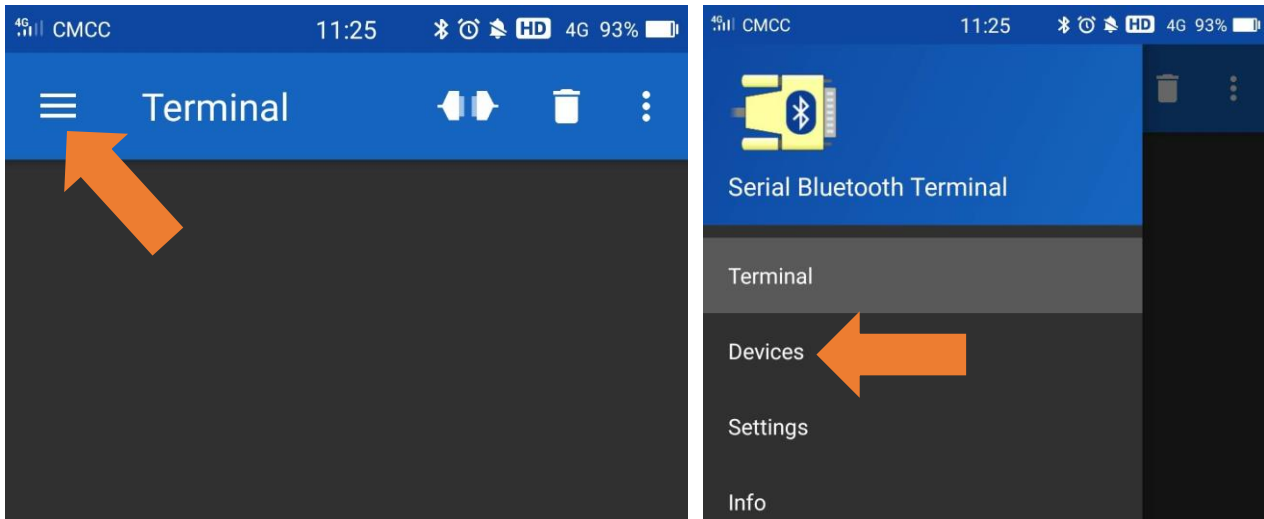
Make sure that the Bluetooth of your phone has been turned on and Serial Bluetooth Terminal has been installed.



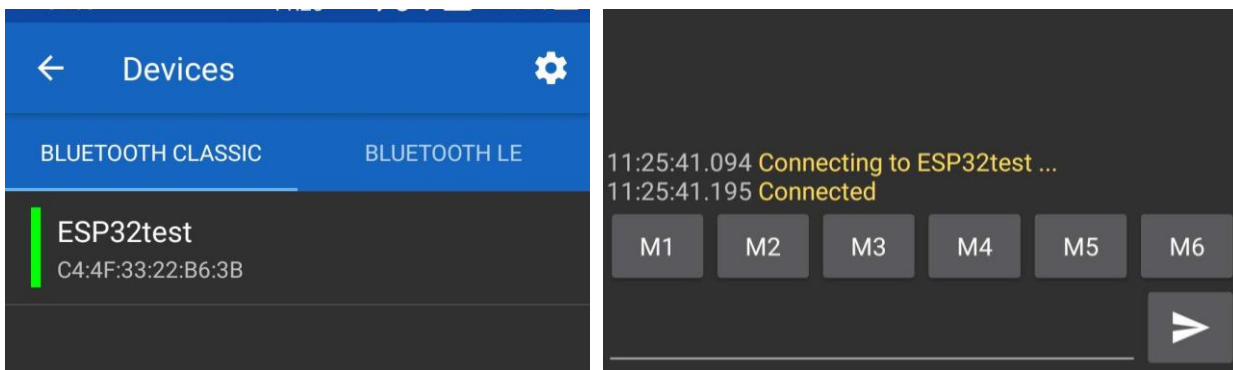
Click "Search" to search Bluetooth devices nearby and select "ESP32 test" to connect to.



Turn on software APP, click the left of the terminal. Select "Devices"

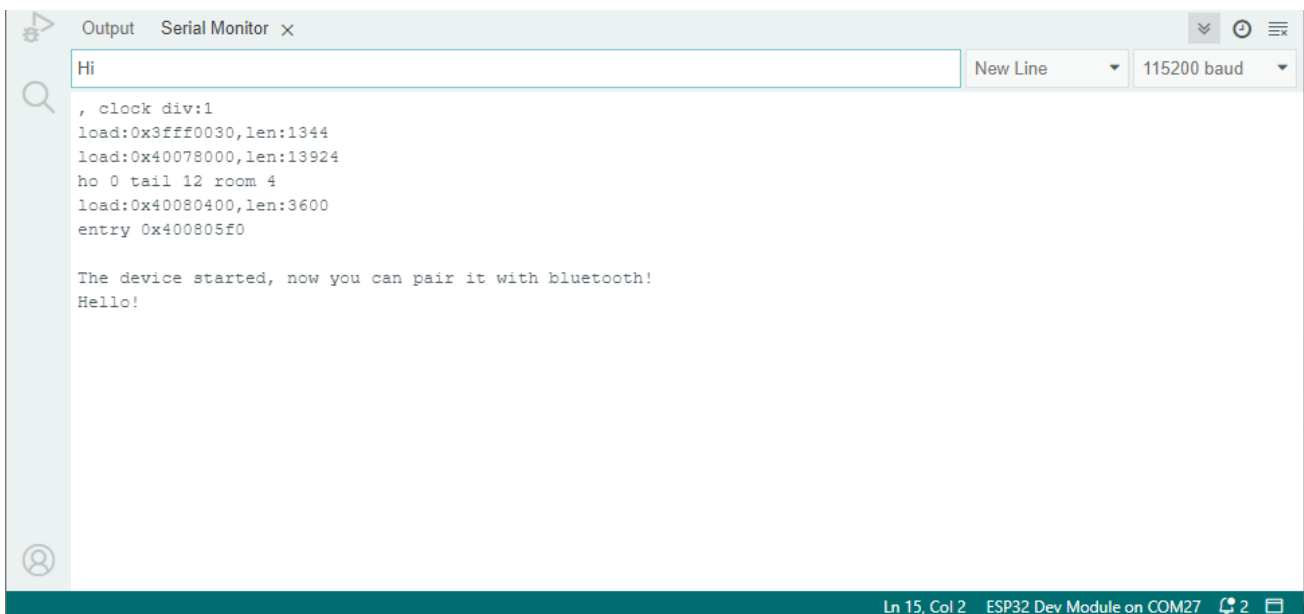


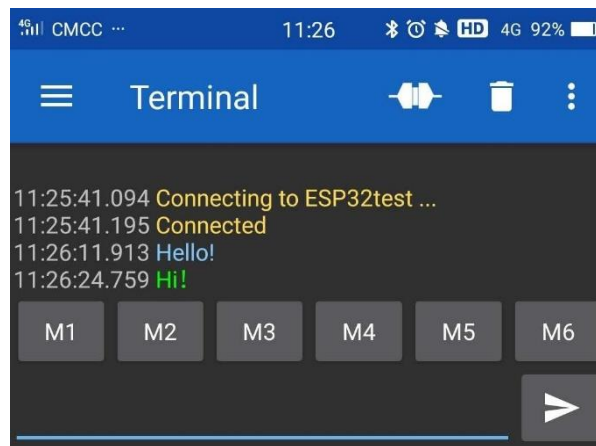
Select ESP32test in classic Bluetooth mode, and a successful connecting prompt will appear as shown on the right illustration.



And now data can be transferred between your mobile phone and computer via ESP32-WROOM.

Send 'Hello!' from your phone, when the computer receives it, reply 'Hi' to your phone.





## Reference

### Class BluetoothSerial

This is a class library used to operate **BluetoothSerial**, which can directly read and set **BluetoothSerial**. Here are some member functions:

**begin(localName, isMaster)**: Initialization function of the Bluetooth

name: name of Bluetooth module; Data type: String

**isMaster**: bool type, whether to set Bluetooth as Master. By default, it is false.

**available()**: acquire digits sent from the buffer, if not, return 0.

**read()**: read data from Bluetooth, data type of return value is int.

**readString()**: read data from Bluetooth, data type of return value is String.

**write(val)**: send an int data val to Bluetooth.

**write(str)**: send an String data str to Bluetooth.

**write(buf, len)**: Sends the first len data in the buf Array to Bluetooth.

**setPin(const char \*pin)**: set a four-digit Bluetooth pairing code. By default, it is 1234

**connect(remoteName)**: connect a Bluetooth named remoteName, data type: String

**connect(remoteAddress[])**: connect the physical address of Bluetooth, data type: uint8-t.

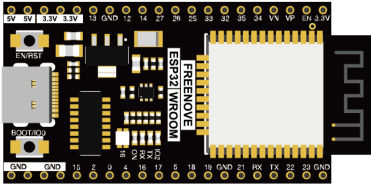
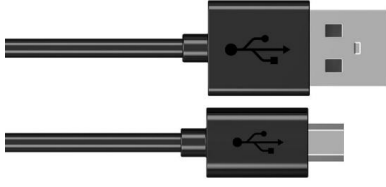
**disconnect()**: disconnect all Bluetooth devices.

**end()**: disconnect all Bluetooth devices and turn off the Bluetooth, release all occupied space



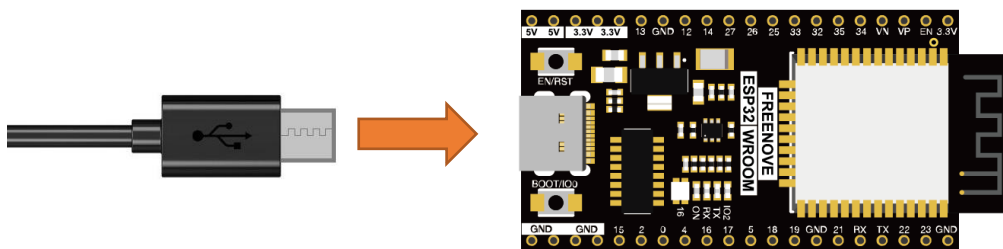
## Project 3.2 Bluetooth Low Energy Data Passthrough

### Component List

|                                                                                                     |                                                                                                      |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| ESP32-WROOM x1<br> | Type C Wire x1<br> |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|

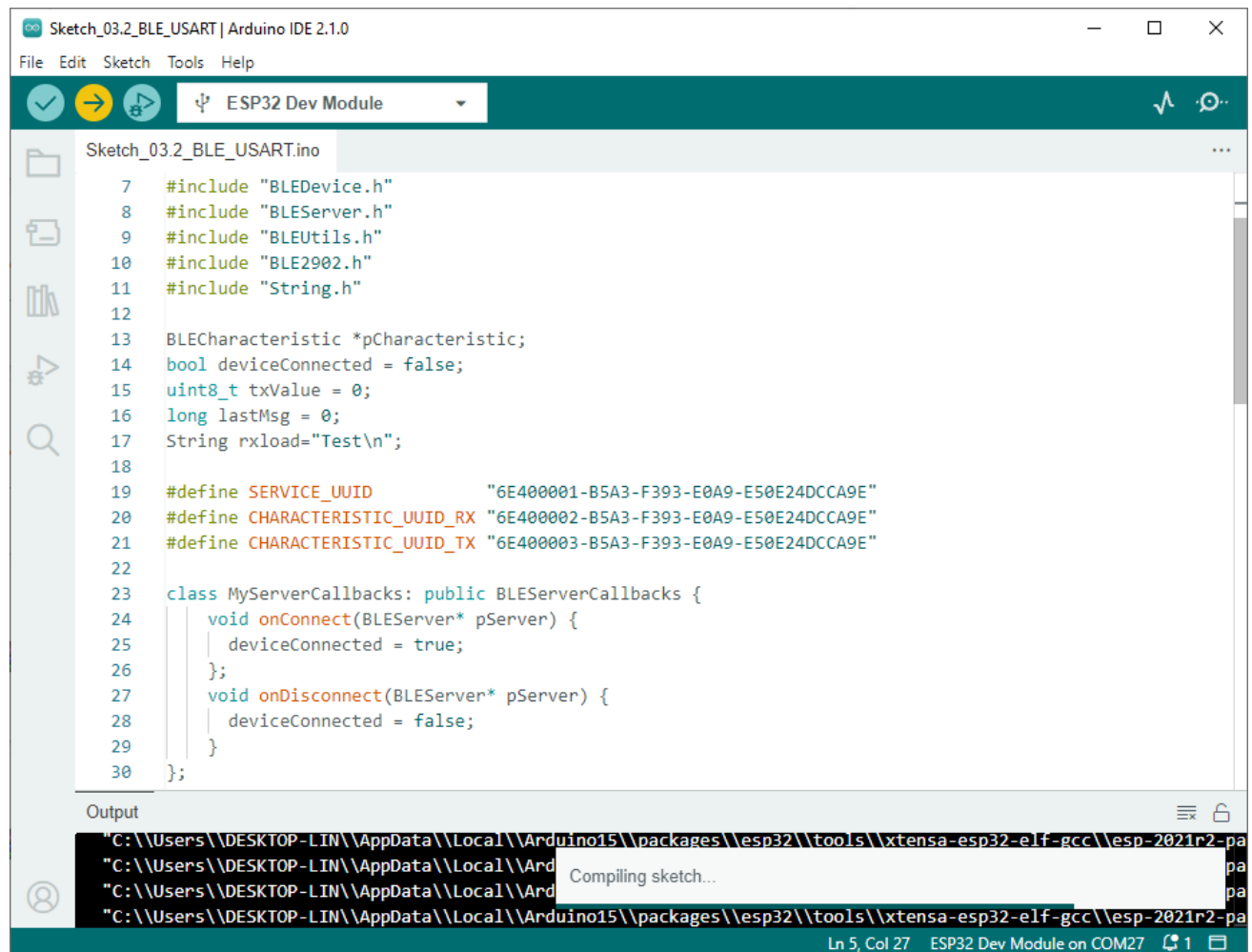
### Circuit

Connect Freenove ESP32 to the computer using the USB cable.



## Sketch

### Sketch\_03.2\_BLE



```

Sketch_03.2_BLE_USART.ino
7  #include "BLEDevice.h"
8  #include "BLEServer.h"
9  #include "BLEUtils.h"
10 #include "BLE2902.h"
11 #include "String.h"
12
13 BLECharacteristic *pCharacteristic;
14 bool deviceConnected = false;
15 uint8_t txValue = 0;
16 long lastMsg = 0;
17 String rxload="Test\n";
18
19 #define SERVICE_UUID          "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
20 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
21 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
22
23 class MyServerCallbacks: public BLEServerCallbacks {
24     void onConnect(BLEServer* pServer) {
25         deviceConnected = true;
26     };
27     void onDisconnect(BLEServer* pServer) {
28         deviceConnected = false;
29     }
30 };
  
```

Output

```

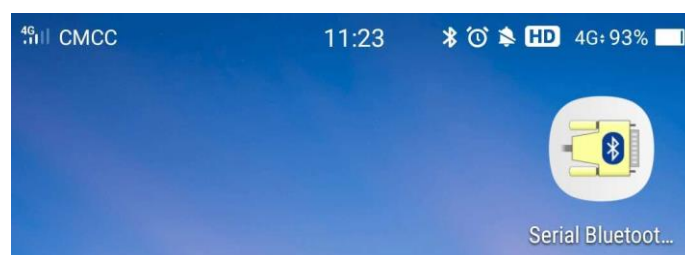
"C:\Users\DESKTOP-LIN\AppData\Local\Arduino15\packages\esp32\tools\xtensa-esp32-elf-gcc\esp-2021r2-pa
"C:\Users\DESKTOP-LIN\AppData\Local\Ard
"C:\Users\DESKTOP-LIN\AppData\Local\Ard
Compiling sketch...
"C:\Users\DESKTOP-LIN\AppData\Local\Arduino15\packages\esp32\tools\xtensa-esp32-elf-gcc\esp-2021r2-pa
  
```

Ln 5, Col 27 ESP32 Dev Module on COM27

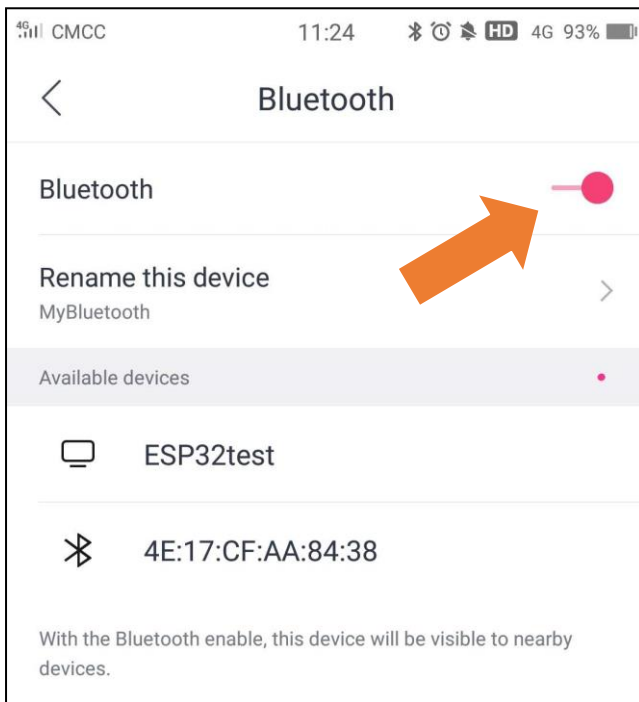
### Serial Bluetooth

Compile and upload code to ESP32, the operation is similar to the last section.

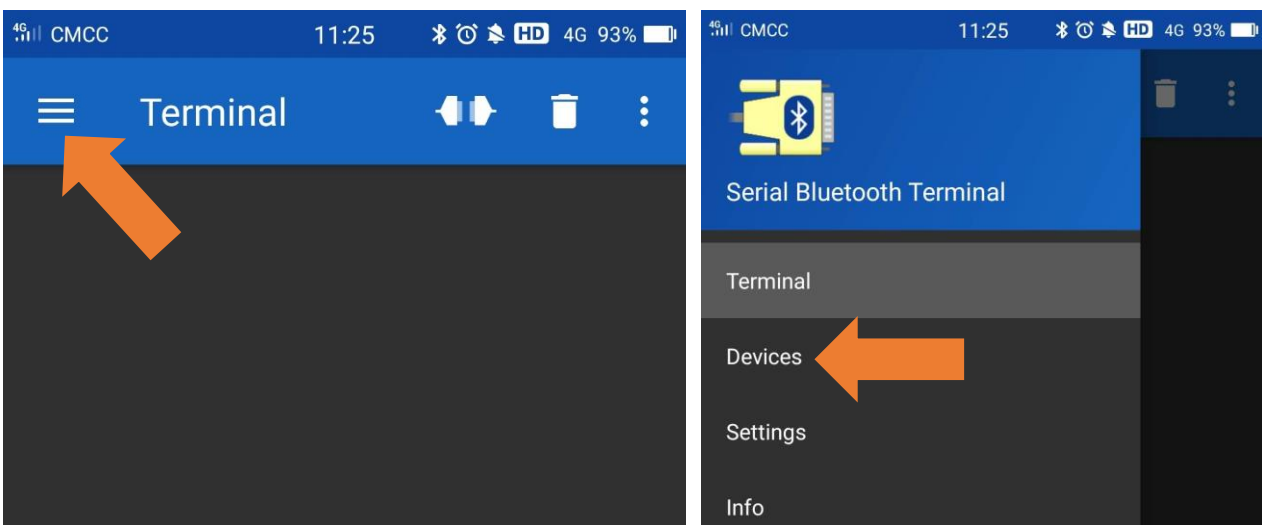
First, make sure you've turned on the mobile phone Bluetooth, and then open the software.



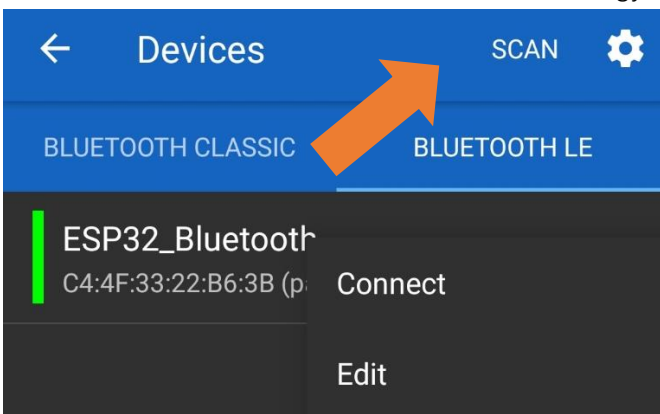
Click "Search" to search Bluetooth devices nearby and select "ESP32 test" to connect to.



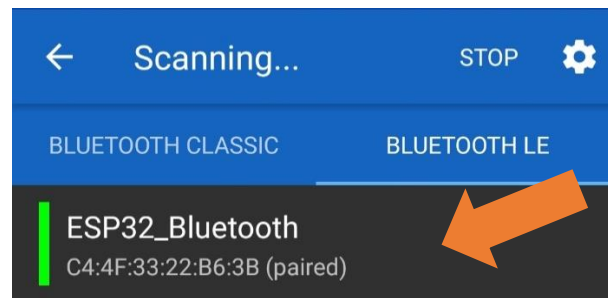
Turn on software APP, click the left of the terminal. Select "Devices"



Select BLUETOOTHLE, click SCAN to scan Low Energy Bluetooth devices nearby.



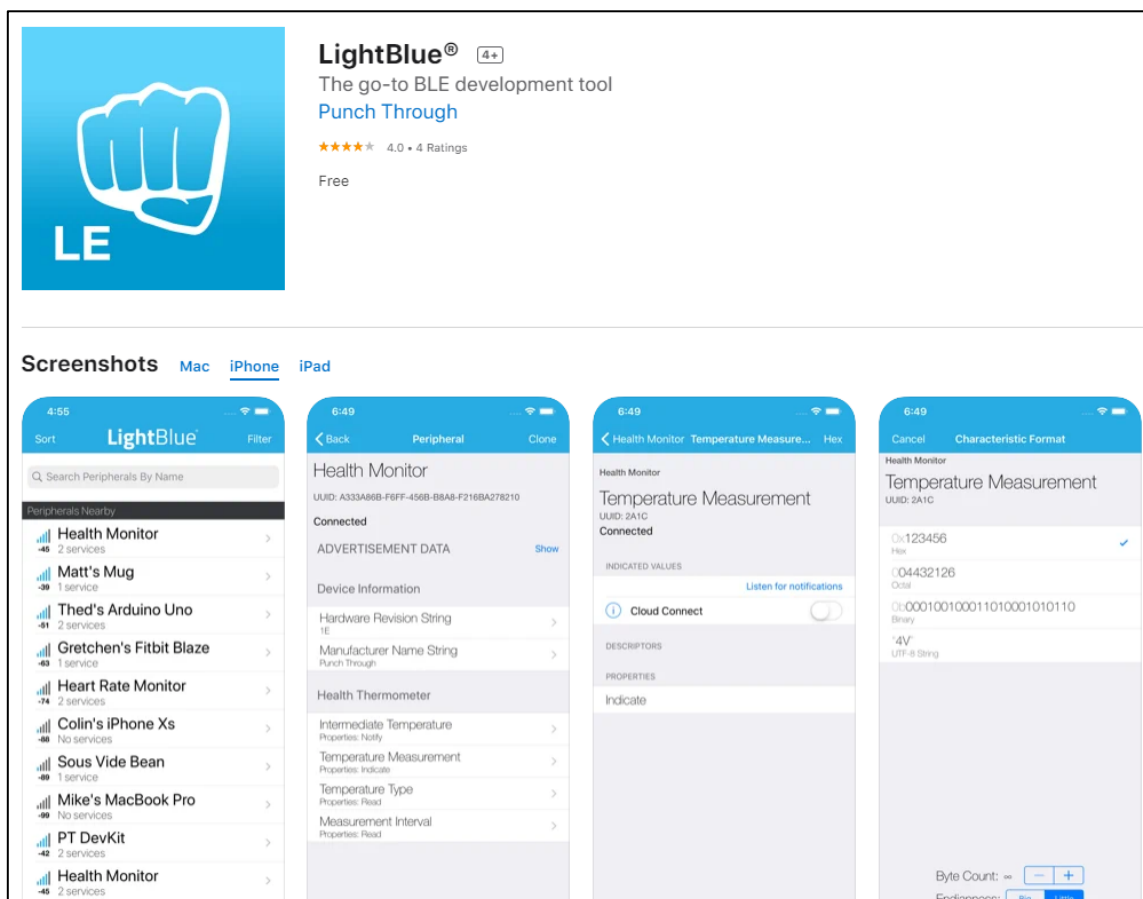
Select "ESP32-Bluetooth"



### Lightblue

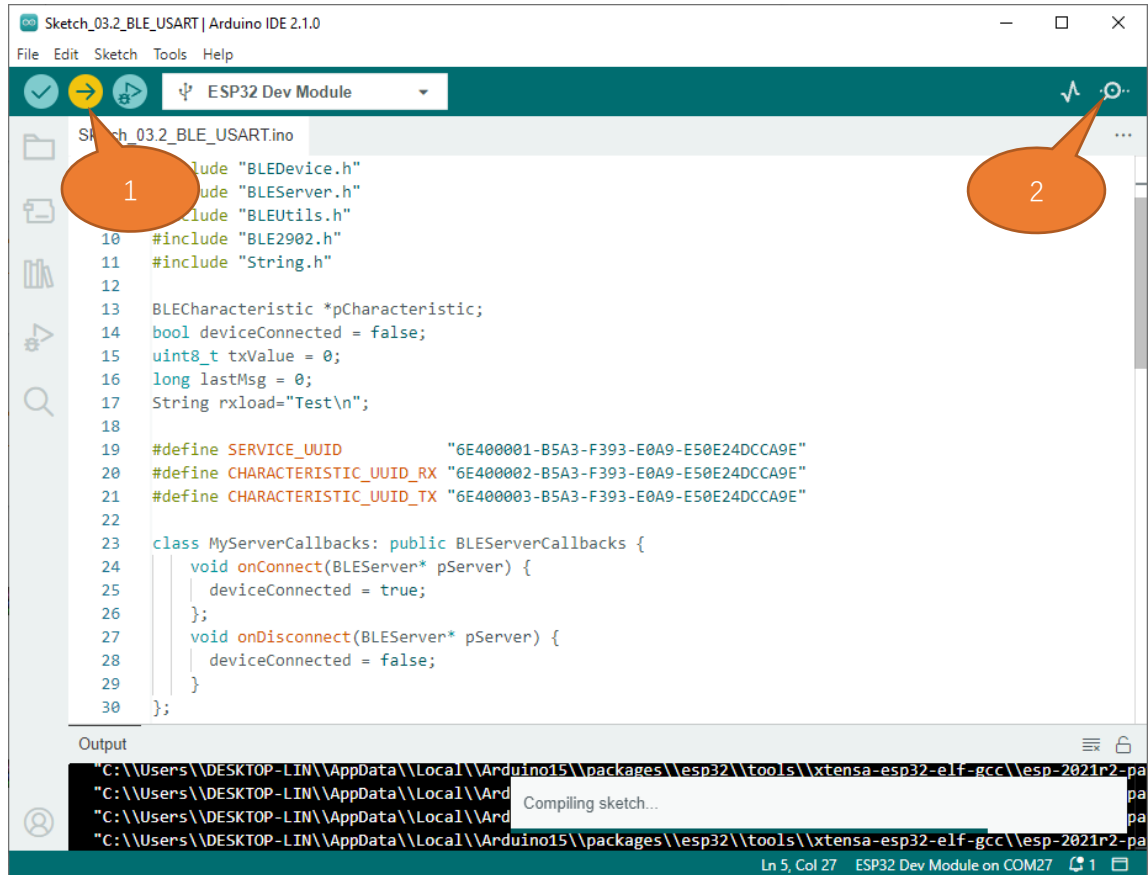
If you can't install Serial Bluetooth on your phone, try LightBlue. If you do not have this software installed on your phone, you can refer to this link:

<https://apps.apple.com/us/app/lightblue/id557428110/?platform=iphone>.



Step1. Upload the code to ESP32.

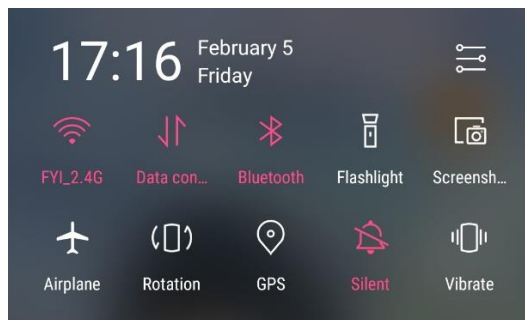
Step2. Click on serial monitor.



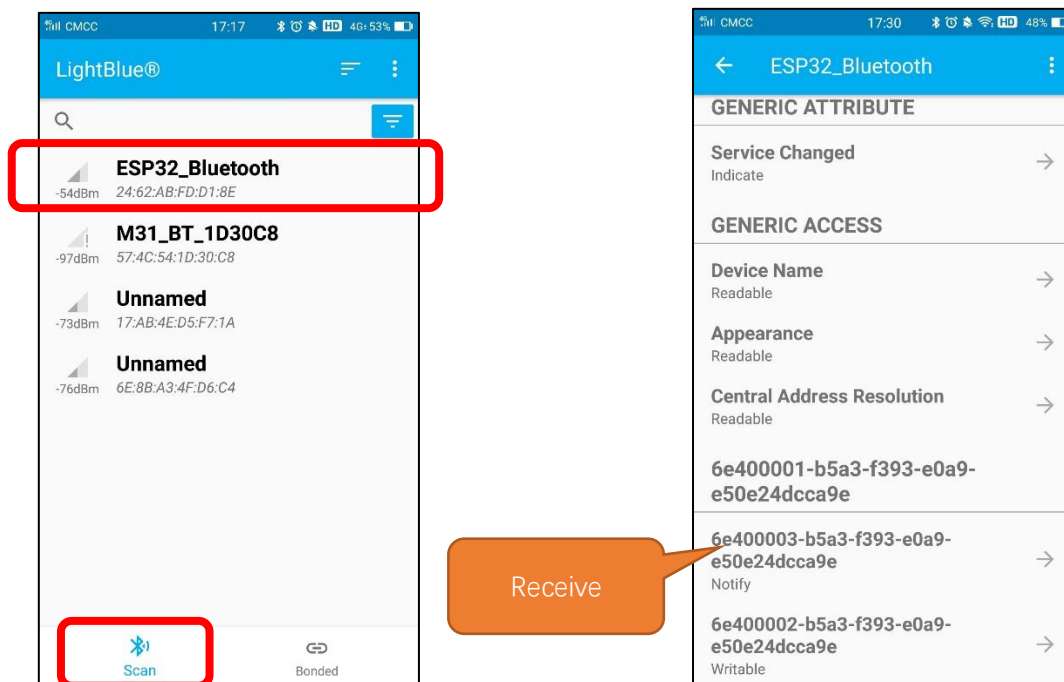
Step3. Set baud rate to 115200.



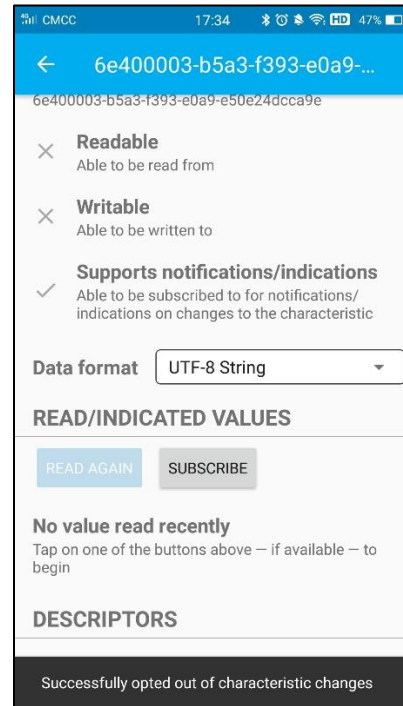
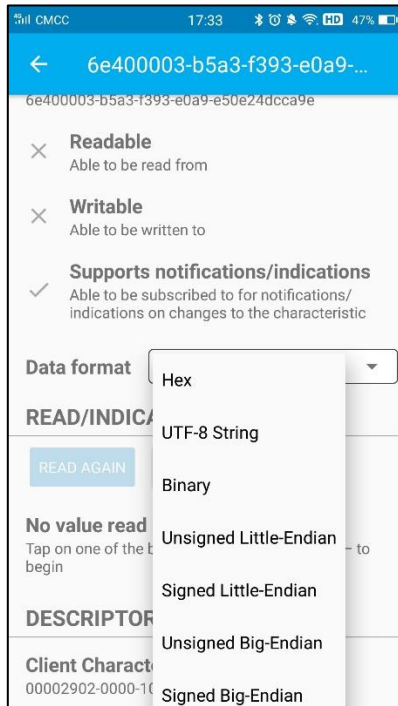
Turn ON Bluetooth on your phone, and open the Lightblue APP.



In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click ESP32\_Bluetooth.



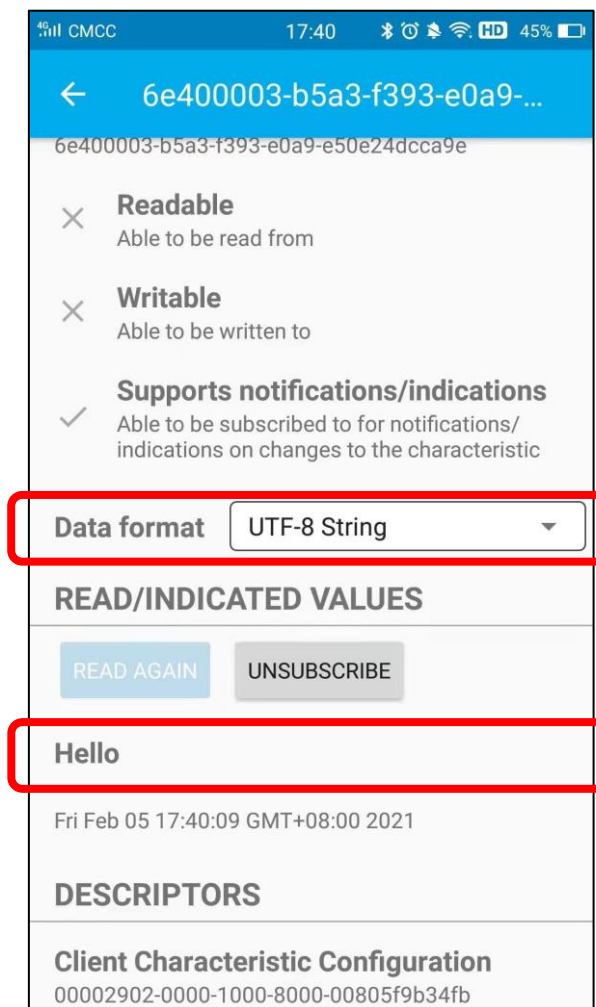
Click "Receive". Select the appropriate Data format in the box to the right of Data Format. For example, HEX for hexadecimal, utf-string for character, Binary for Binary, etc. Then click SUBSCRIBE.



Back to the serial monitor on your computer. You can type anything in the left border of Send, and then click Send.

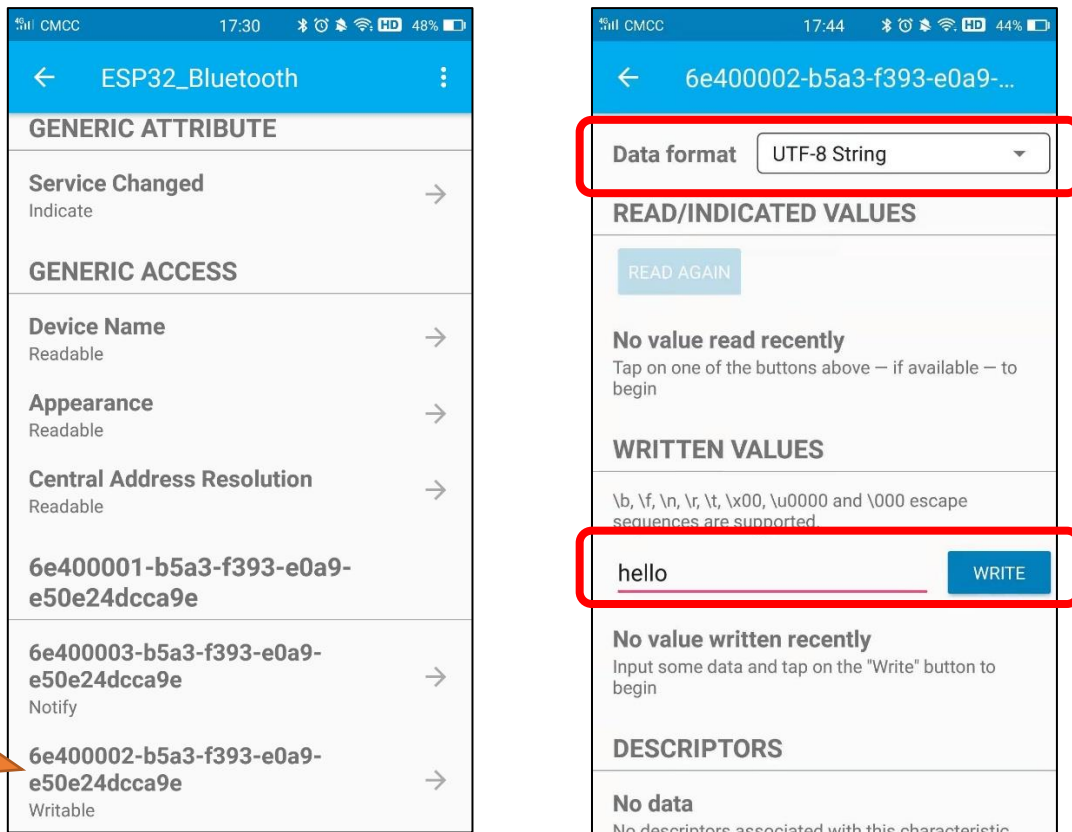


And then you can see the mobile Bluetooth has received the message.

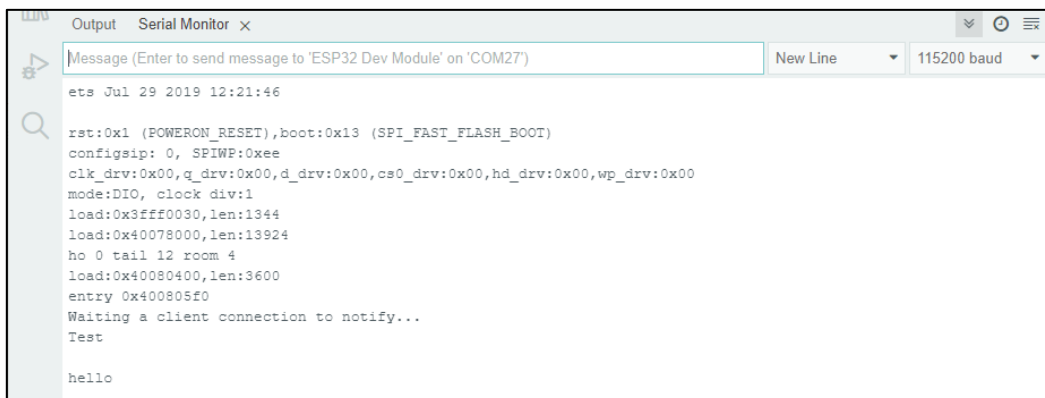




Similarly, you can select “Send” on your phone. Set Data format, and then enter anything in the sending box and click Write to send.



And the computer will receive the message from the mobile Bluetooth.



And now data can be transferred between your mobile phone and computer via ESP32-WROOM.

The following is the program code:

```

1  #include <BLEDevice.h>
2  #include <BLEServer.h>
3  #include <BLEUtils.h>
4  #include <BLE2902.h>
5  #include <String.h>
6
7  BLECharacteristic *pCharacteristic;
8  bool deviceConnected = false;
9  uint8_t txValue = 0;
10 long lastMsg = 0;
11 String rxload="Test\n";
12
13 #define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
14 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
15 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
16
17 class MyServerCallbacks: public BLEServerCallbacks {
18     void onConnect(BLEServer* pServer) {
19         deviceConnected = true;
20     };
21     void onDisconnect(BLEServer* pServer) {
22         deviceConnected = false;
23     }
24 };
25
26 class MyCallbacks: public BLECharacteristicCallbacks {
27     void onWrite(BLECharacteristic *pCharacteristic) {
28         String rxValue = pCharacteristic->getValue();
29         if (rxValue.length() > 0) {
30             rxload="";
31             for (int i = 0; i < rxValue.length(); i++){
32                 rxload +=(char)rxValue[i];
33             }
34         }
35     }
36 };
37
38 void setupBLE(String BLEName){
39     const char *ble_name=BLEName.c_str();
40     BLEDevice::init(ble_name);
41     BLEServer *pServer = BLEDevice::createServer();
42     pServer->setCallbacks(new MyServerCallbacks());

```

```

43  BLEService *pService = pServer->createService(SERVICE_UUID);
44  pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX, BLECharacteristic::PROPERTY_NOTIFY);
45  pCharacteristic->addDescriptor(new BLE2902());
46  BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX, BLECharacteristic::PROPERTY_WRITE);
47  pCharacteristic->setCallbacks(new MyCallbacks());
48  pService->start();
49  pServer->getAdvertising()->start();
50  Serial.println("Waiting a client connection to notify...");
51  }
52
53  void setup() {
54    Serial.begin(115200);
55    setupBLE("ESP32_Bluetooth");
56  }
57
58  void loop() {
59    long now = millis();
60    if (now - lastMsg > 1000) {
61      if (deviceConnected&&rxload.length()>0) {
62        Serial.println(rxload);
63        rxload="";
64      }
65      if(Serial.available()>0){
66        String str=Serial.readString();
67        const char *newValue=str.c_str();
68        pCharacteristic->setValue(newValue);
69        pCharacteristic->notify();
70      }
71      lastMsg = now;
72    }
73  }

```

Define the specified UUID number for BLE vendor.

```

13  #define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
14  #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
15  #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

```

Write a Callback function for BLE server to manage connection of BLE.

```

17 class MyServerCallbacks: public BLEServerCallbacks {
18     void onConnect(BLEServer* pServer) {
19         deviceConnected = true;
20     };
21     void onDisconnect(BLEServer* pServer) {
22         deviceConnected = false;
23     }
24 };

```

Write Callback function with BLE features. When it is called, as the mobile terminal send data to ESP32, it will store them into reload.

```

26 class MyCallbacks: public BLECharacteristicCallbacks {
27     void onWrite(BLECharacteristic *pCharacteristic) {
28         std::string rxValue = pCharacteristic->getValue();
29         if (rxValue.length() > 0) {
30             rxload="";
31             for (int i = 0; i < rxValue.length(); i++) {
32                 rxload +=(char)rxValue[i];
33             }
34         }
35     }
36 };

```

Initialize the BLE function and name it.

```

55 setupBLE("ESP32_Bluetooth");

```

When the mobile phone send data to ESP32 via BLE Bluetooth, it will print them out with serial port; When the serial port of ESP32 receive data, it will send them to mobile via BLE Bluetooth.

```

59 long now = millis();
60 if (now - lastMsg > 1000) {
61     if (deviceConnected&&rxload.length()>0) {
62         Serial.println(rxload);
63         rxload="";
64     }
65     if(Serial.available()>0){
66         String str=Serial.readString();
67         const char *newValue=str.c_str();
68         pCharacteristic->setValue(newValue);
69         pCharacteristic->notify();
70     }
71     lastMsg = now;
72 }

```

The design for creating the BLE server is:

1. Create a BLE Server
2. Create a BLE Service
3. Create a BLE Characteristic on the Service
4. Create a BLE Descriptor on the characteristic
5. Start the service.
6. Start advertising.

```
38 void setupBLE(String BLEName) {
39     const char *ble_name=BLEName.c_str();
40     BLEDevice::init(ble_name);
41     BLEServer *pServer = BLEDevice::createServer();
42     pServer->setCallbacks(new MyServerCallbacks());
43     BLEService *pService = pServer->createService(SERVICE_UUID);
44     pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX,BLECharacteristic::PROPERTY_NOTIFY);
45     pCharacteristic->addDescriptor(new BLE2902());
46     BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX,BLECharacteristic::PROPERTY_WRITE);
47     pCharacteristic->setCallbacks(new MyCallbacks());
48     pService->start();
49     pServer->getAdvertising()->start();
50     Serial.println("Waiting a client connection to notify...");
51 }
```

## Chapter 4 WiFi Working Modes

In this chapter, we'll focus on the WiFi infrastructure for ESP32-WROOM.

ESP32-WROOM has 3 different WiFi operating modes: station mode, AP mode and AP+station mode. All WiFi programming projects must be configured with WiFi operating mode before using WiFi, otherwise WiFi cannot be used.

### Project 4.1 Station mode

#### Component List

|                |                |
|----------------|----------------|
| Type C Wire x1 | ESP32-WROOM x1 |
|----------------|----------------|

#### Component knowledge

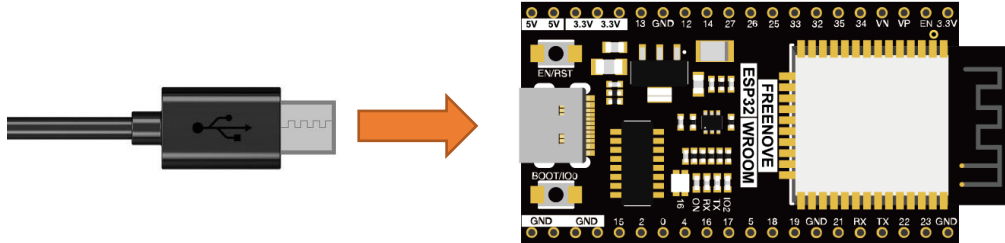
##### Station mode

When ESP32 selects Station mode, it acts as a WiFi client. It can connect to the router network and communicate with other devices on the router via WiFi connection. As shown below, the PC is connected to the router, and if ESP32 wants to communicate with the PC, it needs to be connected to the router.



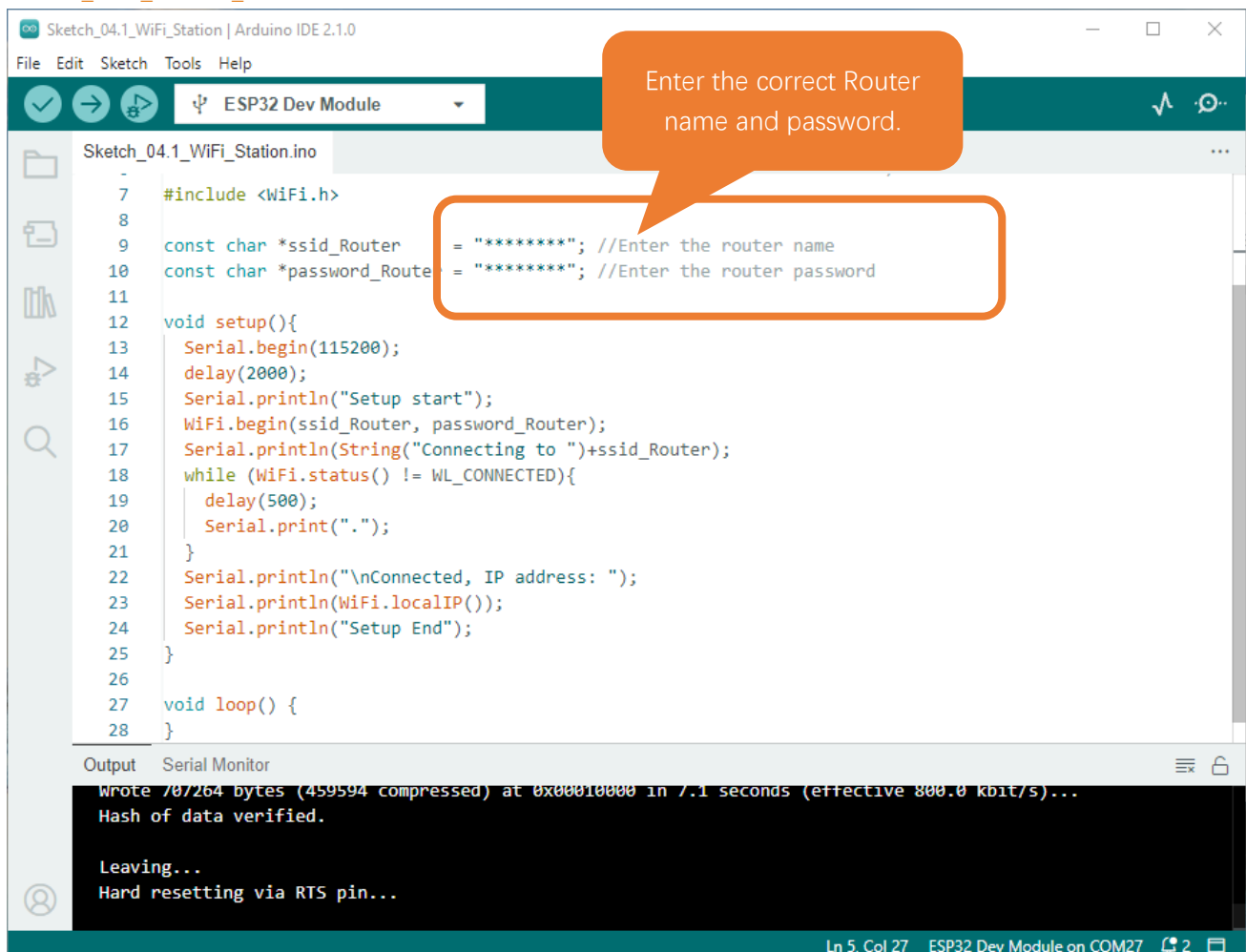
## Circuit

Connect Freenove ESP32 to the computer using the USB cable.



## Sketch

### Sketch\_04.1\_Station\_mode



Because the names and passwords of routers in various places are different, before the Sketch runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to ESP32-WROOM, open serial monitor and set baud rate to 115200. And then it will display as follows:

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

```

Output Serial Monitor x
Message (Enter to send message to 'ESP32 Dev Module' on 'COM27') New Line 115200 baud
conrigsip: 0, SPIWIF:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:1344
load:0x40078000,len:13924
ho 0 tail 12 room 4
load:0x40080400,len:3600
entry 0x400805f0
Setup start
Connecting to FYI_2.4G
..
Connected, IP address:
192.168.1.121
Setup End
Ln 28, Col 2 ESP32 Dev Module on COM27

```

When ESP32-WROOM successfully connects to “ssid\_Router”, serial monitor will print out the IP address assigned to ESP32-WROOM by the router.

The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5
6  void setup() {
7      Serial.begin(115200);
8      delay(2000);
9      Serial.println("Setup start");
10     WiFi.begin(ssid_Router, password_Router);
11     Serial.println(String("Connecting to ") + ssid_Router);
12     while (WiFi.status() != WL_CONNECTED) {
13         delay(500);
14         Serial.print(".");
15     }
16     Serial.println("\nConnected, IP address: ");
17     Serial.println(WiFi.localIP());
18     Serial.println("Setup End");
19 }
20
21 void loop() {
22 }

```

Include the WiFi Library header file of ESP32.

```

1  #include <WiFi.h>

```

Enter correct router name and password.

```

3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password

```

Set ESP32 in Station mode and connect it to your router.

```

10     WiFi.begin(ssid_Router, password_Router);

```



Check whether ESP32 has connected to router successfully every 0.5s.

```
12 while (WiFi.status() != WL_CONNECTED) {
13     delay(500);
14     Serial.print(".");
15 }
```

Serial monitor prints out the IP address assigned to ESP32-WROOM

```
17 Serial.println(WiFi.localIP());
```

## Reference

### Class Station

Every time when using WiFi, you need to include header file "WiFi.h".

**begin(ssid, password, channel, bssid, connect):** ESP32 is used as Station to connect hotspot.

**ssid:** WiFi hotspot name

**password:** WiFi hotspot password

**channel:** WiFi hotspot channel number; communicating through specified channel; optional parameter

**bssid:** mac address of WiFi hotspot, optional parameter

**connect:** boolean optional parameter, defaulting to true. If set as false, then ESP32 won't connect WiFi.

**config(local\_ip, gateway, subnet, dns1, dns2):** set static local IP address.

**local\_ip:** station fixed IP address.

**subnet:** subnet mask

**dns1, dns2:** optional parameter. define IP address of domain name server

**status:** obtain the connection status of WiFi

**local IP():** obtain IP address in Station mode

**disconnect():** disconnect wifi

**setAutoConnect(boolean):** set automatic connection Every time ESP32 is power on, it will connect WiFi automatically.

**setAutoReconnect(boolean):** set automatic reconnection Every time ESP32 disconnects WiFi, it will reconnect to WiFi automatically.

## Project 4.2 AP mode

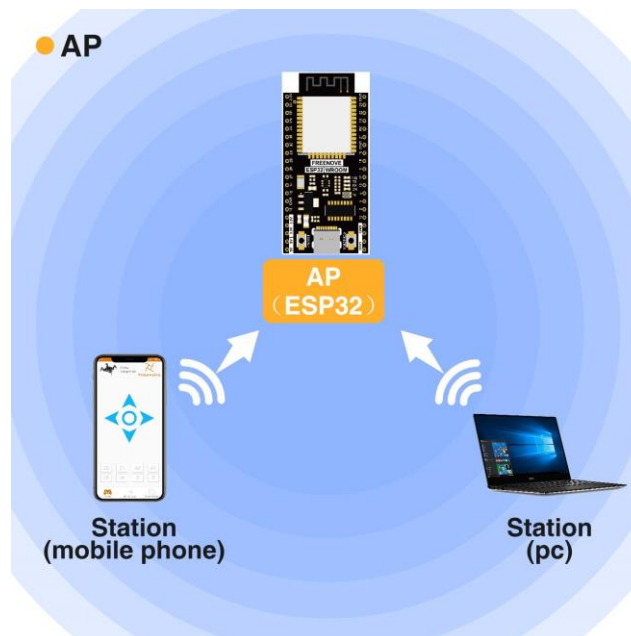
### Component List & Circuit

Component List & Circuit are the same as in Section 30.1.

### Component knowledge

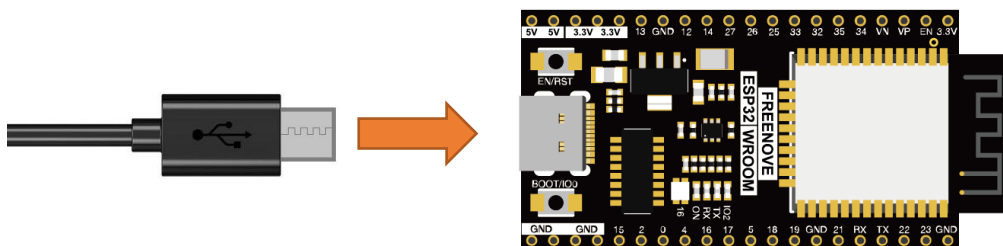
#### AP mode

When ESP32 selects AP mode, it creates a hotspot network that is separate from the Internet and waits for other WiFi devices to connect. As shown in the figure below, ESP32 is used as a hotspot. If a mobile phone or PC wants to communicate with ESP32, it must be connected to the hotspot of ESP32. Only after a connection is established with ESP32 can they communicate.



### Circuit

Connect Freenove ESP32 to the computer using the USB cable.



## Sketch



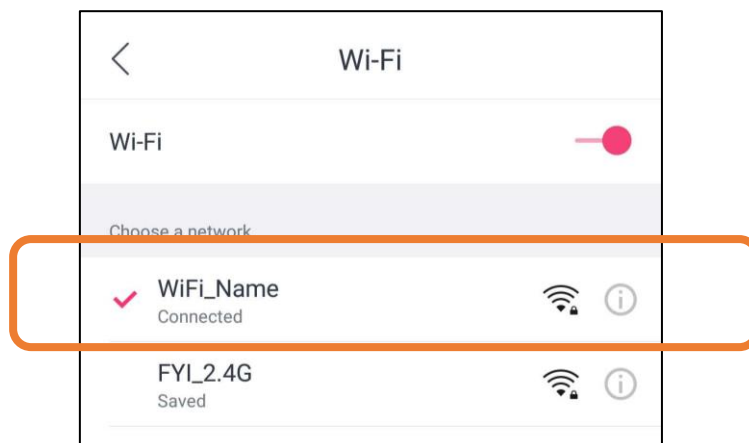
Before the Sketch runs, you can make any changes to the AP name and password for ESP32 in the box as shown in the illustration above. Of course, you can leave it alone by default.

Compile and upload codes to ESP32-WROOM, open the serial monitor and set the baud rate to 115200. And then it will display as follows.



```
ets Jul 29 2019 12:21:46
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:1344
load:0x40078000,len:13924
ho 0 tail 12 room 4
load:0x40080400,len:3600
entry 0x400805f0
Setting soft-AP configuration ...
Ready
Setting soft-AP ...
Ready
Soft-AP IP address = 192.168.1.100
MAC address = CC:DB:A7:4F:58:ED
Setup End
```

When observing the print information of the serial monitor, turn on the WiFi scanning function of your phone, and you can see the ssid\_AP on ESP32, which is called "WiFi\_Name" in this Sketch. You can enter the password "12345678" to connect it or change its AP name and password by modifying Sketch.



## Sketch\_04.2\_AP\_mode

The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_AP    = "WiFi_Name"; //Enter the router name
4  const char *password_AP = "12345678"; //Enter the router password
5
6  IPAddress local_IP(192, 168, 1, 100); //Set the IP address of ESP32 itself
7  IPAddress gateway(192, 168, 1, 10);   //Set the gateway of ESP32 itself
8  IPAddress subnet(255, 255, 255, 0);   //Set the subnet mask for ESP32 itself
9
10 void setup() {
11     Serial.begin(115200);
12     delay(2000);
13     Serial.println("Setting soft-AP configuration ... ");
14     WiFi.disconnect();
15     WiFi.mode(WIFI_AP);
16     Serial.println(WiFi.softAPConfig(local_IP, gateway, subnet) ? "Ready" : "Failed!");
17     Serial.println("Setting soft-AP ... ");
18     boolean result = WiFi.softAP(ssid_AP, password_AP);
19     if(result) {
20         Serial.println("Ready");
21         Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
22         Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
23     } else {
24         Serial.println("Failed!");
25     }
26     Serial.println("Setup End");
27 }
28
29 void loop() {
30 }

```

Include WiFi Library header file of ESP32.

```
1  #include <WiFi.h>
```

Enter correct AP name and password.

```

3  const char *ssid_AP    = "WiFi_Name"; //Enter the router name
4  const char *password_AP = "12345678"; //Enter the router password

```

Set ESP32 in AP mode.

```
15  WiFi.mode(WIFI_AP);
```

Configure IP address, gateway and subnet mask for ESP32.

```
16  WiFi.softAPConfig(local_IP, gateway, subnet)
```

Turn on an AP in ESP32, whose name is set by ssid\_AP and password is set by password\_AP.

```
18  WiFi.softAP(ssid_AP, password_AP);
```

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

Check whether the AP is turned on successfully. If yes, print out IP and MAC address of AP established by ESP32. If no, print out the failure prompt.

```

19  if(result) {
20      Serial.println("Ready");
21      Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
22      Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
23  } else {
24      Serial.println("Failed!");
25  }
26  Serial.println("Setup End");

```

### Reference

#### Class AP

Every time when using WiFi, you need to include header file "WiFi.h".

**softAP(ssid, password, channel, ssid\_hidden, max\_connection):**

**ssid:** WiFi hotspot name

**password:** WiFi hotspot password

**channel:** Number of WiFi connection channels, range 1-13. The default is 1.

**ssid\_hidden:** Whether to hide WiFi name from scanning by other devices. The default is not hide.

**max\_connection:** Maximum number of WiFi connected devices. The range is 1-4. The default is 4.

**softAPConfig(local\_ip, gateway, subnet):** set static local IP address.

**local\_ip:** station fixed IP address.

**Gateway:** gateway IP address

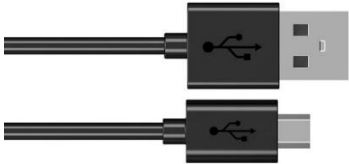
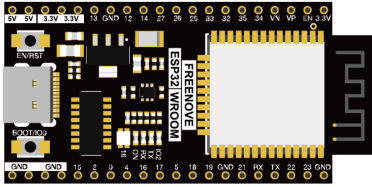
**subnet:** subnet mask

**softAP():** obtain IP address in AP mode

**softAPdisconnect ():** disconnect AP mode.

## Project 4.3 AP+Station mode

### Component List

|                                                                                                     |                                                                                                      |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Type C Wire x1<br> | ESP32-WROOM x1<br> |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|

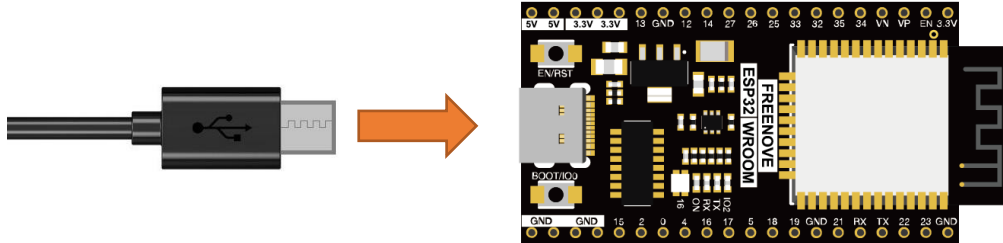
### Component knowledge

#### AP+Station mode

In addition to AP mode and station mode, ESP32 can also use AP mode and station mode at the same time. This mode contains the functions of the previous two modes. Turn on ESP32's station mode, connect it to the router network, and it can communicate with the Internet via the router. At the same time, turn on its AP mode to create a hotspot network. Other WiFi devices can choose to connect to the router network or the hotspot network to communicate with ESP32.

## Circuit

Connect Freenove ESP32 to the computer using the USB cable.



## Sketch

### Sketch\_04.3\_AP\_Station\_mode

Sketch\_04.3\_AP\_Station | Arduino IDE 2.1.0

File Edit Sketch Tools Help

ESP32 Dev Module

Sketch\_04.3\_AP\_Station.ino

```

7  #include <WiFi.h>
8
9  const char *ssid_Router = "*****"; //Enter the router name
10 const char *password_Router = "*****"; //Enter the router password
11 const char *ssid_AP = "WiFi_Name"; //Enter the router name
12 const char *password_AP = "12345678"; //Enter the router password
13
14 void setup(){
15   Serial.begin(115200);
16   Serial.println("Setting soft-AP configuration ... ");
17   WiFi.disconnect();
18   WiFi.mode(WIFI_AP);
19   Serial.println("Setting soft-AP ... ");
20   boolean result = WiFi.softAP(ssid_AP, password_AP);
21   if(result){
22     Serial.println("Ready");
23     Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
24     Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().toString());

```

Please enter the correct names and passwords of Router and AP.

Output

```

Writing at 0x00055138... (37 %)
Writing at 0x0005a28a... (41 %)
Writing at 0x0005f766... (44 %)
Writing at 0x00064b45... (48 %)
Writing at 0x0006a9ad... (51 %)
Writing at 0x0006fe5e... (55 %)
Writing at 0x000750af... (58 %)
Writing at 0x0007a4aa... (62 %)
Writing at 0x0007f8b7... (65 %)

```

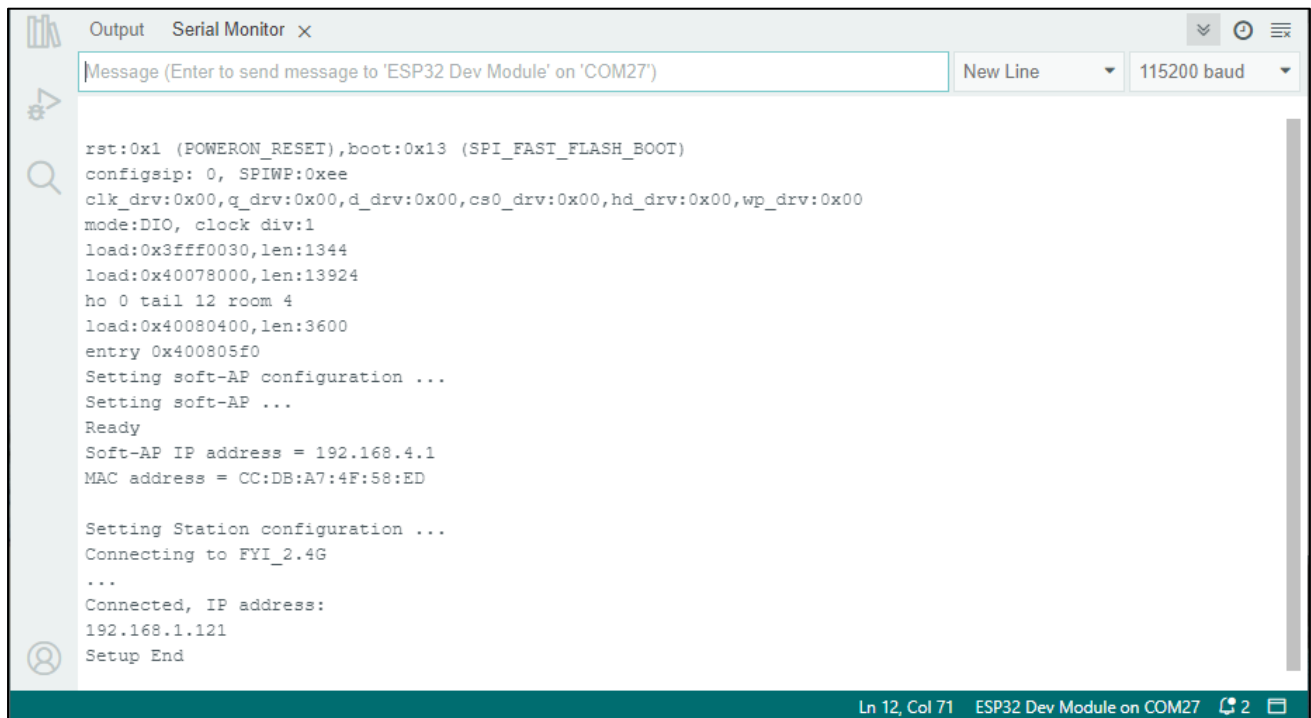
Uploading...

Ln 12, Col 71 ESP32 Dev Module on COM27

It is analogous to Project 4.1 and Project 4.2. Before running the Sketch, you need to modify `ssid_Router`, `password_Router`, `ssid_AP` and `password_AP` shown in the box of the illustration above.



After making sure that Sketch is modified correctly, compile and upload codes to ESP32-WROOM, open serial monitor and set baud rate to 115200. And then it will display as follows:

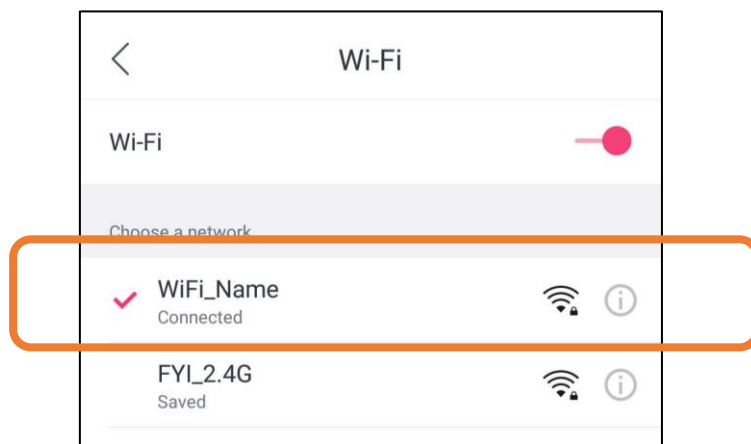


```

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:1344
load:0x40078000,len:13924
ho 0 tail 12 room 4
load:0x40080400,len:3600
entry 0x400805f0
Setting soft-AP configuration ...
Setting soft-AP ...
Ready
Soft-AP IP address = 192.168.4.1
MAC address = CC:DB:A7:4F:58:ED

Setting Station configuration ...
Connecting to FYI_2.4G
...
Connected, IP address:
192.168.1.121
Setup End
  
```

When observing the print information of the serial monitor, turn on the WiFi scanning function of your phone, and you can see the ssid\_AP on ESP32.



The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  const char *ssid_AP       = "WiFi_Name"; //Enter the AP name
6  const char *password_AP   = "12345678"; //Enter the AP password
7
8  void setup() {
9      Serial.begin(115200);
  
```

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

```
10 Serial.println("Setting soft-AP configuration ... ");
11 WiFi.disconnect();
12 WiFi.mode(WIFI_AP);
13 Serial.println("Setting soft-AP ... ");
14 boolean result = WiFi.softAP(ssid_AP, password_AP);
15 if(result){
16     Serial.println("Ready");
17     Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
18     Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
19 }else{
20     Serial.println("Failed!");
21 }
22
23 Serial.println("\nSetting Station configuration ... ");
24 WiFi.begin(ssid_Router, password_Router);
25 Serial.println(String("Connecting to ") + ssid_Router);
26 while (WiFi.status() != WL_CONNECTED) {
27     delay(500);
28     Serial.print(".");
29 }
30 Serial.println("\nConnected, IP address: ");
31 Serial.println(WiFi.localIP());
32 Serial.println("Setup End");
33 }
34
35 void loop() {
36 }
```

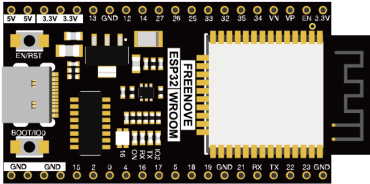
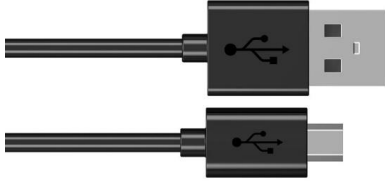
## Chapter 5 TCP/IP

In this chapter, we will introduce how ESP32 implements network communications based on TCP/IP protocol. There are two roles in TCP/IP communication, namely Server and Client, which will be implemented respectively with two projects in this chapter.

### Project 5.1 As Client

In this section, ESP32 is used as Client to connect Server on the same LAN and communicate with it.

### Component List

|                                                                                                      |                                                                                                       |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| ESP32-WROOM x1<br> | Type C Wire x1<br> |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|

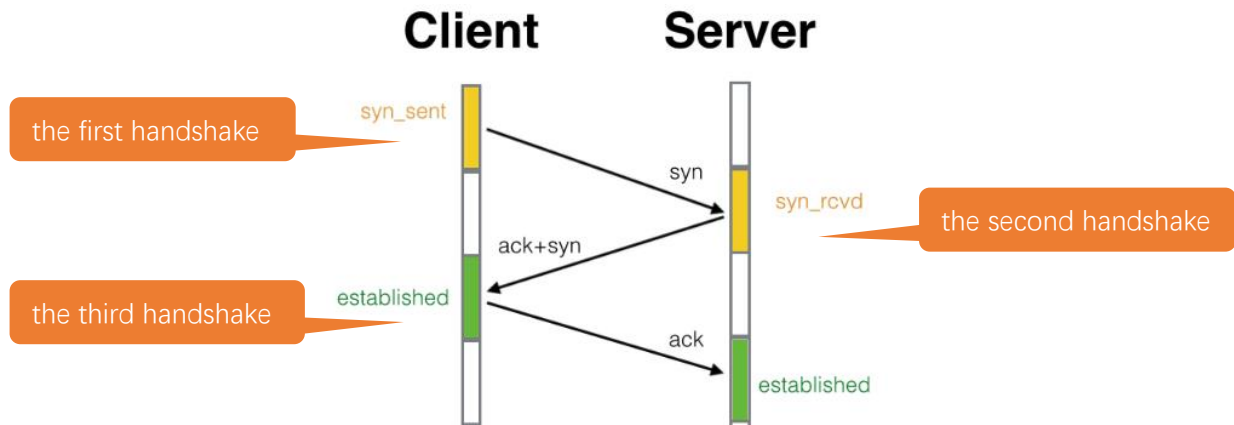
### Component knowledge

#### TCP connection

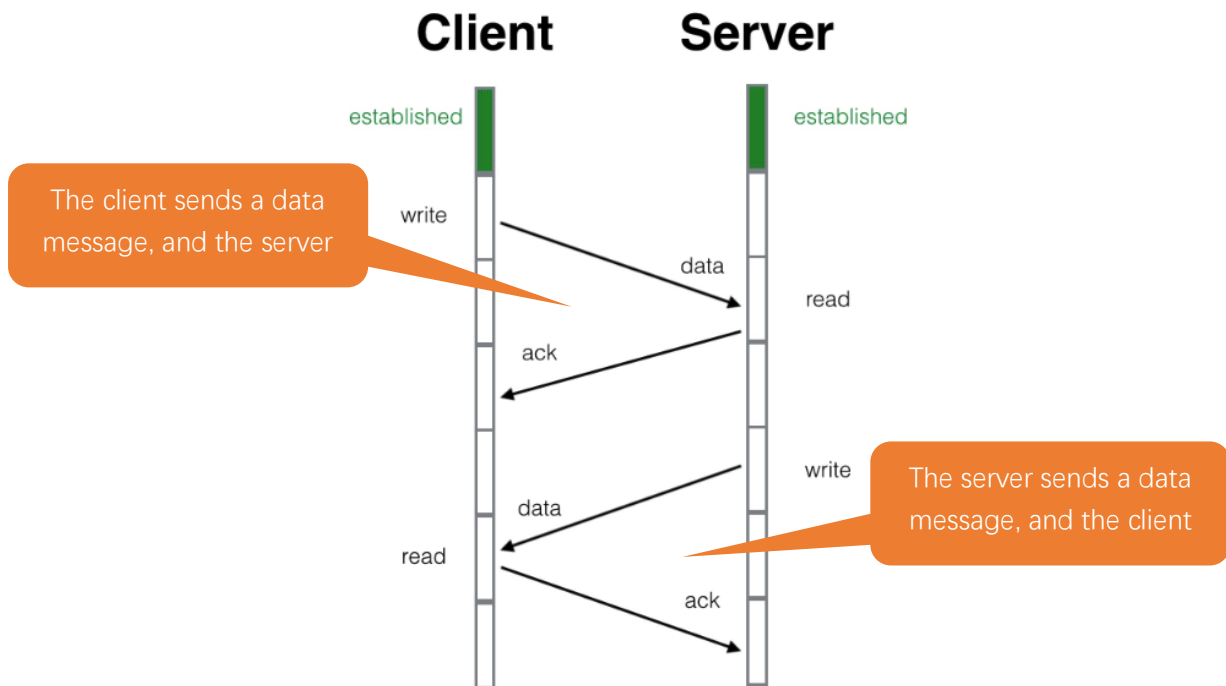
Before transmitting data, TCP needs to establish a logical connection between the sending end and the receiving end. It provides reliable and error-free data transmission between the two computers. In the TCP connection, the client and the server must be clarified. The client sends a connection request to the server, and each time such a request is proposed, a "three-times handshake" is required.

Three-times handshake: In the TCP protocol, during the preparation phase of sending data, the client and the server interact three times to ensure the reliability of the connection, which is called "three-times handshake". The first handshake, the client sends a connection request to the server and waits for the server to confirm. The second handshake, the server sends a response back to the client informing that it has received the connection request.

The third handshake, the client sends a confirmation message to the server again to confirm the connection.



TCP is a connection-oriented, low-level transmission control protocol. After TCP establishes a connection, the client and server can send and receive messages to each other, and the connection will always exist as long as the client or server does not initiate disconnection. Each time one party sends a message, the other party will reply with an ack signal.



## Install Processing

In this tutorial, we use Processing to build a simple TCP/IP communication platform.

If you've not installed Processing, you can download it by clicking <https://processing.org/download/>. You can choose an appropriate version to download according to your PC system.

**Processing** p5.js Processing.py Processing for Android Processing for Pi Processing Foundation

# Processing

Download Processing. Processing is available for Linux, Mac OS X, and Windows. Select your choice to download the software below.

3.5.4 (17 January 2020)

**Windows** 64-bit **Linux** 64-bit **Mac OS X**

**Windows** 32-bit

» [Github](#) » [Report Bugs](#) » [Wiki](#) » [Supported Platforms](#)

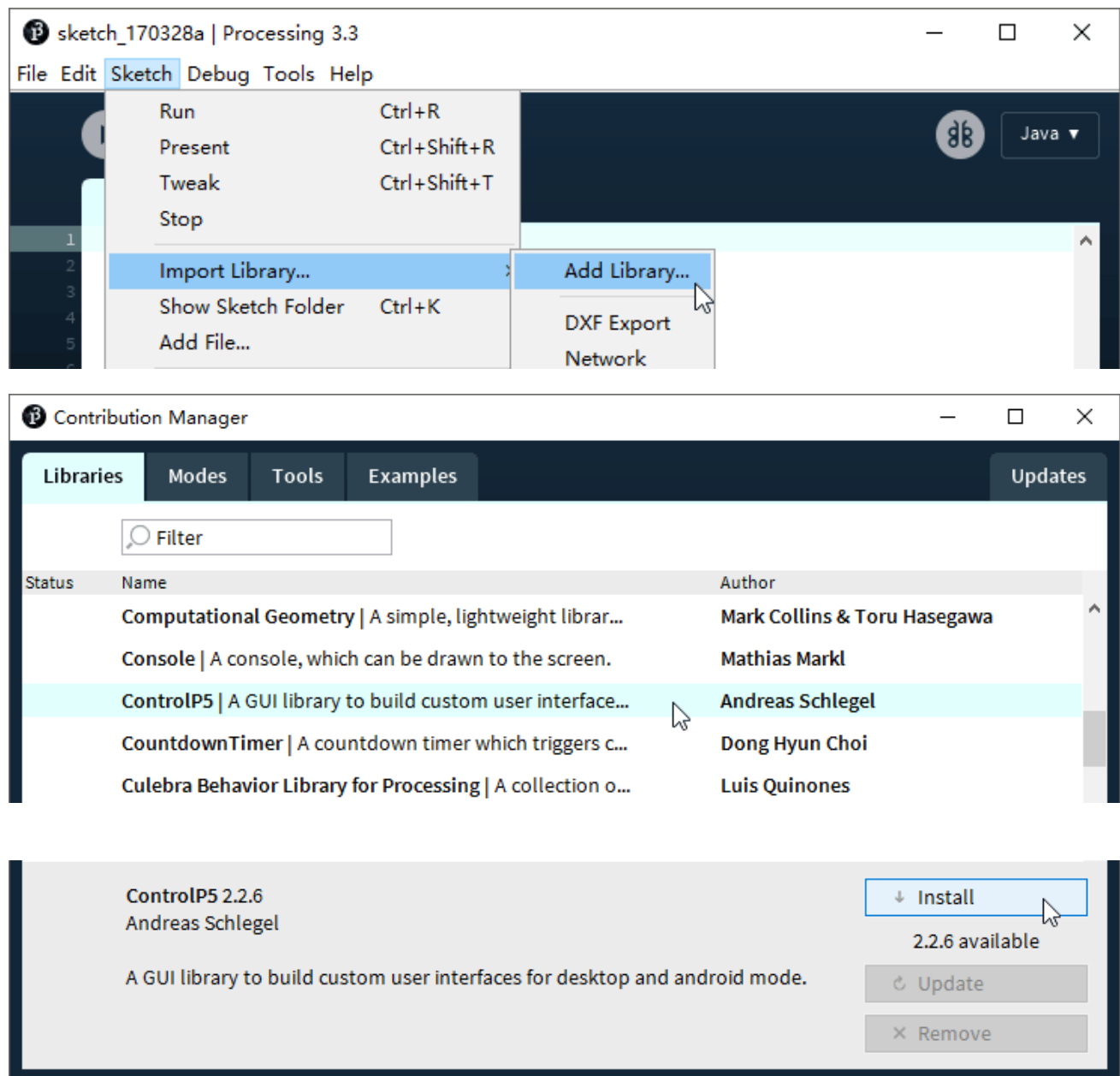
Read about the [changes in 3.0](#). The [list of revisions](#) covers the differences between releases in detail.

Unzip the downloaded file to your computer. Click "processing.exe" as the figure below to run this software.

|                     |                 |
|---------------------|-----------------|
| core                | 2020/1/17 12:16 |
| java                | 2020/1/17 12:17 |
| lib                 | 2020/1/17 12:16 |
| modes               | 2020/1/17 12:16 |
| tools               | 2020/1/17 12:16 |
| processing.exe      | 2020/1/17 12:16 |
| processing-java.exe | 2020/1/17 12:16 |
| revisions.txt       | 2020/1/17 12:16 |

Use Server mode for communication

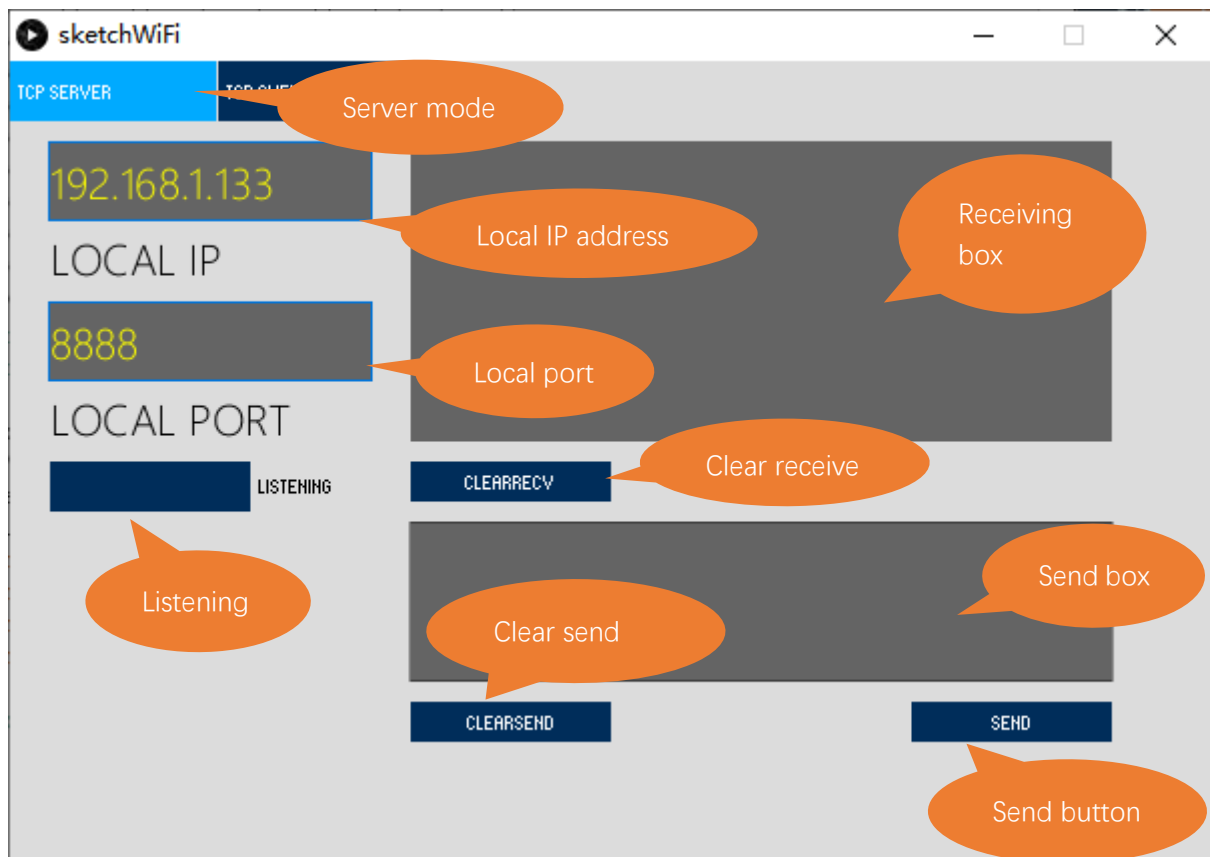
Install ControlP5.



Open the "Freenove\_ESP32\_WROOM\_Board\Sketches\Sketches\Sketch\_05.1\_WiFiClient\sketchWiFi\sketchWiFi.pde", and click "Run".

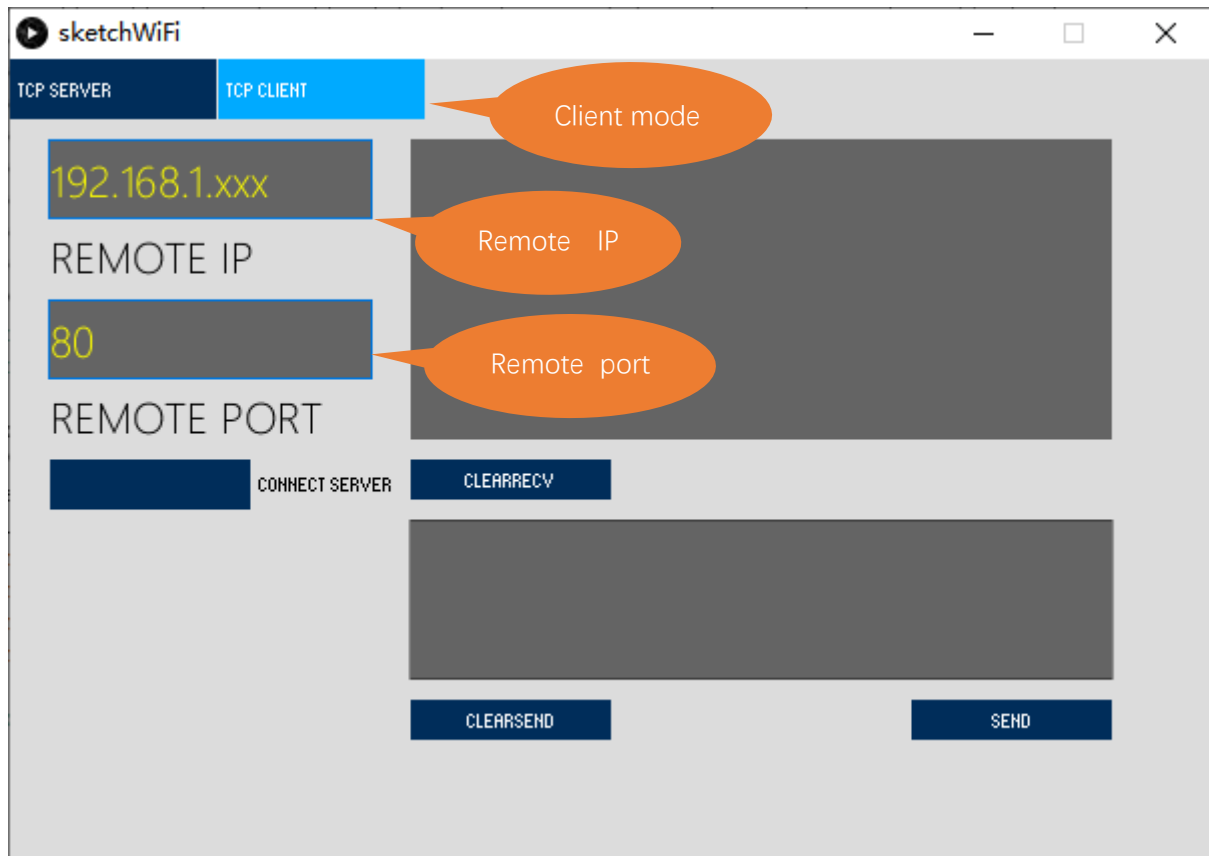


The new pop-up interface is as follows. If ESP32 is used as client, select TCP SERVER mode for sketchWiFi.



When sketchWiFi selects TCP SERVER mode, ESP32 Sketch needs to be changed according to sketchWiFi's displaying of LOCAL IP or LOCAL PORT.

If ESP32 serves as server, select TCP CLIENT mode for sketchWiFi.



When sketchWiFi selects TCP CLIENT mode, the LOCAL IP and LOCAL PORT of sketchWiFi need to be changed according to the IP address and port number printed by the serial monitor.

**Mode selection:** select **Server mode/Client mode**.

**IP address:** In server mode, this option does not need to be filled in, and the computer will automatically obtain the IP address.

In client mode, fill in the remote IP address to be connected.

**Port number:** In server mode, fill in a port number for client devices to make an access connection.

In client mode, fill in port number given by the Server devices to make an access connection.

**Start button:** In server mode, push the button, then the computer will serve as server and open a port number for client to make access connection. During this period, the computer will keep monitoring.

In client mode, before pushing the button, please make sure the server is on, remote IP address and remote port number is correct; push the button, and the computer will make access connection to the remote port number of the remote IP as a client.

**clear receive:** clear out the content in the receiving text box

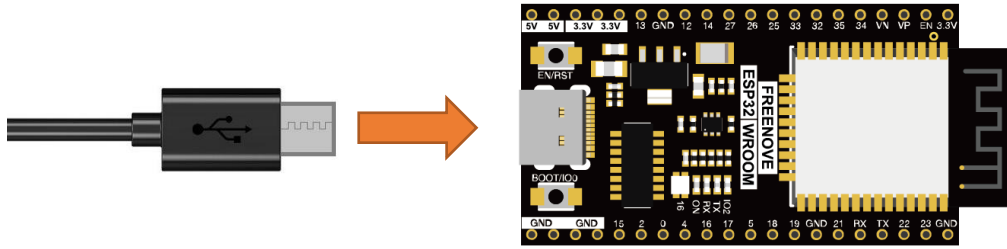
**clear send:** clear out the content in the sending text box

**Sending button:** push the sending button, the computer will send the content in the text box to others.

## Circuit

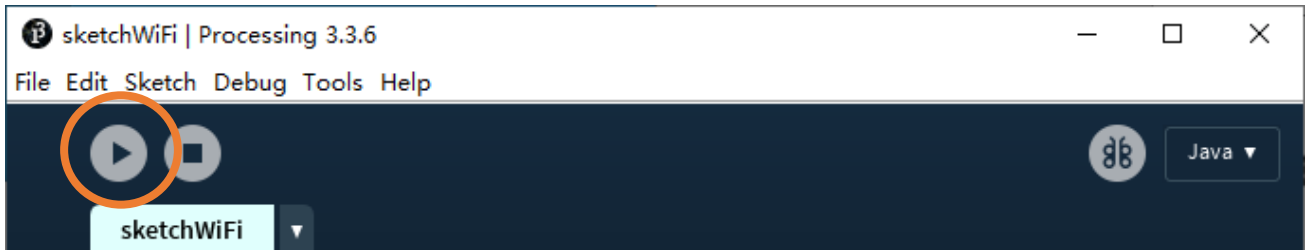
Connect Freenove ESP32 to the computer using USB cable.



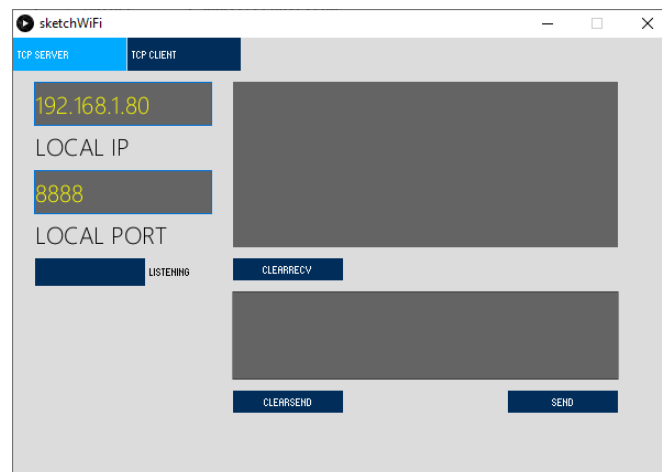


## Sketch

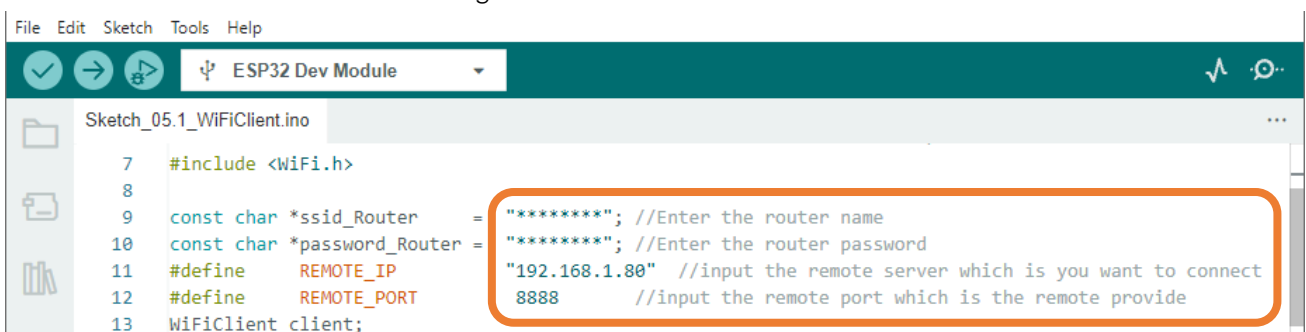
Before running the Sketch, please open "sketchWiFi.pde." first, and click "Run".



The newly pop up window will use the computer's IP address by default and open a data monitor port.

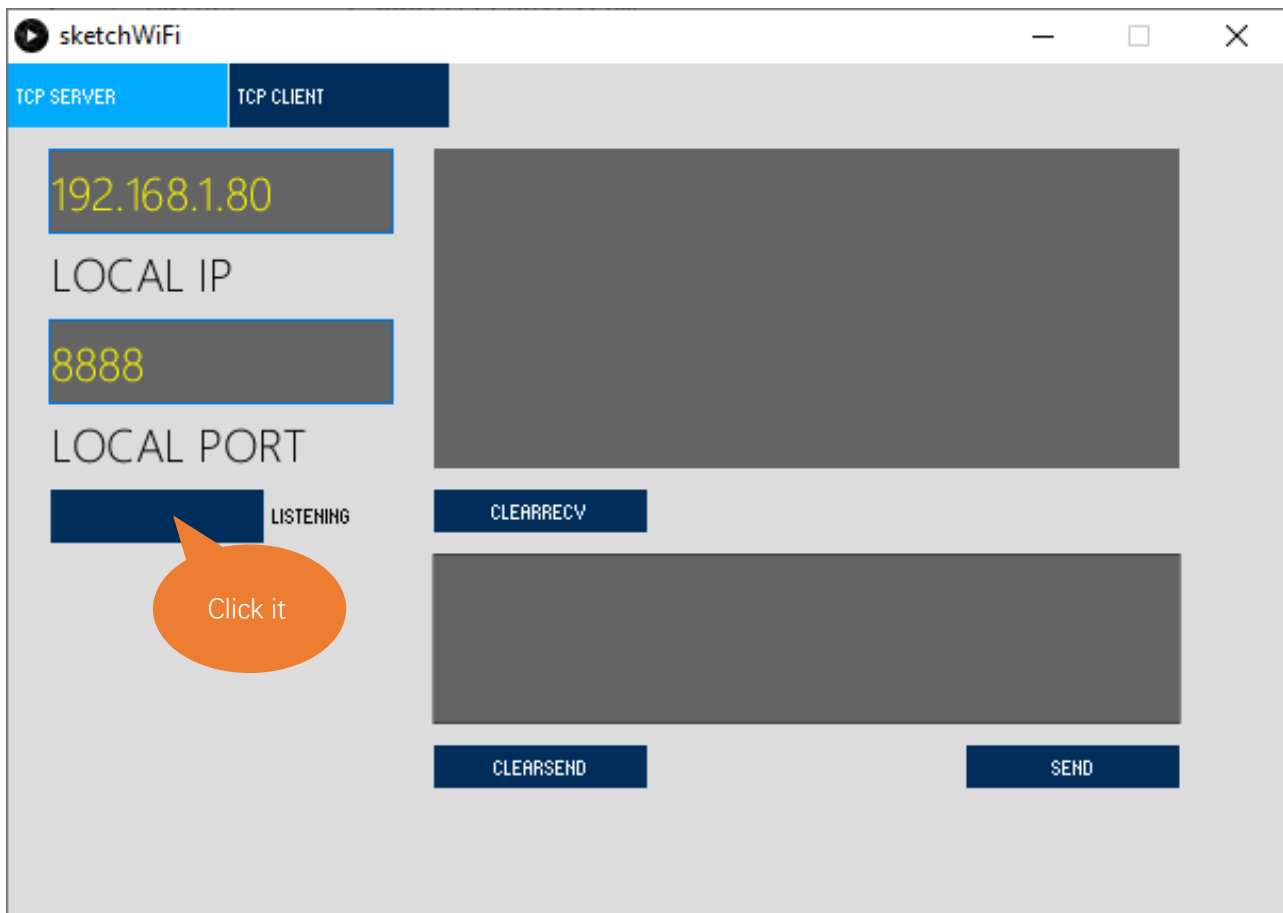


Next, open Sketch\_05.1\_WiFiClient.ino. Before running it, please change the following information based on "LOCAL IP" and "LOCAL PORT" in the figure above.

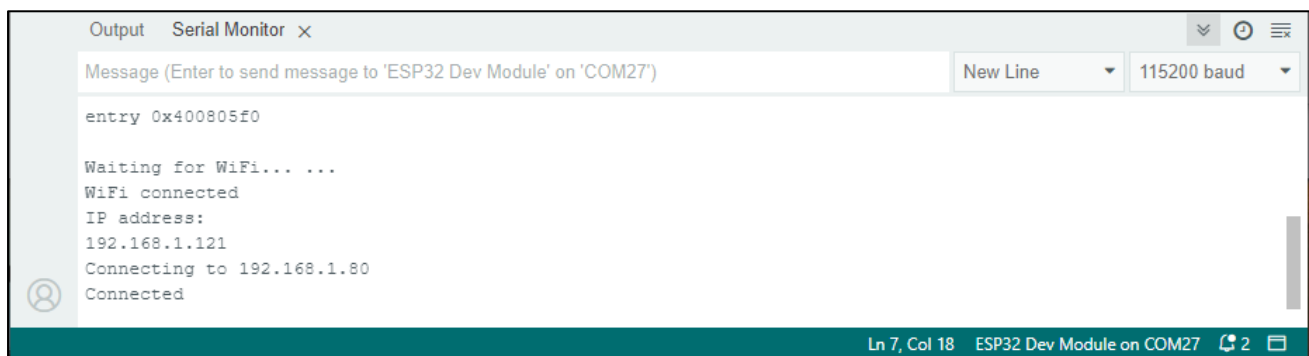


REMOTE\_IP needs to be filled in according to the interface of sketchWiFi.pde. Taking this tutorial as an example, its REMOTE\_IP is "192.168.1.80". Generally, by default, the ports do not need to change its value.

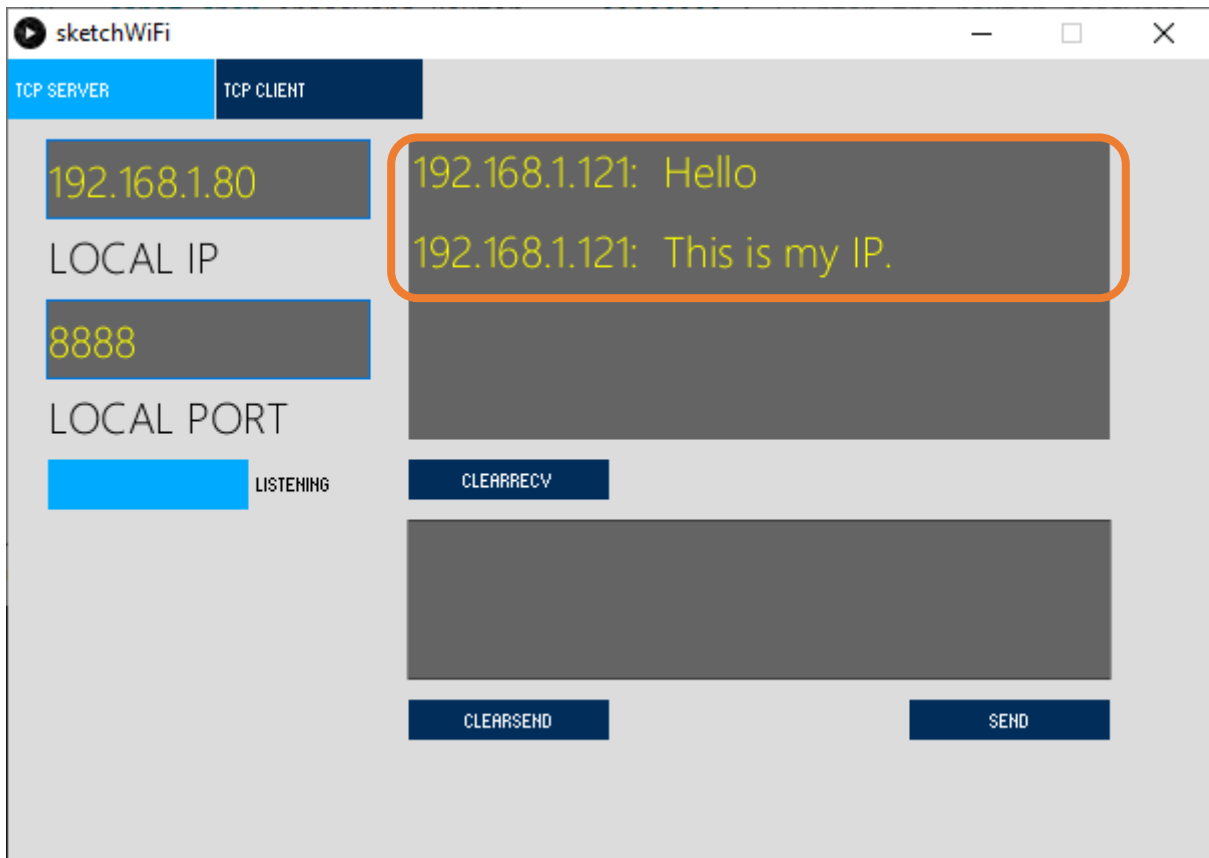
Click LISTENING, turn on TCP SERVER's data listening function and wait for ESP32 to connect.



Compile and upload code to ESP32-WROOM, open the serial monitor and set the baud rate to 115200. ESP32 connects router, obtains IP address and sends access request to server IP address on the same LAN till the connection is successful. When connect successfully, ESP32 can send messages to server.



ESP32 connects with TCP SERVER, and TCP SERVER receives messages from ESP32, as shown in the figure below.



## Sketch\_05.1\_As\_Client

The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  #define     REMOTE_IP       "*****" //input the remote server which is you want to connect
6  #define     REMOTE_PORT     8888     //input the remote port which is the remote provide
7  WiFiClient client;
8
9  void setup() {
10     Serial.begin(115200);
11     delay(10);
12
13     WiFi.begin(ssid_Router, password_Router);
14     Serial.print("\nWaiting for WiFi... ");
15     while (WiFi.status() != WL_CONNECTED) {
16         Serial.print(".");
17         delay(500);
18     }
19     Serial.println("");
20     Serial.println("WiFi connected");
21     Serial.println("IP address: ");
22     Serial.println(WiFi.localIP());
23     delay(500);
24
25     Serial.print("Connecting to ");
26     Serial.println(REMOTE_IP);
27
28     while (!client.connect(REMOTE_IP, REMOTE_PORT)) {
29         Serial.println("Connection failed.");
30         Serial.println("Waiting a moment before retrying...");
31     }
32     Serial.println("Connected");
33     client.print("Hello\n");
34     client.print("This is my IP.\n");
35
36     void loop() {
37         if (client.available() > 0) {
38             delay(20);
39             //read back one line from the server
40             String line = client.readString();
41             Serial.println(REMOTE_IP + String(":") + line);

```

```

42 }
43 if (Serial.available() > 0) {
44     delay(20);
45     String line = Serial.readString();
46     client.print(line);
47 }
48 if (client.connected () == 0) {
49     client.stop();
50     WiFi.disconnect();
51 }
52 }

```

Add WiFi function header file.

```

1 #include <WiFi.h>

```

Enter the actual router name, password, remote server IP address, and port number.

```

3 const char *ssid_Router      = "*****"; //Enter the router name
4 const char *password_Router = "*****"; //Enter the router password
5 #define     REMOTE_IP        "*****" //input the remote server which is you want to connect
6 #define     REMOTE_PORT      8888     //input the remote port which is the remote provide

```

Apply for the method class of WiFiClient.

```

7 WiFiClient client;

```

Connect specified WiFi until it is successful. If the name and password of WiFi are correct but it still fails to connect, please push the reset key.

```

13 WiFi.begin(ssid_Router, password_Router);
14 Serial.print("\nWaiting for WiFi... ");
15 while (WiFi.status() != WL_CONNECTED) {
16     Serial.print(".");
17     delay(500);
18 }

```

Send connection request to remote server until connect successfully. When connect successfully, print out the connecting prompt on the serial monitor and send messages to remote server.

```

28 while (!client.connect(REMOTE_IP, REMOTE_PORT)) { //Connect to Server
29     Serial.println("Connection failed.");
30     Serial.println("Waiting a moment before retrying...");
31 }
32 Serial.println("Connected");
33 client.print("Hello\n");

```

When ESP32 receive messages from servers, it will print them out via serial port; Users can also send messages to servers from serial port.

```

37 if (client.available() > 0) {
38     delay(20);
39     //read back one line from the server
40     String line = client.readString();
41     Serial.println(REMOTE_IP + String(":") + line);
42 }

```

```

43     if (Serial.available() > 0) {
44         delay(20);
45         String line = Serial.readString();
46         client.print(line);
47     }

```

If the server is disconnected, turn off WiFi of ESP32.

```

48     if (client.connected () == false) {
49         client.stop();
50         WiFi.disconnect();
51     }

```

### Reference

#### Class Client

Every time when using Client, you need to include header file "WiFi.h."

**connect(ip, port, timeout)/connect(\*host, port, timeout):** establish a TCP connection.

**ip, \*host:** ip address of target server

**port:** port number of target server

**timeout:** connection timeout

**connected():** judge whether client is connecting. If return value is 1, then connect successfully; If return value is 0, then fail to connect.

**stop():** stop tcp connection

**print():** send data to server connecting to client

**available():** return to the number of bytes readable in receive buffer, if no, return to 0 or -1.

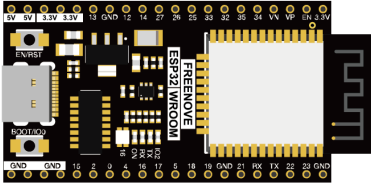
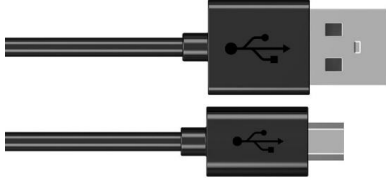
**read():** read one byte of data in receive buffer

**readString():** read string in receive buffer

## Project 5.2 As Server

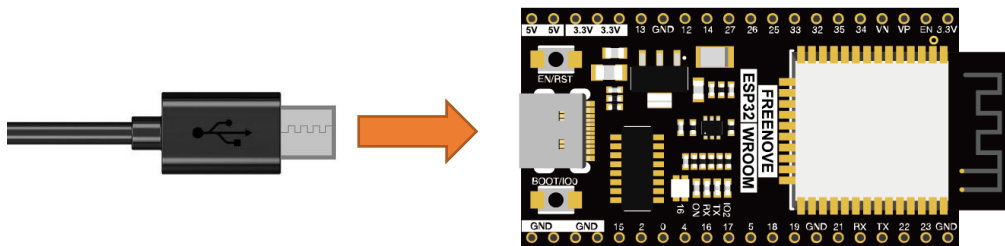
In this section, ESP32 is used as a server to wait for the connection and communication of client on the same LAN.

### Component List

|                                                                                                     |                                                                                                      |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| ESP32-WROOM x1<br> | Type C Wire x1<br> |
|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|

### Circuit

Connect Freenove ESP32 to the computer using a USB cable.



Before running Sketch, please modify the contents of the box below first.

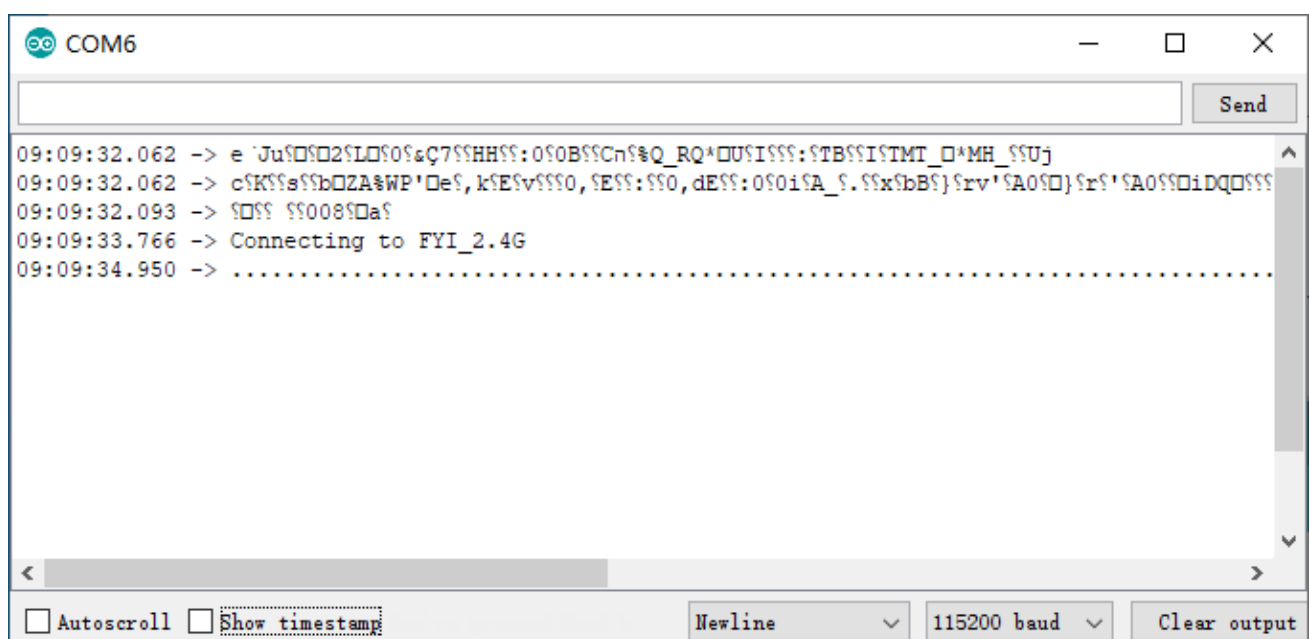


The screenshot shows the Arduino IDE interface. The title bar reads "Sketch\_05\_2\_WiFiServer | Arduino IDE 2.1.0". The menu bar includes "File", "Edit", "Sketch", "Tools", and "Help". The toolbar shows icons for checking, running, and uploading, along with a dropdown menu set to "ESP32 Dev Module". The code editor displays the file "Sketch\_05\_2\_WiFiServer.ino" with the following code:

```
8  #include <WiFi.h>
9
10 #define port 80
11 const char *ssid_Router = "*****"; //input your wifi name
12 const char *password_Router = "*****"; //input your wifi passwords
13 WiFiServer server(port);
14
```

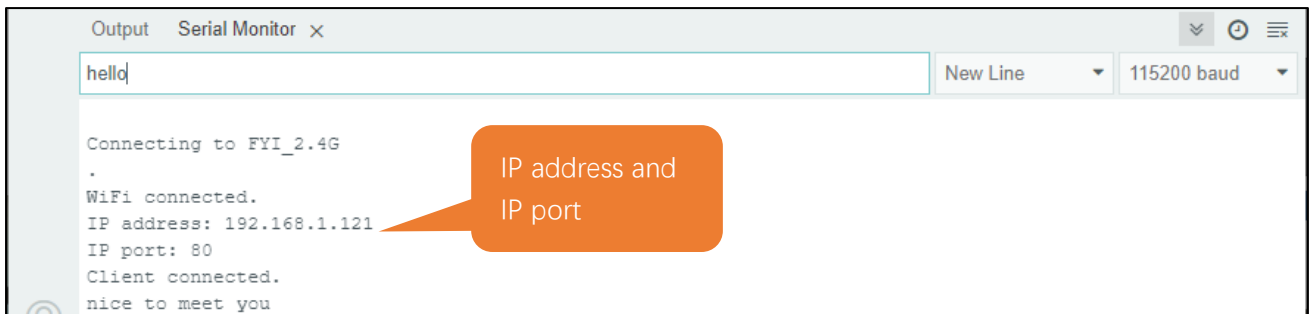
The code is enclosed in an orange rounded rectangle.

If the ESP32 fails to connect to router, press the reset button as shown below and wait for ESP32 to run again.





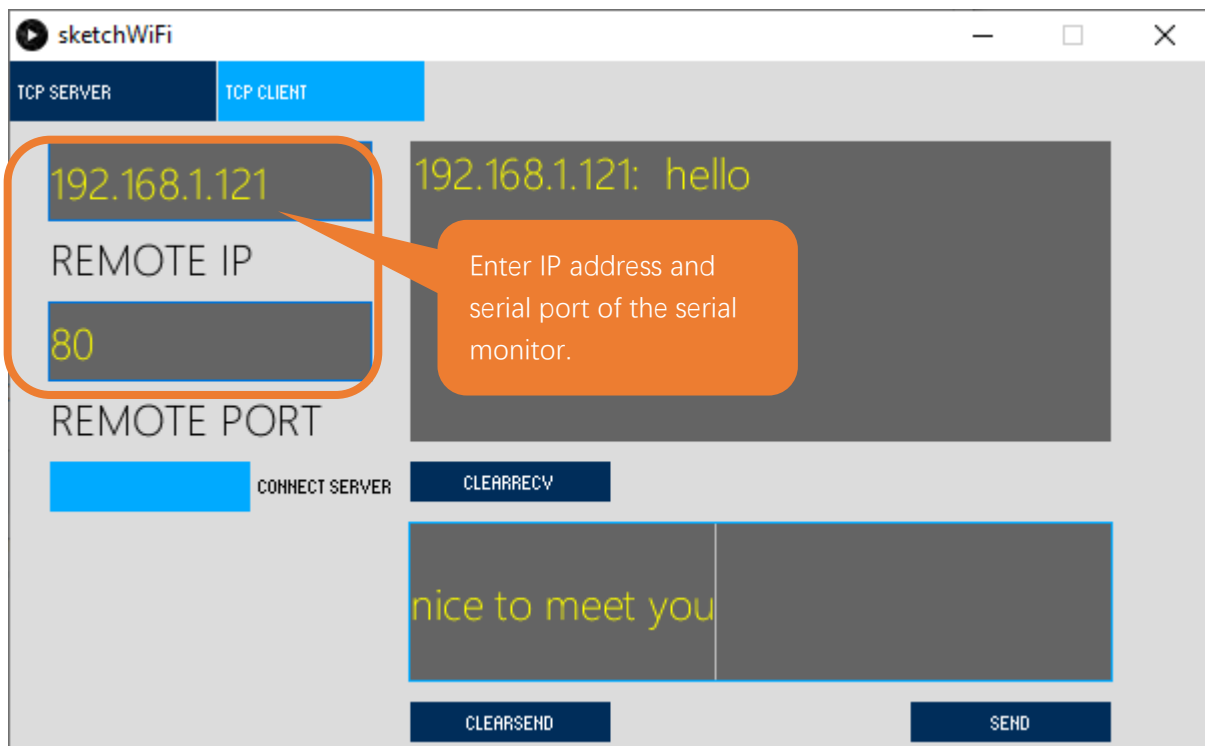
## Serial Monitor



Processing:

Open the “Freenove\_ESP32\_WROOM\_Board\Sketches\Sketches\Sketch\_05.2\_WiFiServer\sketchWiFi\sketchWiFi.pde”.

Based on the messages printed by the serial monitor, enter correct IP address and IP port in Processing to establish connection and make communication.



The following is the program code:

```

1  #include <WiFi.h>
2
3  #define port 80
4  const char *ssid_Router    = "*****"; //input your wifi name
5  const char *password_Router = "*****"; //input your wifi passwords
6  WiFiServer server(port);
7
8  void setup()
9  {
10     Serial.begin(115200);

```

Any concerns? [✉ support@freenove.com](mailto:support@freenove.com)

```

11 Serial.printf("\nConnecting to ");
12 Serial.println(ssid_Router);
13 WiFi.disconnect();
14 WiFi.begin(ssid_Router, password_Router);
15 delay(1000);
16 while (WiFi.status() != WL_CONNECTED) {
17     delay(500);
18     Serial.print(".");
19 }
20 Serial.println("");
21 Serial.println("WiFi connected.");
22 Serial.print("IP address: ");
23 Serial.println(WiFi.localIP());
24 Serial.printf("IP port: %d\n",port);
25 server.begin(port);
26 WiFi.setAutoConnect(true);
27 WiFi.setAutoReconnect(true);
28 }
29
30 void loop() {
31     WiFiClient client = server.accept();           // listen for incoming clients
32     if (client) {                                   // if you get a client
33         Serial.println("Client connected.");
34         while (client.connected()) {                // loop while the client's connected
35             if (client.available()) {               // if there's bytes to read from the
client
36                 Serial.println(client.readStringUntil('\n')); // print it out the serial monitor
37                 while(client.read()>0);             // clear the wifi receive area cache
38             }
39             if(Serial.available()){                 // if there's bytes to read from the
serial monitor
40                 client.print(Serial.readStringUntil('\n')); // print it out the client.
41                 while(Serial.read()>0);             // clear the wifi receive area cache
42             }
43         }
44         client.stop();                               // stop the client connecting.
45         Serial.println("Client Disconnected.");
46     }
47 }

```

Apply for method class of WiFiServer.

```

6 WiFiServer server(port);           //Apply for a Server object whose port number is 80

```

Connect specified WiFi until it is successful. If the name and password of WiFi are correct but it still fails to connect, please push the reset key.

```

13 WiFi.disconnect();

```

```

14  WiFi.begin(ssid_Router, password_Router);
15  delay(1000);
16  while (WiFi.status() != WL_CONNECTED) {
17      delay(500);
18      Serial.print(".");
19  }
20  Serial.println("");
21  Serial.println("WiFi connected.");

```

Print out the IP address and port number of ESP32.

```

22  Serial.print("IP address: ");
23  Serial.println(WiFi.localIP());           //print out IP address of ESP32
24  Serial.printf("IP port: %d\n", port);     //Print out ESP32's port number

```

Turn on server mode of ESP32, start automatic connection and turn on automatic reconnection.

```

25  server.begin();                          //Turn ON ESP32 as Server mode
26  WiFi.setAutoConnect(true);
27  WiFi.setAutoReconnect(true);

```

When ESP32 receive messages from servers, it will print them out via serial port; Users can also send messages to servers from serial port.

```

35  if (client.available()) {                // if there's bytes to read from the
client
36      Serial.println(client.readStringUntil('\n')); // print it out the serial monitor
37      while(client.read()>0);              // clear the wifi receive area cache
38  }
39  if(Serial.available()){                  // if there's bytes to read from the
serial monitor
40      client.print(Serial.readStringUntil('\n')); // print it out the client.
41      while(Serial.read()>0);              // clear the wifi receive area cache
42  }

```

## Reference

### Class Server

Every time use Server functionality, we need to include header file "WiFi.h".

**WiFiServer(uint16\_t port=80, uint8\_t max\_clients=4):** create a TCP Server.

**port:** ports of Server; range from 0 to 65535 with the default number as 80.

**max\_clients:** maximum number of clients with default number as 4.

**begin(port):** start the TCP Server.

**port:** ports of Server; range from 0 to 65535 with the default number as 0.

**setNoDelay(bool nodelay):** whether to turn off the delay sending functionality.

**nodelay:** true stands for forbidden Nagle algorithm.

**close():** close tcp connection.

**stop():** stop tcp connection.

## What's next?

Thanks for your reading. This tutorial is all over here. If you find any mistakes, omissions or you have other ideas and questions about contents of this tutorial or the kit and etc., please feel free to contact us:

[support@freenove.com](mailto:support@freenove.com)

We will check and correct it as soon as possible.

If you want learn more about ESP32, you view our ultimate tutorial:

[https://github.com/Freenove/Freenove\\_ESP32\\_WROOM\\_Board/archive/master.zip](https://github.com/Freenove/Freenove_ESP32_WROOM_Board/archive/master.zip)

If you want to learn more about Arduino, Raspberry Pi, smart cars, robots and other interesting products in science and technology, please continue to focus on our website. We will continue to launch cost-effective, innovative and exciting products.

<http://www.freenove.com/>

## End of the Tutorial

Thank you again for choosing Freenove products.